

EXEMPLAR DE PREZENTARE

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

Informatică

Curriculum
de specialitate (CS),
filiera teoretică, profilul
real, specializarea științe
ale naturii – clasa a IX-a

9

CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

CORINT
LOGISTIC

Informatică

Curriculum
de specialitate (CS),
filiera teoretică, profilul
real, specializarea științe
ale naturii – clasa a IX-a

9

CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

Date despre autori:

CEZARA BENEGUI – lector universitar doctor la Facultatea de Matematică și Informatică a Universității din București, doctor în Informatică, cu activitate de cercetare în inteligența artificială și publicații științifice naționale și internaționale. A primit titlul de Profesor Bologna, distincție acordată de Alianța Națională a Organizațiilor Studențești din România (ANOSR) cadrelor didactice apreciate de studenți. Parcursul profesional reunește experiența academică și cea din industria IT, inclusiv în dezvoltare software. Este expertă în baze de date și dezvoltare software, discipline pe care le predă de peste un deceniu la Facultatea de Matematică și Informatică a Universității din București. A primit Premiul Senatului Universității din București pentru cel mai inovator curs și cea mai inovatoare metodă de predare, în cadrul disciplinei Dezvoltarea Aplicațiilor Web. În paralel, activează și în domeniul consilierii pentru dezvoltare personală și orientare în carieră, sprijinind tinerii să își clarifice direcția și să își construiască un parcurs autentic, în acord cu propriile valori. Crede într-o educație care depășește transmiterea de informații și creează un spațiu în care fiecare tânăr se simte văzut, înțeles și încurajat să-și descopere potențialul. Promovează o educație care formează nu doar competențe tehnice, ci și curaj, responsabilitate și autonomie.

ANA-CRISTINA IOVA – lector universitar doctor la Universitatea din București și cercetător în domeniul informaticii. Activitatea sa îmbină pasiunea pentru educație cu cercetarea în programare funcțională, algoritmi evolutivi și testare software. Activitatea sa de cercetare s-a concretizat în publicarea unor lucrări științifice în reviste și volume de specialitate naționale și internaționale. Teza sa de doctorat a fost distinsă cu Premiul Senatului Universității din București pentru cea mai bună teză în domeniul științelor exacte. Prin acest manual contribuie la dezvoltarea competențelor digitale, rezolvare de probleme și utilizare responsabilă a tehnologiei.

RODICA SMÎNTÎNĂ – profesor gradul I, titular la Colegiul Național „Gheorghe Șincai” din București și are o experiență de peste 35 de ani în învățământul preuniversitar. Parcursul său profesional este completat de 15 ani în calitate de metodist al ISMB, 17 ani de coordonare a practicii pedagogice a studenților Facultății de Matematică și Informatică, Universitatea din București și formator național pe programele oferite de Oracle Academy. Din anul 2019 este membru în cadrul Comisiei de asigurare a omologării mijloacelor de învățământ în vederea utilizării lor în sistemul de învățământ preuniversitar. Coautoare la următoarele culegeri: „Teste admitere Universitatea Politehnică București” (2019), „Informatică – breviar teoretic, exerciții și teste de evaluare pentru bacalaureat, modele pentru teză” (2015), „Informatică Atestat – subiecte rezolvate” (2012, 2015, 2020), „Competențe digitale – bacalaureat – 50 de variante pentru proba de competențe digitale” (2015).

ANDREI PĂUN – profesor al Universității din București, a fost directorul școlii doctorale de Informatică a UB (2019-2024), iar din 2024 a acceptat poziția de director al Institutului de Cercetare pentru Inteligență Artificială „Mihai Drăgănescu” al Academiei Române. A predat cursuri de programare POO în calitate de lector/conferențiar universitar la Louisiana Tech University, SUA. De asemenea, predă cursul de Programare Orientată pe Obiecte la Facultatea de Matematică și Informatică a Universității din București de peste 15 ani. În anul 2011 a primit titlul de Profesor Bologna din partea asociațiilor studențești (ANOSR), la propunerea studenților de la cursul de Programare Orientată pe Obiecte (la acel moment 2 profesori din FMI dețineau această distincție). Din 2021, acesta este expert permanent în comisia C1 (Științe exacte și științele naturii) a ARACIS.

CIPRIAN-MIHAI CEAUȘESCU – asistent universitar doctorand (din anul 2023) la Facultatea de Matematică și Informatică a Universității din București, cu activitate de cercetare în domeniul vederii artificiale aplicată în domeniul medical. Este câștigătorul unei burse ANIS pentru Laboratorul de „Învățare Automată” (<https://unibuc.ro/cadre-didactice-si-doctoranzi-universitatii-din-bucuresti-printr-castigatorii-burselor-anis-premiile-acordate-in-cadrul-galei-bursele-anis-joi-7-noiembrie-2024/>). A predat cursuri de informatică și robotică în sistem Cambridge (IGCSE) la Cambridge School of Bucharest (2020-2023) având și diploma de QTS (profesor în sistem britanic din noiembrie 2021). Îmbină activitatea educațională cu cercetarea în domeniul inteligenței artificiale, participând constant la conferințe internaționale și proiecte care susțin dezvoltarea tehnologică și formarea noilor generații (<https://sites.google.com/view/ciprian-ceausescu/activity>).

PAUL ENACHE – profesor gradul I la Liceul Teoretic „Jean Monnet” din București. Având o dublă perspectivă – 20 de ani la catedră și 15 ani în industria IT ca programator/software developer –, aduce în sala de clasă conexiunea directă dintre teorie și tehnologia reală. Dedicat performanței, a ghidat generații de elevi în centre de excelență renumite, precum „Hai la Olimpiadă” și „Nerdvana/Nerdvana 2.0”, și a mentorat viitorii profesori prin coordonarea practicii pedagogice în parteneriat cu POLITEHNICA București. Implicat activ în comisiile de specialitate municipale și naționale, autorul este un promotor al modernizării educației.

Cuprins

CAPITOLUL 1

Strategii de rezolvare a problemelor

Ghid de utilizare a manualului	4
Competențe generale și specifice conform programei școlare	5
Evaluare inițială	6
1.1. Principii de elaborare a unui program	7
1.1.1. Noțiuni de bază	7
1.1.2. Modelul Intrare – Prelucrare – ieșire	8
1.1.3. Gândirea computațională și proiectarea soluției	8
1.1.4. Etapele elaborării unui program și rolul fiecăreia	8
1.1.5. Reprezentarea algoritmilor	9
1.1.6. Introducere în limbajul Python	10
1.1.7. Citirea și afișarea datelor	11
1.1.8. Expresii și operatori	12
1.1.9. Structuri fundamentale de control	13
1.1.10. Urmărirea execuției algoritmilor și identificarea erorilor	16
1.1.11. Eficiența algoritmilor – abordare intuitivă	17
1.1.12. Proiectarea modulară a algoritmilor	18
1.2. Prelucrări ale datelor numerice	20
1.2.1. Operații cu cifrele unui număr	20
1.2.2. Divizori, multipli și CMMDC (cel mai mare divizor comun)	22
1.2.3. Prelucrarea numerelor și descompunerea în factori primi	26
1.3. Recapitulare	28
1.4. Evaluare sumativă	29

CAPITOLUL 2

Memorarea datelor și organizarea codului în limbaj de programare

2.1. Subprograme	30
2.1.1. Subprograme – rol, caracteristici	30
2.1.2. Subprograme definite de utilizator – implementare în limbajul Python	33
2.1.3. Variabile locale, variabile globale	39
2.1.4. Funcții predefinite Python	41
2.2. Introducere în programarea orientată pe obiecte în limbaj de programare	44
2.2.1. Noțiuni de bază în POO	44
2.2.2. Clase predefinite în limbajul Python	46
2.3. Fișiere text	48
2.3.1. Caracteristici, principii de lucru	48
2.3.2. Implementarea fișierelor text în limbajul Python	48
2.4. Recapitulare	55
2.5. Evaluare sumativă	56

CAPITOLUL 3

Liste

3.1. Modelul conceptual liniar	57
3.1.1. Introducere	57
3.1.2. Lista	58
3.2. Clasa „list” din Python	61
3.2.1. Crearea unei liste	61
3.2.2. Indexarea elementelor din listă. Parcurgerea unei liste	62
3.2.3. Operatori	63
3.2.4. Metode ale clasei „list”	64
3.2.5. Funcții predefinite pentru liste	66
3.3. Metode de generare sistematică a elementelor unei liste	68
3.4. Metode de sortare	72
3.4.1. Introducere	72
3.4.2. Sortarea prin Selecție (Selection Sort)	73
3.4.3. Sortare cu liste de frecvență (Counting Sort)	76
3.5. Recapitulare	79
3.6. Evaluare sumativă	81
Recapitulare și evaluare finală	82
Soluții (selectiv)	84
Bibliografie	84

Ghid de utilizare a manualului

Manualul cuprinde variantele tipărită și digitală



Simboluri folosite
în varianta digitală



Rezolvă



Privește



Vizionează

Manualul **Informatică**, curriculum de specialitate (CS), filiera teoretică, profilul real, specializarea științe ale naturii pentru clasa a IX-a este structurat în trei capitole, care respectă domeniile și conținuturile existente în programa școlară. Lecțiile sunt însoțite de un număr considerabil de exemple, care determină formarea competențelor specifice cu care acestea sunt corelate.

Manualul cuprinde un număr mare de **exemple**, **probleme rezolvate**, probleme propuse și teste de evaluare/autoevaluare. La finalul manualului se regăsesc răspunsuri la testele de evaluare.

Unele lecții conțin și activități precum:

Fișă de lucru, **Activitate în echipă**,
Știi că..., **Miniproiecte**, **Investigație**,
Studiu de caz, **Important** etc.

Miniproiectele se atașează
portofoliului personal.

Exemple

Activitate în echipă

Important

Fișă de observare a comportamentului și a activității elevului

Miniproiect

RECAPITULARE

Fișă de observare a comportamentului și a activității elevului

Capitolul 1

Probleme rezolvate

Fișă de lucru

Important

Aplicații

COMPETENȚE GENERALE

1. Identifică principalele caracteristici ale modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru înțelegerea fundamentelor programării
2. Explică principii care stau la baza modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru a fundamenta în mod logic proiectarea și implementarea soluțiilor informatice
3. Utilizează modele conceptuale și operaționale ale dezvoltării produselor software, în scopul obținerii de soluții informatice funcționale și eficiente
4. Analizează caracteristicile și aplicabilitatea modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru a selecta soluțiile cele mai potrivite în funcție de contextul informatic dat
5. Evaluează corectitudinea și eficiența soluțiilor informatice, în vederea optimizării și asigurării funcționalității în diverse scenarii de utilizare
6. Elaborează algoritmi și programe personalizate, pentru a crea soluții informatice coerente și adaptate cerințelor.

COMPETENȚE SPECIFICE

- 1.1. Identifică principalele caracteristici ale organizării datelor în cadrul unor modele conceptuale fundamentale – date simple sau liste, pentru a structura și accesa date în vederea prelucrării acestora
- 1.2. Identifică specificul, caracteristicile și etapele unor algoritmi specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 1.3. Identifică specificul, caracteristicile și etapele unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 1.4. Identifică principalele elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale – date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 1.5. Identifică elementele de sintaxă și componentele din definiția și apelul subprogramelor și subprogramelor predefinite pentru operații uzuale, pentru a le utiliza în implementarea algoritmilor în limbaj de programare
- 2.1. Explică principiile de organizare a datelor în cadrul unor modele conceptuale fundamentale - date simple sau liste, pentru a le gestiona eficient
- 2.2. Explică etapele algoritmilor specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 2.3. Explică etapele unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 2.4. Explică rolul principalelor elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale – date simple sau liste, respectiv pentru transmiterea datelor de la și către un program în vederea rezolvării eficiente a problemelor informatice
- 2.5. Explică mecanismul de executare a subprogramelor și subprogramelor predefinite pentru operații uzuale, pentru a le utiliza în implementarea algoritmilor în limbaj de programare
- 3.1. Utilizează modul de organizare și prelucrare a datelor în cadrul unor modele conceptuale fundamentale – date simple sau liste, pentru a rezolva probleme de gestionare a datelor
- 3.2. Aplică algoritmi specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 3.3. Aplică strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 3.4. Utilizează principalele elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale – date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 3.5. Utilizează elementele de sintaxă și componentele din definiția și apelul subprogramelor și subprogramelor predefinite pentru operații uzuale, în implementarea algoritmilor în limbaj de programare
- 4.1. Analizează caracteristicile, modul de prelucrare, avantajele și dezavantajele utilizării unor modele conceptuale fundamentale – date simple sau liste, pentru a alege structura adecvată în rezolvarea unor probleme concrete
- 4.2. Analizează comportamentul, aplicabilitatea, avantajele și dezavantajele utilizării unor algoritmi specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 4.3. Analizează aplicabilitatea, avantajele și dezavantajele utilizării unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 4.4. Analizează aplicabilitatea, avantajele și dezavantajele utilizării unor elemente ale limbajului de programare, pentru a identifica soluția adecvată prelucrării datelor organizate în modele fundamentale – date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 4.5. Analizează aplicabilitatea, avantajele și dezavantajele utilizării subprogramelor, în implementarea algoritmilor în limbaj de programare
- 5.1. Argumentează alegerea unor modele conceptuale fundamentale - date simple sau liste - și a modurilor de prelucrare a acestora pentru rezolvarea unor probleme informatice concrete
- 5.2. Evaluează corectitudinea și eficiența soluțiilor cu algoritmi specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 5.3. Evaluează corectitudinea și eficiența algoritmilor care utilizează strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 5.4. Evaluează corectitudinea și eficiența aplicațiilor care utilizează elemente specifice de limbaj de programare pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 5.5. Evaluează corectitudinea și eficiența programelor care utilizează subprograme, pentru a rezolva probleme informatice
- 6.1. Proiectează algoritmi personalizați care utilizează modele conceptuale fundamentale - date simple sau liste pentru organizarea și prelucrarea eficientă a datelor, utilizând algoritmi de bază, în vederea rezolvării unor probleme
- 6.2. Proiectează soluții cu aplicarea personalizată a algoritmilor specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 6.3. Proiectează modular algoritmi personalizați pentru rezolvarea problemelor
- 6.4. Implementează aplicații care integrează în mod personalizat elemente specifice de limbaj de programare pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea realizării unor soluții funcționale și performante
- 6.5. Implementează aplicații în limbaj de programare care integrează subprograme personalizate, pentru a modulariza și organiza eficient codul în cadrul soluțiilor informatice

Evaluare inițială

Timp de lucru: 50 minute

Toate subiectele sunt obligatorii. Se acordă 1 punct din oficiu.

Notății utilizate: $a \bmod b$ – restul împărțirii, $a \div b$ – câtul împărțirii întregi, unde a și b sunt numere întregi și $b \neq 0$.

Subiectul I – Întrebări de teorie

2 x 0,5 p. = 1 p.

1. Să se definească noțiunea de algoritm.
2. Să se numească două structuri repetitive studiate în gimnaziu.

Subiectul II – Citirea unor algoritmi în pseudocod

3 x 2 p. = 6 p.

1. Se consideră algoritmul:

```
citește a,b      // a, b numere reale
dacă a > b atunci
    scrie a
altfel
    scrie b
sfârșit_dacă
```

Ce se afișează dacă se citesc $a = 5$, $b = 8$? Ce rol are acest algoritm?

2. Se consideră algoritmul:

```
S ← 0
pentru i ← 1,5 execută
    S ← S+i
sfârșit_pentru
scrie S
```

Ce valoare are S la finalul algoritmului?

3. Se consideră algoritmul:

```
citește n      //n - număr natural
nenul
cât_timp n ≠ 0 execută
    c ← n mod 10
    dacă c mod 2 = 1 atunci
        scrie c
    sfârșit_dacă
    n ← n div 10
sfârșit_cât_timp
```

Ce se afișează dacă se citește $n=27961$? Ce rol are acest algoritm?

Subiectul III – Scrierea unui algoritm

2 p.

Alege UNA dintre cerințele de mai jos și scrie algoritmul corespunzător în limbaj pseudocod:

1. Se citește un număr natural nenul n , să se afișeze suma numerelor naturale pare de la 1 la n .
2. Se citesc trei numere reale a , b , c , să se afișeze cel mai mare dintre ele.



1.1. Principii de elaborare a unui program

1.1.1. Noțiuni de bază

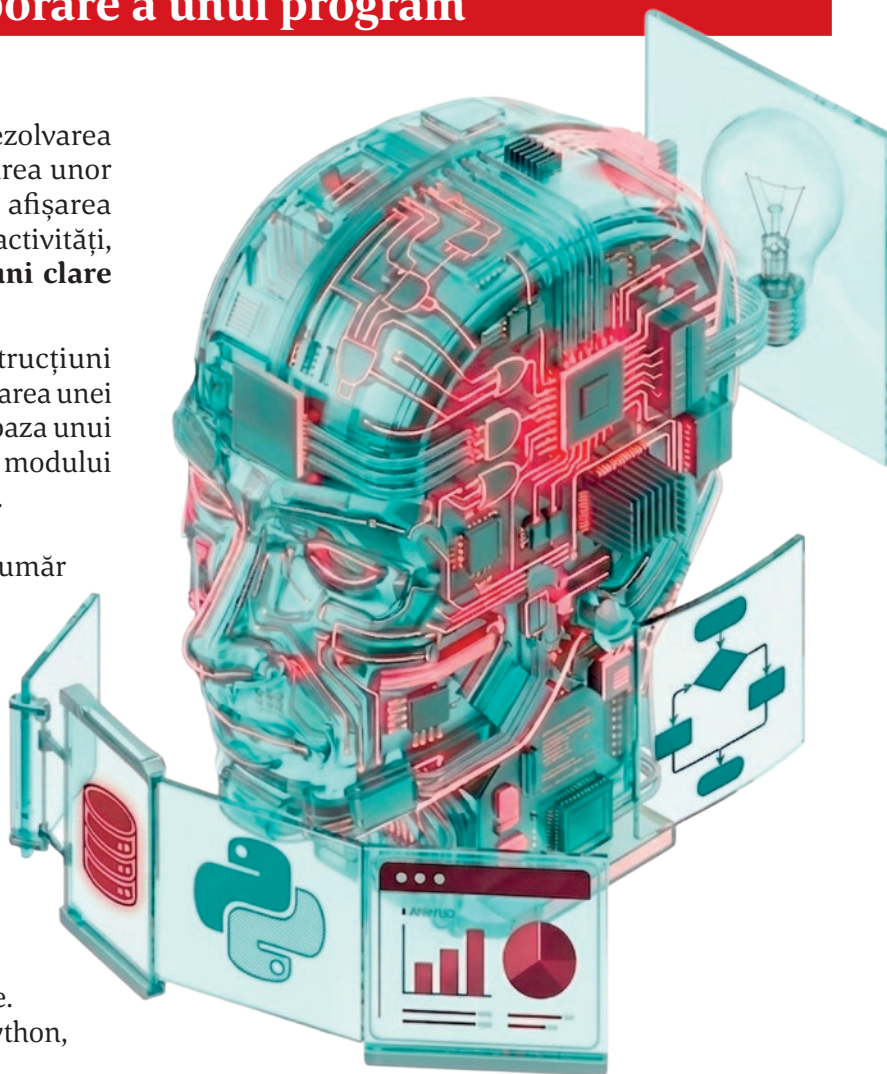
Calculatorul poate fi utilizat pentru rezolvarea diferitelor probleme, cum ar fi efectuarea unor calcule, prelucrarea informațiilor sau afișarea unor rezultate. Pentru a realiza aceste activități, calculatorul are nevoie de **instrucțiuni clare** și **bine ordonate**.

Un **program** este o succesiune de instrucțiuni care descriu pașii necesari pentru rezolvarea unei probleme. Programul este construit pe baza unui **algoritm**, adică o descriere pas cu pas a modului în care poate fi obținut rezultatul dorit.

Proprietățile unui algoritm:

- ▶ **fini**t (procesul se încheie după un număr limitat de pași),
- ▶ **corect** (conduce la obținerea rezultatului așteptat),
- ▶ **general** (se aplică unei clase de probleme, nu doar unui singur caz particular) și
- ▶ **complet** (tratează toate cazurile necesare pentru rezolvarea problemei).

Algoritmul este transpus într-un **limbaj de programare**, astfel încât calculatorul să poată executa instrucțiunile. În acest manual este utilizat limbajul Python, ales pentru claritatea și simplitatea sa.



Un **limbaj de programare** este un sistem de reguli și simboluri care permite scrierea programelor. Prin intermediul unui limbaj de programare, algoritmi sunt transformați în instrucțiuni precise, exprimate într-un limbaj înțeles de calculator. Există două moduri principale prin care un program este transpus într-un limbaj înțeles de calculator:

- 1 limbaje compilate** (de exemplu, C++ sau Pascal), în care programul este tradus integral înainte de execuție;
- 2 limbaje interpretate** (de exemplu, Python sau JavaScript), în care instrucțiunile sunt analizate și executate pe rând.

Pentru a scrie programe corecte, trebuie respectate: **sintaxa** (ansamblul regulilor de scriere a instrucțiunilor), **semantica** (semnificația instrucțiunilor și modul în care sunt executate) și **vocabularul limbajului** (totalitatea cuvintelor-cheie, simbolurilor și elementelor utilizate în programare).

Calculatorul execută instrucțiunile exact în ordinea în care sunt scrise: nu deduce informații lipsă și produce același rezultat pentru aceleași date de intrare.



1.1.2. Modelul Intrare – Prelucrare – Ieșire



Rezolvarea unei probleme cu ajutorul calculatorului poate fi gândită ca o succesiune de trei etape: **Intrare** (datele primite), **Prelucrare** (operațiile aplicate asupra datelor) și **Ieșire** (rezultatul afișat). Acest model ajută la organizarea logică a soluției înainte de scrierea programului.



EXEMPLU

Un magazin vinde 3 caiete la prețul de 15 lei. Se cere să se determine cât vor costa 5 caiete, presupunând că prețul este direct proporțional cu numărul de caiete.

Rezolvare (modelul IPI):

Intrare – numărul cunoscut de caiete (3), prețul total (15 lei) și numărul de caiete dorit (5);

Prelucrare – se determină prețul unui caiet prin împărțirea lui 15 la 3, apoi se înmulțește rezultatul cu 5;

Ieșire – prețul total pentru 5 caiete.

1.1.3. Gândirea computațională și proiectarea soluției

Gândirea computațională presupune analizarea unei probleme și formularea unei soluții clare, alcătuită din pași logici ce pot fi executați de calculator. Aceasta implică: înțelegerea exactă a problemei, identificarea datelor necesare, descompunerea soluției în pași simpli și logici și formularea instrucțiunilor într-un mod clar.



ÎNVĂȚARE PRIN DESCOPERIRE

Analizați problema calculării perimetrului unui dreptunghi și indicați: datele de intrare, pașii de calcul și rezultatul final.



IMPORTANT

Etapile elaborării unui program sunt strâns legate între ele și pot fi parcurse de mai multe ori, până când soluția finală este corectă și completă.

1.1.4. Etapele elaborării unui program și rolul fiecăreia

Rezolvarea unei probleme cu ajutorul calculatorului se realizează într-un mod organizat, urmând un set de etape bine definite.

Etapa	Rol
1. Analiza problemei	Se clarifică cerința și se identifică datele de intrare (informațiile cunoscute; acestea sunt introduse în calculator cu ajutorul unor dispozitive de intrare, cum ar fi tastatura), datele de ieșire (rezultatele dorite; acestea sunt prezentate utilizatorului cu ajutorul unor dispozitive de ieșire, precum monitorul) și datele de manevră (valori intermediare folosite în calcule).
2. Proiectarea soluției	Se stabilesc pașii logici necesari, conducând la construirea algoritmului.
3. Implementarea	Transpunerea algoritmului într-un limbaj de programare, respectând regulile de scriere ale acestuia.
4. Testarea	Se rulează programul și se compară rezultatele obținute cu cele așteptate, folosind exemple concrete.
5. Depanarea	Identificarea și corectarea erorilor. Poate necesita revenirea la implementare sau la proiectarea soluției, până când programul funcționează corect.

1.1.5. Reprezentarea algoritmilor



Un algoritm poate fi reprezentat în mai multe moduri, fiecare având rolul de a exprima pașii de rezolvare cu un grad diferit de precizie.

Mod de reprezentare	Caracteristici	Avantaje	Limite
Limbaaj natural	Pași descriși în propoziții obișnuite.	Ușor de citit și de înțeles.	Ambiguu; nu poate fi executat direct de calculator.
Pseudocod	Formă intermediară: folosește cuvinte-cheie (citește, scrie, dacă, cât timp) fără reguli stricte de sintaxă.	Evidențiază clar datele și ordinea operațiilor; ușor de transpus în orice limbaj.	Nu se execută pe calculator; depinde de convenția autorului.
Diagramă de flux (schemă logică)	Reprezentare grafică prin simboluri și săgeți.	Vizualizează clar fluxul și deciziile.	Devine dificil de urmărit pentru algoritmi lungi.
Limbaaj de programare	Transpunere într-un limbaj (Python, C++ etc.) cu sintaxă strictă. Programul este executat de calculator printr-un interpretor (Python) sau printr-un compilator (C++).	Poate fi executat direct; sintaxa garantează caracterul neambiguu.	Necesită cunoașterea regulilor limbajului.

Simbolurile uzuale în diagramele de flux

Simbol	Denumire	Descriere
	Bloc terminal	Marchează începutul (START) și sfârșitul (STOP) procesului.
	Linie de flux	Indică direcția execuției și leagă etapele algoritmului.
	Bloc de intrare/ieșire	Reprezintă citirea datelor sau afișarea rezultatelor.
	Bloc de proces	Reprezintă o operație sau un calcul efectuat.
	Bloc de decizie	Indică verificarea unei condiții și ramificarea (Da/Nu).
	Bloc conector	Conectează mai multe puncte ale schemei logice.
	Bloc de subprogram	Reprezintă un apel al unui subprogram.



STUDIU DE CAZ

Se citesc trei note (n_1 , n_2 , n_3) și se cere afișarea mediei aritmetice a acestora.

Analiza problemei:

► Date de intrare: n_1 , n_2 , n_3 . ► Date de ieșire: *media*. ► Date de manevră: *suma*.

Limbaj natural	Proiectarea soluției (pseudocod)	Implementarea (Python)
Se cunosc trei note. Se calculează suma acestora și se împarte la 3. Rezultatul este media aritmetică.	<pre> citește n_1, n_2, n_3 suma ← $n_1 + n_2 + n_3$ media ← suma / 3 scrie media </pre>	<pre> n1 = int(input("n1 = ")) n2 = int(input("n2 = ")) n3 = int(input("n3 = ")) suma = n1 + n2 + n3 media = suma / 3 print(media) </pre>

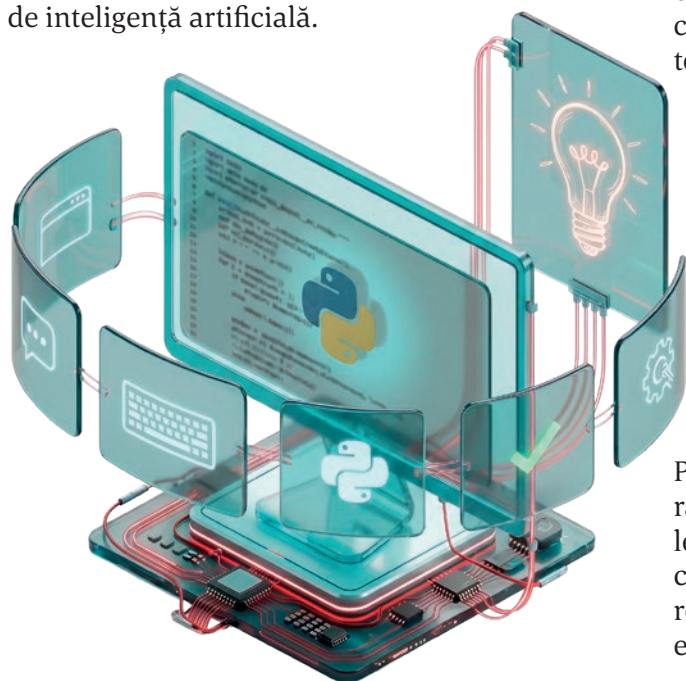
Testarea (exemplu de rulare în consolă):

$n_1 = 9$; $n_2 = 10$; $n_3 = 8$; *media* = 9.0

Depanarea: dacă *media* afișată nu corespunde celei așteptate, se verifică dacă împărțirea se face la 3 și dacă s-au folosit corect parantezele. O greșeală frecventă este scrierea $n_1 + n_2 + n_3 / 3$, care împarte doar pe n_3 la 3.

1.1.6. Introducere în limbajul Python

Python este un limbaj de programare modern, ușor de învățat datorită sintaxei sale simple și clare. Din acest motiv, este o alegere excelentă pentru începători. Cu ajutorul limbajului Python, se pot crea jocuri, aplicații web sau chiar sisteme de inteligență artificială.



Vocabularul limbajului Python este alcătuit din: setul de caractere (litere, cifre, simboluri), identificatori (nume de variabile, funcții), separatori (spații, paranteze, virgule), constante (valori fixe), cuvinte-cheie – cuvinte rezervate ale limbajului, care au o semnificație predefinită și nu pot fi folosite ca nume de variabile sau funcții (de exemplu: **if**, **while**, **for**, **def**, **return**, **import**) și operatori (de exemplu: **+**, **-**, *****, **/**).

Comentariile în Python sunt texte care încep cu simbolul **#** și sunt ignorate de program la executare. Ele sunt utilizate pentru descrierea codului, astfel încât acesta să poată fi înțeles mai ușor.

În informatică, informațiile cu care lucrează un program poartă numele de **date**. Ele pot reprezenta valori numerice (vârsta, nota, temperatura), texte (nume, adresa) sau valori logice (adevărat/fals).

Pentru a fi utilizate de program, datele trebuie memorate. Memorarea se realizează cu ajutorul variabilelor. O **variabilă** este o zonă de memorie identificată printr-un nume, în care este stocată o valoare ce poate fi utilizată sau modificată pe parcursul execuției.

**IMPORTANT**

Operatorul `=` nu exprimă egalitatea matematică, ci operația de atribuire. O instrucțiune de forma **variabilă = expresie** se execută în doi pași: (1) se evaluează expresia din dreapta; (2) rezultatul este atribuit variabilei din stânga.

**EXEMPLU**

```
x = 1
x = x + 1
```

După executare, variabila `x` are valoarea `2`. Valoarea anterioară a variabilei este utilizată în calcul.



Identificatorii sunt numele atribuite variabilelor (și altor elemente: funcții, constante, clase).

- Reguli de denumire a identificatorilor:
- ▶ numele trebuie să înceapă cu o literă sau cu simbolul underscore (`_`);
 - ▶ nu sunt permise spațiile sau caracterele speciale;
 - ▶ se pot utiliza litere, cifre și caracterul underscore (`_`);
 - ▶ se recomandă folosirea unor denumiri sugestive;
 - ▶ nu se pot folosi cuvintele-cheie (rezervate) ale limbajului.

Constantele sunt variabile ale căror valori, prin convenție, nu trebuie să se modifice pe durata execuției și se scriu, de regulă, cu litere mari.

De exemplu, `NUMAR_LUNI_DIN_AN = 12`.

Fiecare variabilă are asociat un tip de date, care indică natura informației stocate. Principalele tipuri din Python sunt prezentate în tabelul următor.

Tip	Semnificație	Exemplu
<code>int</code>	numere întregi (fără zecimale)	<code>nota = 10</code> <code>nivel = -1</code>
<code>float</code>	numere reale (cu zecimale)	<code>media = 9.75</code> <code>temperatura = -3.5</code>
<code>str</code>	șiruri de caractere (text), între <code>" "</code> sau <code>' '</code>	<code>oras = "București"</code> <code>mesaj = 'O zi bună'</code>
<code>bool</code>	valori logice: <code>True</code> / <code>False</code>	<code>este_prezent = True</code> <code>are_tema = False</code>

**IMPORTANT**

Șirurile de caractere pot fi scrise fie între ghilimele (`" "`), fie între apostrofuri (`' '`). Cele două forme sunt echivalente. Alegerea uneia devine importantă atunci când textul conține deja unul dintre aceste simboluri, de exemplu: `mesaj = 'El a spus "Bună ziua"'`.

1.1.7. Citirea și afișarea datelor

Pentru scrierea și rularea programelor se utilizează un mediu de dezvoltare integrat (IDE – Integrated Development Environment), precum **PyCharm**, care permite scrierea, verificarea și rularea programelor. Rezultatele se afișează în **consolă** – o interfață de tip text integrată în IDE, prin care programul comunică cu utilizatorul: afișează mesaje și preia date introduse de la tastatură.

Citirea datelor se realizează cu ajutorul funcției `input()`. Aceasta afișează un mesaj și așteaptă ca utilizatorul să introducă o valoare, care este preluată și memorată într-o variabilă. Valoarea citită este inițial de tip `str` (șir de caractere). Pentru a o utiliza în calcule, se realizează **conversia de tip**: `int()` pentru numere întregi și `float()` pentru numere reale.

Afișarea datelor se realizează cu funcția `print()`. Aceasta primește una sau mai multe valori (texte, numere, expresii) separate prin virgulă și le afișează pe ecran, urmate de trecerea la linia următoare.



> > > EXEMPLU

Cod Python	Explicații
<code>print("O zi bună!")</code>	Afișează un text fix.
<code>nume = input("Nume: ")</code> <code>print("Salut,", nume)</code>	Citește un text și afișează un mesaj personalizat.
<code>a = int(input("a= "))</code> <code>b = int(input("b= "))</code> <code>print("Suma:", a + b)</code>	Citește două numere întregi și afișează suma.
<code>print("Raport:")</code> <code>print("Expresie: (12-4)*3")</code> <code>print("Rezultat:", (12-4)*3)</code>	Afișează un mic raport de calcul pe trei linii.
<code>varsta = int(input("Introduceți vârsta: "))</code> <code>print(f"Vârsta este: {varsta}")</code>	Citește un număr întreg și îl afișează folosind un f-string – șir prefixat cu f , în care expresia dintre {} este înlocuită automat cu valoarea variabilei varsta .

1.1.8. Expresii și operatori

O **expresie** este o **combinație de operatori și operanți** legați între ei după reguli specifice limbajului de programare. Operanzii pot fi: variabile, constante, funcții. Rezultatul evaluării unei expresii este o valoare numerică (expresii aritmetice) sau logică, de tip **True/False** (expresii logice).

Operatorii aritmetici permit efectuarea calculelor matematice.



IMPORTANT

În Python, operația de împărțire (`/`) produce un rezultat de tip **float**, chiar dacă împărțirea este exactă (de exemplu, `10 / 2` este `5.0`). Pentru împărțirea cu rezultat întreg se folosește `//`.

Operator	+	-	*	/	//	%	**
Semnificație	adunare	scădere	înmulțire	împărțire	împărțire întreagă	rest	putere

Operatorii relaționali (de comparație) – rezultatul este **True** sau **False**:

Operator	<	<=	>	>=	==	!=
Semnificație	mai mic	mai mic sau egal	mai mare	mai mare sau egal	egal	diferit

Operatorii logici – combină valori adevărate/false:

Operator	Semnificație	Exemplu
and	și (ambele condiții adevărate)	<code>(24 > 0) and (24 < 100)</code>
or	sau (cel puțin o condiție adevărată)	<code>(1 == 1) or (1 == 5)</code>
not	negație	<code>not (1 == 0)</code>

Prioritatea operatorilor:

Prioritate	1	2	3	4	5	6	7	8
Operator	()	**	*, /, //, %	+, -	relaționali (<, <=, >, >=, ==, !=)	not	and	or

Pentru claritate, se recomandă folosirea parantezelor.



PROBLEMĂ REZOLVATĂ

În programul de mai jos sunt utilizate mai multe tipuri de date, expresii aritmetice și logice.

Cod Python

```

nume = input("Introduceți numele elevului: ")
nota1 = float(input("Introduceți prima notă: "))
nota2 = float(input("Introduceți a doua notă: "))
prezent = True

media = (nota1 + nota2) / 2
promovat = media >= 5 and prezent == True

print("Elev:", nume)
print("Media:", media)
print("Este promovat:", promovat)

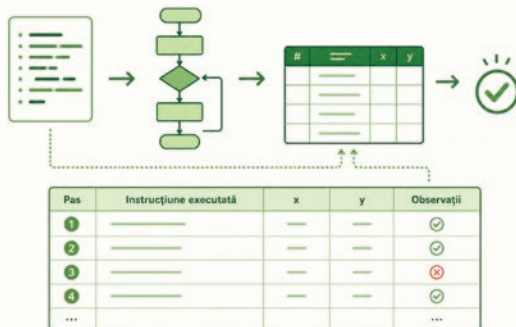
```

Explicații

- variabila `nume` citește și memorează un text introdus de la tastatură;
- variabilele `nota1` și `nota2` citesc de la tastatură valori numerice, convertite cu `float()` din șirul returnat de `input()`;
- variabila `prezent` memorează o valoare logică (`True`);
- expresia aritmetică calculează media notelor;
- expresia logică verifică dacă elevul este promovat;
- rezultatele sunt afișate în consolă cu `print()`.

1.1.9. Structuri fundamentale de control

În practică, execuția unui algoritm poate depinde de anumite condiții sau poate presupune repetarea unor pași. Pentru astfel de situații se folosesc **structurile fundamentale de control**, care stabilesc modul de parcurgere a pașilor de rezolvare: decizia, alternativa, alegerea multiplă și repetarea.



IMPORTANT

În Python, blocurile de cod sunt delimitate prin indentare (un TAB sau 4 spații spre dreapta), nu prin acolade. Indentarea corectă este esențială: toate instrucțiunile scrise la același nivel fac parte din același bloc. Amestecarea spațiilor și a tab-urilor în același bloc de cod poate produce eroarea `TabError`.

1.1.9.1. Decizia, alternativa și alegerea multiplă: **if**, **if-else**, **elif****Instrucțiunea if – decizia.**

Permite executarea unui bloc de instrucțiuni doar dacă o condiție este adevărată.

```
if condiție:
    instrucțiuni
```

Instrucțiunea if-else – alternativa.

Algoritmul alege între două acțiuni diferite: dacă condiția este **True** se execută ramura **if**, altfel se execută ramura **else**.

```
if condiție:
    instrucțiuni_1
else:
    instrucțiuni_2
```

Instrucțiunea elif – alegerea multiplă.

Verifică succesiv mai multe condiții și execută blocul corespunzător primei condiții adevărate. Dacă niciuna nu este adevărată, se execută ramura **else** (opțională).

```
if condiție_1:
    instrucțiuni_1
elif condiție_2:
    instrucțiuni_2
else:
    instrucțiuni_finale
```



PROBLEMĂ REZOLVATĂ

Exemplu – în limbaj natural	Pseudocod
Se citește nota unui elev. Se cere afișarea calificativului: - "Insuficient" dacă nota este sub 5; - "Suficient" dacă nota este 5 sau 6; - "Bine" dacă nota este 7 sau 8; - "Foarte bine" dacă nota este 9 sau 10.	<pre>citește nota dacă nota < 5 atunci afișează "Insuficient" altfel dacă nota <= 6 atunci afișează "Suficient" altfel dacă nota <= 8 atunci afișează "Bine" altfel afișează "Foarte bine" sfârșit_dacă sfârșit_dacă</pre>
Cod Python	Explicații
<pre>nota = int(input("Nota: ")) if nota < 5: print("Insuficient") elif nota <= 6: print("Suficient") elif nota <= 8: print("Bine") else: print("Foarte bine")</pre>	- se citește variabila <i>nota</i> (de exemplu 8); - prima condiție verificată este <i>nota < 5</i> → fals; - a doua condiție <i>nota <= 6</i> → fals; - a treia condiție <i>nota <= 8</i> → adevărat; - se execută instrucțiunea corespunzătoare; - se afișează mesajul "Bine"; - verificările ulterioare nu mai sunt realizate.

**IMPORTANT**

O buclă **while** trebuie să aibă o **condiție de oprire**, bazată de obicei pe o **variabilă de control** care trebuie modificată în corpul buclei. Dacă variabila nu este actualizată, bucla devine infinită și programul nu se mai termină.

1.1.9.2. Repetarea — bucle: `while` și `for`

O buclă este o structură de control care permite **executarea repetată** a unui bloc de instrucțiuni atât timp cât este îndeplinită o anumită condiție sau pentru un număr cunoscut de pași. Instrucțiunile care se repetă formează **corpul buclei**, iar fiecare parcurgere a corpului se numește **iterație**.

Instrucțiunea `while` – repetare cu test inițial. Se folosește atunci când nu se cunoaște dinainte numărul de repetări. La fiecare iterație condiția este evaluată; dacă este **True**, se execută instrucțiunile; dacă este **False**, bucla se oprește.

```
while condiție:
    instrucțiuni
```

Instrucțiunea `for` – repetare cu număr cunoscut de pași. Parcurge valorile unei secvențe, de exemplu generată cu `range()`.

```
for variabilă in secvență:
    instrucțiuni
```

> > >

EXEMPLU

Se cere afișarea numerelor de la 1 la 5.

Cod Python – folosind while	Cod Python – folosind for	Exemplu de rulare
<pre>i = 1 while i <= 5: print(i) i = i + 1</pre>	<pre>for i in range(1, 6): print(i)</pre>	<pre>1 2 3 4 5</pre>

**PROBLEMĂ REZOLVATĂ – șiruri de valori cu număr necunoscut de pași**

Se citesc note de la tastatură până la introducerea valorii 0. Să se calculeze media acestora.

Cod Python	Explicații
<pre>suma = 0 numar_note = 0 nota = int(input("Introdu o notă: ")) while nota != 0: suma = suma + nota numar_note = numar_note + 1 nota = int(input("Introdu o notă: ")) if numar_note > 0: media = suma / numar_note print("Media notelor este:", media) else: print("Nicio notă introdusă")</pre>	– se inițializează suma cu 0; – se inițializează numărul de note cu 0; – se citește prima notă de la tastatură; – funcția <code>input()</code> este utilizată pentru a prelua valorile introduse de utilizator; – funcția <code>int()</code> transformă aceste valori din text în numere întregi; – se repetă cât timp nota este diferită de 0; – se adaugă nota la sumă; – se crește numărul de note; – se citește următoarea notă; – se verifică dacă există note introduse; – se calculează media; – se afișează media, folosind instrucțiunea <code>print()</code>; – altfel, se tratează cazul fără note; – se afișează mesaj corespunzător.



1.1.10. Urmărirea execuției algoritmilor și identificarea erorilor

Pentru a înțelege corect un algoritm, se urmărește pas cu pas evoluția valorilor variabilelor, folosind un tabel de urmărire (pas, instrucțiune executată, valorile variabilelor după execuție).

Acest tabel ajută la identificarea erorilor și la verificarea rezultatului.

EXEMPLU > > >

În exemplul următor, în stânga este codul Python, iar în dreapta tabelul de execuție.

```
s = 0
for i in range(1, 4):
    s = s + i
print(s)
```

Pas	Instrucțiune	i	s
1	s = 0	-	0
2	i = 1	1	0
3	s = s + i	1	1
4	i = 2	2	1
5	s = s + i	2	3
6	i = 3	3	3
7	s = s + i	3	6

Tip eroare	Descriere	Exemplu
Eroare de sintaxă	Instrucțiunile nu respectă regulile limbajului; programul nu poate fi rulat.	<pre>if x > 5 print("x mai mare") # lipsește simbolul :</pre>
Eroare de logică	Programul rulează, dar rezultatul este greșit (algoritm greșit).	<pre>x = 10 y = 5 media = x + y / 2 # corect: (x+y)/2</pre>
Eroare de execuție	Apare în timpul rulării; programul se oprește brusc.	<pre>x = 10 y = 0 print(x / y) # împărțire la 0</pre>



ÎNVĂȚARE PRIN DESCOPERIRE

Analizați programul de mai jos și explicați, pe baza codului și a observațiilor făcute la rulare, cum sunt utilizate structurile de control învățate. Rulați programul în PyCharm, observați rezultatul obținut și explicați: a) cum se modifică valorile variabilelor pe parcursul execuției; b) în ce situație se afișează mesajul “buget sigur” și în ce situație se afișează “buget critic”; c) câte cheltuieli mari există, știind că o cheltuială este considerată mare dacă valoarea ei este mai mare decât 70.

După rulare, programul poate fi salvat. Un **fișier de tip program sursă** conține instrucțiuni scrise într-un limbaj de programare (de exemplu, Python), pe care calculatorul le poate executa. Extensia sa este, de obicei, **.py**.

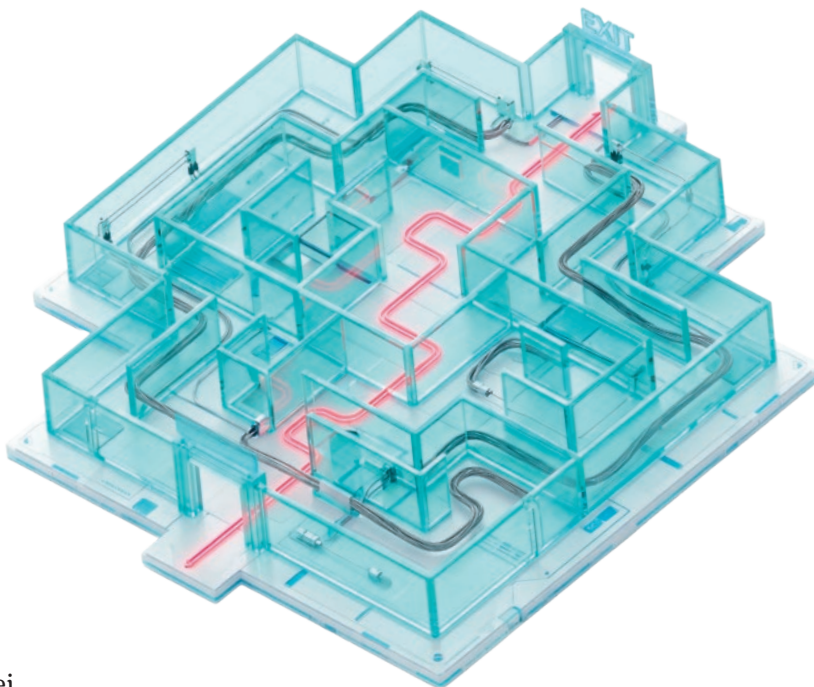
Cod Python	Explicații
<pre> buget = 400 cheltuiala = 70 pas = 1 cheltuieli_mari = 0 while (buget >= 50) and (pas <= 8): buget = buget - cheltuiala if cheltuiala > 70: cheltuieli_mari = cheltuieli_mari + 1 cheltuiala = cheltuiala + 15 pas = pas + 1 if buget >= 100: mesaj = "buget sigur" else: mesaj = "buget critic" print("Buget rămas:", buget) print("Număr pași:", pas - 1) print("Cheltuieli mari:", cheltuieli_mari) print("Mesaj:", mesaj) </pre>	– buget = 400 – suma inițială – cheltuiala = 70 – prima cheltuială – pas = 1 – numără pașii – cheltuieli_mari = 0 – numără cheltuielile > 70 Bucula rulează cât timp: buget >= 50 și pas <= 8. La fiecare pas: – se scade cheltuiala din buget; – dacă cheltuiala > 70, se incrementează cheltuieli_mari – cheltuiala crește cu 15 – pas crește cu 1 După buclă: – dacă buget >= 100 → "buget sigur", altfel → "buget critic" La final se afișează rezultatele.

1.1.11. Eficiența algoritmilor – abordare intuitivă

În rezolvarea unei probleme pot exista mai mulți algoritmi corecți care conduc la același rezultat, dar care diferă prin numărul de pași necesari și prin modul de organizare. Un algoritm este considerat eficient atunci când rezolvă problema folosind cât mai puține operații și cât mai puțină memorie.

Eficiența se analizează din două puncte de vedere:

- 1** **eficiența în timp** – numărul de operații efectuate de calculator pentru obținerea rezultatului într-un timp cât mai scurt;
- 2** **eficiența în memorie (spațiu)** – cantitatea de memorie utilizată pentru stocarea datelor (variabile, liste) pe durata execuției.



>>> **EXEMPLU**

Se cere calcularea sumei primelor 4 numere naturale.

Algoritm A – se adună numerele pe rând, folosind o variabilă **s** și o variabilă contor **i** (ex: pentru **n** = 4: 0+1→1; 1+2→3; 3+3→6; 6+4→10).

```
s = 0
for i in range(1, 4 + 1):
    s = s + i
```

Algoritm B – se aplică Formula lui Gauss:

```
s = n * (n + 1) // 2
s = 4 * (4 + 1) // 2 = 10
```

Ambii algoritmi produc același rezultat, însă B efectuează un număr **fix** de operații (indiferent de **n**), pe când A efectuează **n** adunări. Prin urmare, B este mai **eficient în timp**.

Eficiența în spațiu – ambii algoritmi utilizează un număr constant de variabile (algoritmul A folosește variabilele **s** și **i**, iar algoritmul B doar variabila **s**), astfel că memoria utilizată nu depinde de valoarea lui **n**, fiind de tip constant. În schimb, dacă s-ar memora mai întâi toate cele **n** valori într-o listă și abia apoi s-ar calcula suma, memoria utilizată ar crește proporțional cu **n**, ceea ce ar face soluția mai puțin eficientă din punct de vedere al spațiului.

Numărul de pași și dimensiunea datelor

În practică, eficiența depinde și de dimensiunea datelor de intrare. Dacă un algoritm parcurge **n** valori (de exemplu, **n** note), numărul de pași crește odată cu **n**. Pentru un astfel de algoritm spunem că numărul de operații crește proporțional cu **n** (depinde liniar de **n**). Un algoritm care execută un număr fix de operații (ca Algoritmul B) are timp de executare constant (nu depinde de **n**). Pentru algoritmi cu două bucle imbricate, numărul de pași crește mai rapid – proporțional cu **n**².

1.1.12. Proiectarea modulară a algoritmilor

Un **modul** reprezintă o parte a algoritmului care realizează o sarcină bine definită.

În **proiectarea modulară**, problema este împărțită în mai multe subprobleme (module), fiecare rezolvând o etapă distinctă. Această abordare face algoritmul mai ușor de înțeles, urmărit, verificat și modificat.

Analiză comparativă: soluție compactă vs. modulară

Soluție compactă	Soluție modulară
Toți pașii sunt descriși într-o singură secvență: citire, calcul, afișare.	Problema este separată în etape distincte: citire, calcul parțial, calcul final, afișare. Fiecare etapă are un rol clar.
Mai scurtă; devine dificil de urmărit când problema este complexă.	Mai bine organizată; erorile se identifică mai rapid, fiecare parte este analizată separat.

>>> **EXEMPLU**

Se citește prețul unui produs și procentul de reducere. Să se calculeze prețul final.

Cod Python

```
pret = float(input("Preț inițial: "))
reducere = float(input("Reducere (%): "))

valoare_reducere = pret * reducere / 100
pret_final = pret - valoare_reducere

print("Preț final:", pret_final)
```

Explicații

În acest program se pot identifica modulele:

- citirea datelor (preț și reducere);
- calculul valorii reducerii;
- determinarea prețului final;
- afișarea rezultatului.



FIȘĂ DE LUCRU 1

1. Se consideră următorul algoritmul scris în Python:

```
x = 1
suma = 0

while x <= 5:
    suma = suma + x
    if x % 2 == 0:
        suma = suma + 2
    x = x + 1
```

- Să se identifice variabila de control și condiția de oprire a buclei.
- Să se completeze un tabel de urmărire a execuției pentru primele trei iterații, indicând valorile variabilelor **x** și **suma**.
- Să se precizeze valoarea finală a variabilei **suma**.
- Să se explice rolul instrucțiunii **if** din interiorul buclei.

2. Se consideră următorul program:

```
numar = 10
contor = 1
rezultat = 0

while contor <= numar:
    if contor % 3 == 0:
        rezultat = rezultat + contor
    contor = contor + 1
```

- Să se precizeze ce valori sunt adunate în variabila **rezultat**, în funcție de condiția din instrucțiunea **if**.
- Să se precizeze numărul de iterații ale buclei **while**.
- Să se determine valoarea finală a variabilei **rezultat**.

3. Se citește de la tastatură un număr natural **n**, cu **n** > 0. Apoi se citesc **n** numere întregi. Să se realizeze un program care determină câte dintre numere sunt pare, câte sunt impare și câte sunt negative, calculează suma numerelor pare și media numerelor negative, apoi afișează rezultatele obținute și un mesaj care indică dacă există mai multe numere pare decât impare. Dacă nu există numere negative, se va afișa un mesaj corespunzător.

- Identificați datele de intrare, datele de ieșire și descrieți rezolvarea folosind modelul Intrare–Prelucrare–Ieșire;
- Precizați cele 5 etape ale elaborării unui program;
- Scrieți algoritmul în pseudocod și în limbajul Python;
- Pentru valorile **n** = 4 și numerele 2, -5, 4, -1 urmăriți execuția algoritmului și precizați valorile variabilelor: variabila de control, valoarea curentă citită, numărul de valori pare, numărul de valori negative și sumele calculate.



1.2. Prelucrări ale datelor numerice



1.2.1. Operații cu cifrele unui număr

Numerele naturale sunt formate din cifre, iar în programare apare frecvent necesitatea de a lucra cu cifrele individuale ale unui număr: aflarea ultimei cifre, numărarea cifrelor sau construirea unui număr nou. Prin operații simple de împărțire, putem accesa fiecare cifră a unui număr și putem construi numere noi, cifră cu cifră.

Pentru a lucra cu cifrele unui număr n se folosesc două operații esențiale: $n \% 10$ (returnează ultima cifră) și $n // 10$ (returnează numărul fără ultima cifră).

PROBLEME REZOLVATE



Exemplul 1. Aflarea ultimei cifre a unui număr

Se citește un număr natural n . Să se afișeze ultima cifră a numărului.

Exemplu: $n = 4527$. Ieșire așteptată: 7.

Cod Python

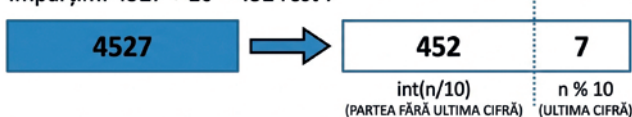
```
n = int(input("Numărul n este: "))
uc = n % 10
print("Ultima cifră este:", uc)
```

Fig. 1 – Extragerea ultimei cifre cu operația MODULO

PAS 1

$n = 4527 \rightarrow$ Operație: $4527 \% 10$

Împărțim: $4527 \div 10 = 452$ rest 7



PAS 2

Rezultat ultima cifră = 7

Exemplul 2. Numărarea cifrelor unui număr

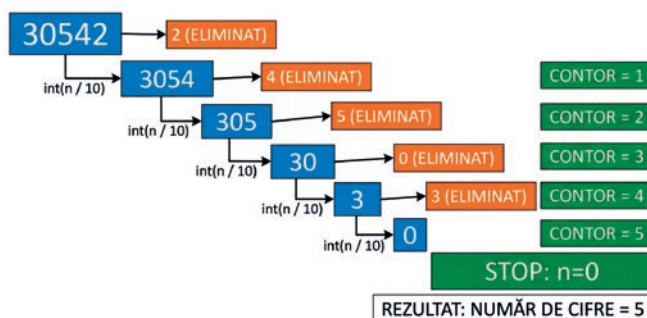
Se citește un număr natural n ($n > 0$). Să se determine câte cifre are numărul.

Exemplu: $n = 30542$. Ieșire așteptată: 5.

Cod Python

```
n = int(input("Numărul n este: "))
contor = 0
while n > 0:
    n = n // 10
    contor = contor + 1
print("Număr de cifre =", contor)
```

Fig. 2 – Numărarea cifrelor prin eliminare repetată





PROBLEME REZOLVATE

Exemplul 3. Adăugarea unei cifre la dreapta unui număr

Se citește un număr natural n și o cifră c . Să se construiască numărul obținut prin adăugarea cifrei c la dreapta lui n .

Exemplu: $n = 123$, $c = 7$. Ieșire așteptată: 1237.

Cod Python

```
n = int(input("Numărul n este: "))
c = int(input("Cifra c este: "))
rezultat = n * 10 + c
print("Rezultat =", rezultat)
```

Fig. 3 – Adăugarea unei cifre la dreapta

Adăugarea cifrei 7 la dreapta numărului 123

Număr inițial:

1	2	3
---	---	---



Pas 1: $123 \times 10 = 1230 \rightarrow$ se „face loc” la dreapta

1	2	3	0
---	---	---	---



Pas 2: $1230 + 7 = 1237 \rightarrow$ se „pune” cifra 7 pe locul liber

1	2	3	
1	2	3	7

Exemplul 4. Adăugarea unei cifre la stânga unui număr

Se citește un număr natural n și o cifră c . Să se construiască numărul obținut prin adăugarea cifrei c la stânga lui n .

Exemplu: $n = 456$, $c = 9$. Ieșire așteptată: 9456.

Cod Python

```
n = int(input("n = "))
c = int(input("c = "))
putere = 1
copie = n
while copie > 0:
    putere = putere * 10
    copie = copie // 10
rezultat = c * putere + n
print("Rezultat =", rezultat)
```

Fig. 4 – Adăugarea unei cifre la stânga

Adăugarea cifrei 9 la stânga numărului 456

4	5	6
---	---	---

Număr inițial: 456

Pas 1: Se numără cifrele $\rightarrow$ 3 cifre $\rightarrow$ putere = $10^3 = 1000$

Pas 2: $9 \times 1000 = 9000$

Pas 3: $9000 + 456 = 9456$

9	0	0	0
9	4	5	6

Cifra 9 se plasează în fața numărului, cifrele existente rămân pe pozițiile lor.



APLICAȚII



1. Se citește un număr natural n . Să se calculeze suma cifrelor sale. Indicații: se inițializează suma cu 0, se extrage ultima cifră cu $n \% 10$, se adaugă la sumă, se elimină ultima cifră cu $n // 10$ și se repetă cât timp $n > 0$.

2. Se citește un număr natural n . Să se construiască și să se afișeze *oglinzitul* (inversarea cifrelor). Exemplu: $n = 1234$, oglinzitul este 4321.

Cod Python

```
n = int(input("n = "))
oglindit = 0
while n > 0:
    cifra = n % 10
    oglindit = oglindit * 10 + cifra
    n = n // 10
print("Oglindit =", oglindit)
```

Explicații

- se inițializează `oglindit = 0`;
- la fiecare pas se extrage ultima cifră și se adaugă la dreapta lui `oglindit`;
- pentru $n = 1234$:
 - pas 1: `oglindit = 0*10+4 = 4`
 - pas 2: `oglindit = 4*10+3 = 43`
 - pas 3: `oglindit = 43*10+2 = 432`
 - pas 4: `oglindit = 432*10+1 = 4321`
- pentru $n = 50810$, oglinzitul este 1805: zeroul interior se păstrează, dar cel de la final dispare.

1.2.2. Divizori, multipli și CMMDC (cel mai mare divizor comun)

1.2.2.1. Divizori și multipli ai unui număr

Un număr d este **divizor** al lui n dacă restul împărțirii lui n la d este zero, ceea ce înseamnă că $n \% d == 0$. Un **multiplu** al unui număr este un număr obținut prin înmulțirea acelui număr cu un număr natural. De exemplu, multiplii lui 4 sunt: 0, 4, 8, 12, 16, 20, ...

Divizibilitatea se verifică în Python folosind operatorul **% (modulo)**. Dacă $n \% d == 0$, atunci d este divizor al lui n . De exemplu, 3 este divizor al lui 12 deoarece $12 \% 3 = 0$.

Pentru a găsi toți divizorii unui număr n , se parcurg toate valorile de la 1 la n și se verifică condiția de divizibilitate. Un caz special important este cel al **numerelor prime**: un număr natural n ($n \geq 2$) este **prim** dacă are exact doi divizori: 1 și n însuși.



PROBLEME REZOLVATE

Tăierea unei benzi în bucăți egale

Se citește lungimea L a unei benzi. Să se afișeze toate posibilitățile de tăiere în bucăți egale (adică toți divizorii lui L). Date de intrare: $L = 30$. Date de ieșire: 1 2 3 5 6 10 15 30.

Cod Python

```
L = int(input("Lungime bandă: "))
for d in range(1, L + 1):
    if L % d == 0:
        print(d, end=" ")
```

Explicații

- se citește lungimea benzii L ;
- se parcurg valorile d de la 1 la L ;
- pentru fiecare d , se testează dacă $L \% d == 0$ (dacă d divide pe L);
- dacă da, d este o lungime posibilă pentru bucăți egale;

- pentru $L = 30$:
 - $d=1$: $30\%1=0$, se afișează 1
 - $d=2$: $30\%2=0$, se afișează 2
 - $d=5$: $30\%5=0$, se afișează 5
 - ...
 - $d=30$: $30\%30=0$, se afișează 30

1.2.2.2. Cel mai mare divizor comun a două numere naturale nenule (CMMDC) și Algoritmul lui Euclid

Cel mai mare divizor comun (CMMDC) a două numere naturale nenule este cel mai mare număr natural care le divide pe ambele. De exemplu, $\text{CMMDC}(12, 18) = 6$, deoarece 6 este cel mai mare număr care divide și pe 12, și pe 18.

Algoritmul lui Euclid – varianta cu scăderi repetate

Acest algoritm se bazează pe o proprietate matematică simplă: dacă două numere a și b au un divizor comun d , atunci și diferența lor ($a - b$) este divizibilă cu d . Concret, algoritmul funcționează astfel: se compară cele două numere; din cel mai mare se scade cel mai mic. Se repetă acest pas până când cele două numere devin egale. Valoarea comună obținută este CMMDC-ul.

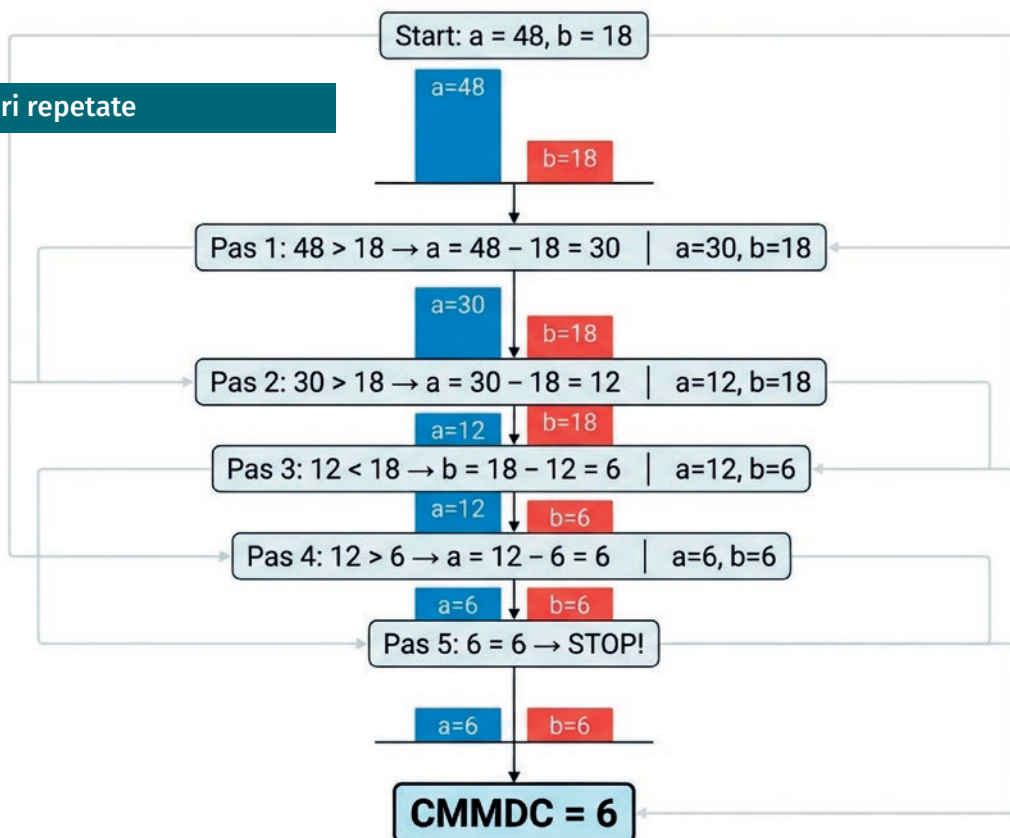
În Fig. 5 sunt ilustrați pașii algoritmului pentru $a = 48$ și $b = 18$. Rezultatul este 6. Acest rezultat are și o interpretare practică: dacă dorim să acoperim o podea de $48 \text{ m} \times 18 \text{ m}$ cu plăci de gresie pătrate cât mai mari, latura maximă a plăcii este 6 m.

Cod Python

```
# a și b numere naturale, nenule

a = int(input("a = "))
b = int(input("b = "))
while a != b:
    if a > b:
        a = a - b
    else:
        b = b - a
print("CMMDC =", a)
```

Fig. 5 – Euclid cu scăderi repetate





Algoritmul lui Euclid – varianta cu împărțiri repetate

Această variantă este mai eficientă deoarece în loc să se efectueze scăderi repetate, se înlocuiește numărul mai mare cu restul împărțirii.

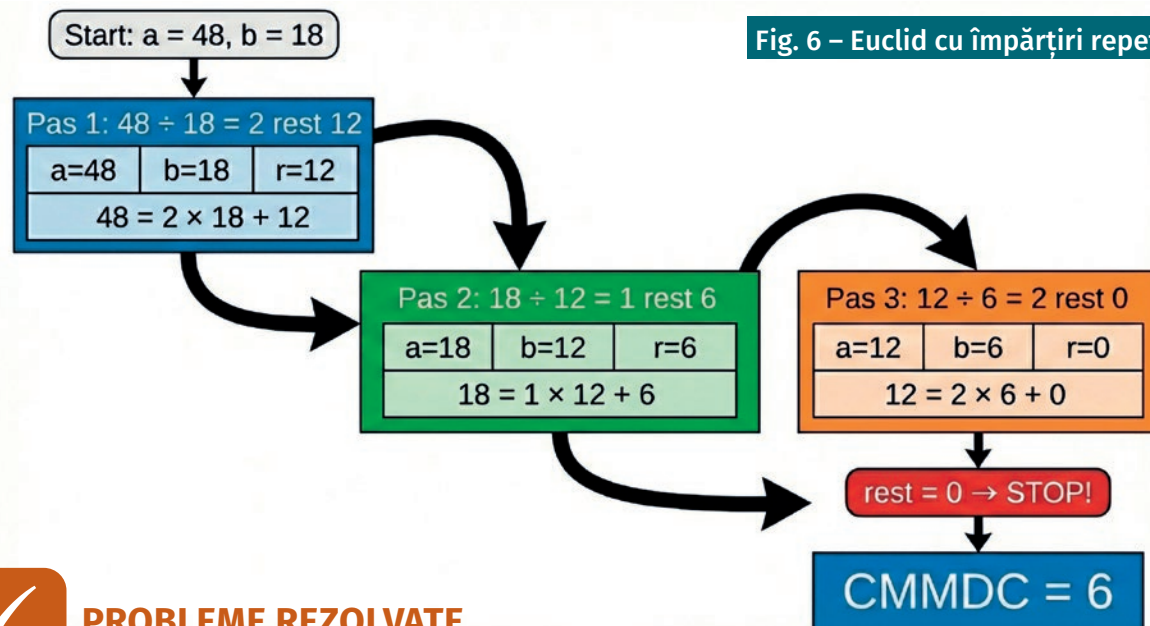
Proprietatea folosită este: $\text{CMMDC}(a, b) = \text{CMMDC}(b, a \% b)$. Procesul se repetă până când restul devine 0, iar CMMDC-ul este ultimul rest nenul.

Să se urmărească algoritmul pentru $a = 48$ și $b = 18$. Se observă că metoda bazată pe împărțiri necesită un număr mai mic de pași decât metoda bazată pe scăderi.

Cod Python

```
a = int(input("a = "))
b = int(input("b = "))
while b != 0:
    r = a % b
    a = b
    b = r
print("CMMDC:", a)
```

Fig. 6 – Euclid cu împărțiri repetate



PROBLEME REZOLVATE

Exemplul 1. Verificarea dacă un număr este prim

Se citește un număr natural n ($n \geq 2$). Să se verifice dacă n este prim.

Un număr este prim dacă are exact 2 divizori: 1 și el însuși.

Exemplu: $n = 13$. Ieșire: 13 este număr prim.

Cod Python

```
n = int(input("n = "))
contor = 0
for d in range(1, n + 1):
    if n % d == 0:
        contor = contor + 1
if contor == 2:
    print("Este număr prim")
else:
    print("Nu este număr prim")
```

Fig. 7 – Verificarea primalității



$d = 1 \rightarrow 13 \% 1 = 0$ ✓ (contor = 1)
 $d = 2 \rightarrow 13 \% 2 = 1$ ✗
 $d = 3 \rightarrow 13 \% 3 = 1$ ✗
 $d = 4 \rightarrow 13 \% 4 = 1$ ✗
 ...
 $d = 12 \rightarrow 13 \% 12 = 1$ ✗
 $d = 13 \rightarrow 13 \% 13 = 0$ ✓ (contor = 2)

Concluzie: contor = 2 → exact 2 divizori → PRIM ✓



Divizorii lui 12:
1, 2, 3, 4, 6, 12

Exemplul 2. Numărarea pașilor algoritmului lui Euclid

Se citește două numere naturale a și b . Să se afișeze câți pași necesită algoritmul lui Euclid cu împărțiri repetate pentru a determina CMMDC-ul lor.

Exemplu: $a = 48, b = 18$. Ieșire: CMMDC = 6, pași = 3.

Cod Python

```
a = int(input("a = "))
b = int(input("b = "))
pași = 0
while b != 0:
    r = a % b
    a = b
    b = r
    pași = pași + 1
print("CMMDC =", a)
print("Pași:", pași)
```

Explicații

- se adaugă o variabilă `pași` care se incrementează la fiecare iterație;
- pentru $a=48, b=18$:
 - pas 1: $r=48\%18=12, a=18, b=12$
 - pas 2: $r=18\%12=6, a=12, b=6$
 - pas 3: $r=12\%6=0, a=6, b=0$
- CMMDC = 6, obținut în 3 pași;

Exemplul 3. Operații cu fracții

Proiectați un program care simplifică o fracție la forma ireductibilă, folosind algoritmul lui Euclid pentru calculul CMMDC-ului. De exemplu, fracția $12/18$ se simplifică la $2/3$.

Exemplu: $a = 12, b = 18$. Ieșire: $2/3$.

Cod Python

```
# a și b numere naturale, b nenul
a = int(input("a = "))
b = int(input("b = "))

x, y = a, b
while y != 0:
    r = x % y
    x = y
    y = r
cmmdc = x
print(a // cmmdc, "/", b // cmmdc)
```

Explicații

- se salvează copii ale lui a și b în x și y pentru a calcula CMMDC;
- se aplică Euclid cu împărțiri repetate:
 - $x=12, y=18 \rightarrow r=12, x=18, y=12$
 - $x=18, y=12 \rightarrow r=6, x=12, y=6$
 - $x=12, y=6 \rightarrow r=0, x=6, y=0$
- CMMDC = 6;
- se împart a și b la CMMDC: $12/6 = 2, 18/6 = 3$;
- rezultat: fracția $2/3$.

**APLICAȚII**

1. Extindeți Exemplul 3 pentru a realiza și adunarea a două fracții: $a_1/b_1 + a_2/b_2 = (a_1*b_2 + a_2*b_1) / (b_1*b_2)$, apoi simplificați rezultatul.
2. O cameră are dimensiunile a și b (în metri). Se dorește acoperirea podelei cu plăci de gresie pătrate cât mai mari. Determinați latura maximă a plăcii (CMMDC)

și numărul de plăci necesare. Exemplu: $a = 36, b = 24$, latura = CMMDC(36, 24) = 12, număr plăci = $(36/12) * (24/12) = 6$

3. Evaluați diferența de complexitate între parcurgerea divizorilor de la 1 la n , de la 1 la $n/2$ și de la 1 la $\sqrt{n}$. Pentru $n = 100$, câte verificări se fac în fiecare caz?



1.2.3. Prelucrarea numerelor și descompunerea în factori primi

Orice număr natural mai mare decât 1 poate fi scris ca produs de numere prime. Această scriere se numește **descompunere în factori primi** și este unică pentru fiecare număr.

De exemplu, $60 = 2 \times 2 \times 3 \times 5$, care se poate scrie și sub forma: $60 = 2^2 \times 3^1 \times 5^1$.

Pentru a descompune un număr în factori primi, se procedează astfel: se începe cu cel mai mic număr prim, 2, și se împarte numărul la 2 cât timp este posibil. Apoi se trece la următorul divizor posibil și se repetă procesul, până când numărul devine 1.



PROBLEME REZOLVATE

Exemplul 1. Verificarea dacă toate cifrele sunt pare

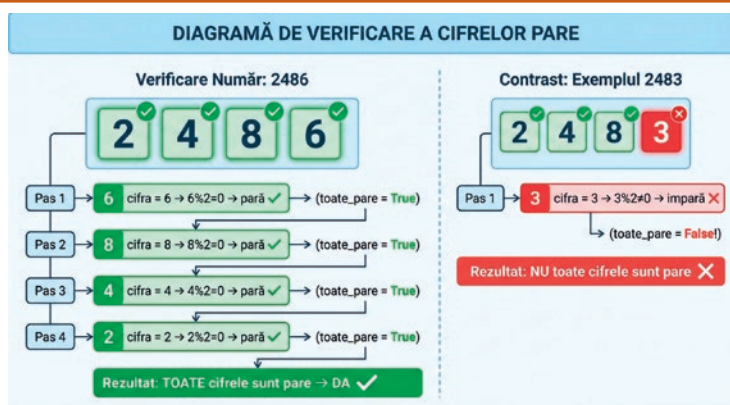
Se citește n . Să se verifice dacă toate cifrele numărului sunt pare.

Exemplu: $n = 2486 \rightarrow \text{DA}$; $n = 2483 \rightarrow \text{NU}$.

Cod Python

```
n = int(input("n = "))
toate_pare = True
while n > 0:
    cifra = n % 10
    if cifra % 2 != 0:
        toate_pare = False
    n = n // 10
if toate_pare:
    print("DA")
else:
    print("NU")
```

Fig. 8 – Verificarea proprietăților cifrelor



Exemplul 2. Construirea numărului din cifrele pare

Se citește n . Să se construiască numărul format doar din cifrele pare ale lui n , păstrând ordinea lor. Exemplu: $n = 132648$, rezultatul este 2648.

Cod Python

```
n = int(input("n = "))
rezultat = 0
putere = 1
while n > 0:
    cifra = n % 10
    if cifra % 2 == 0:
        rezultat = rezultat + cifra * putere
        putere = putere * 10
    n = n // 10
print(rezultat)
```

Explicații

- se parcurg cifrele de la dreapta;
- dacă cifra este pară, se adaugă la rezultat folosind variabila putere;
- putere se înmulțește cu 10 la fiecare cifră pară adăugată;
- pentru $n = 132648$:
 cifra=8 (pară): rezultat=8, putere=10
 cifra=4 (pară): rezultat=48, putere=100
 cifra=6 (pară): rezultat=648, putere=1000
 cifra=2 (pară): rezultat=2648, putere=10000
 cifra=3 (impară): se sare
 cifra=1 (impară): se sare
 – rezultat: 2648.



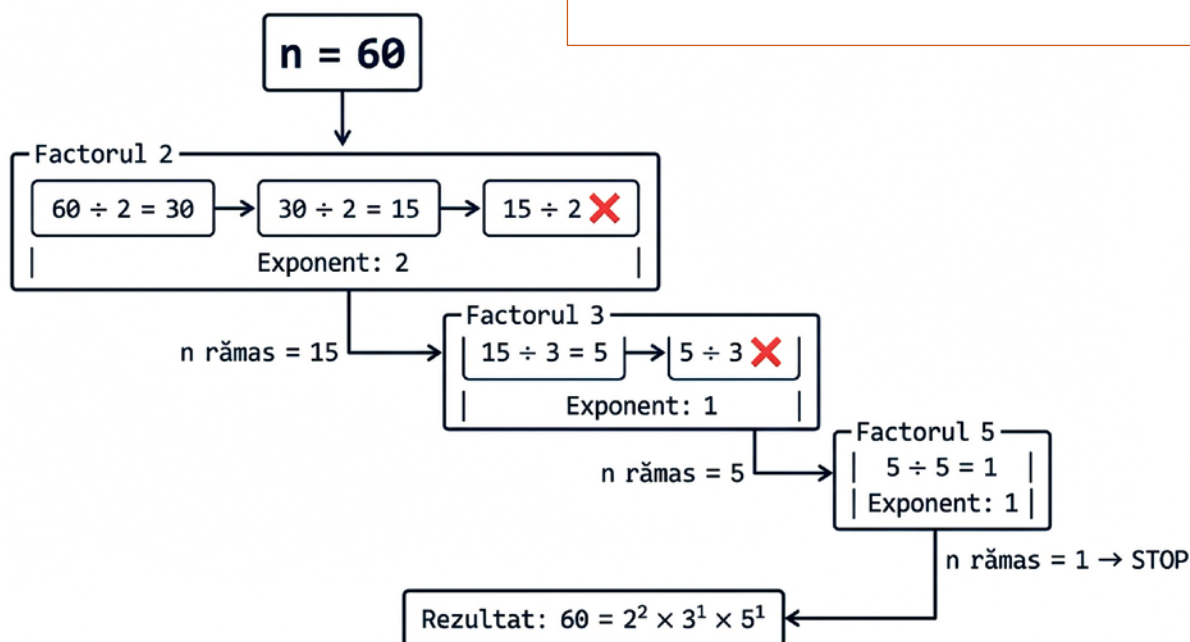
PROBLEME REZOLVATE

Exemplul 3. Descompunerea în factori primi

Se citește n ($n \geq 2$). Să se afișeze descompunerea lui n în factori primi.

Exemplu: $n = 60$. Ieșire: $2^2, 3^1, 5^1$ (în consolă, afișarea este pe verticală).

Fig. 9 – Descompunerea în factori primi



Cod Python

```

n = int(input("n = "))
d = 2
while n > 1:
    e = 0
    while n % d == 0:
        n = n // d
        e = e + 1
    if e > 0:
        print(str(d) + "^" + str(e))
    d = d + 1
  
```



FIȘĂ DE LUCRU 2

1. Se citește n . Să se numere câte cifre nenule ale lui n sunt și divizori ai lui n . De exemplu, pentru $n = 1232$, cifrele 1, 2, 2 sunt divizori, iar 3 nu este → rezultat: 3.

2. Se citește un număr natural n ($n \geq 2$). Să se determine cel mai mic divizor prim al lui n și de câte ori apare acesta în descompunerea în factori primi.
Exemplu: $n = 60 \rightarrow$ cel mai mic divizor prim este 2, apare de 2 ori (deoarece $60 = 2^2 \cdot 3 \cdot 5$).



IMPORTANT

Instrucțiunea `print(str(d) + "^" + str(e))` afișează valorile variabilelor d și e sub forma d^e , folosind `str()` cu rolul de a transforma numerele în text, pentru a fi concatenate cu operatorul `+`.

3. Se citesc două numere naturale a și b . Folosind algoritmul lui Euclid cu împărțiri repetate, să se afișeze cel mai mare divizor comun al lor și numărul de pași efectuați.

Exemplu:

$a = 252, b = 105 \rightarrow \text{CMMDC} = 21, \text{pași} = 3$.



R E C A P I T U L A R E

1.3. Recapitulare

1. Pentru fiecare afirmație de mai jos, să se stabilească dacă este adevărată sau falsă.

- ☐ Operația $n \% 10$ returnează prima cifră a numărului n .
- ☐ Funcția `input()` returnează întotdeauna o valoare numerică.
- ☐ Algoritmul lui Euclid cu împărțiri repetate este mai eficient decât cel cu scăderi.
- ☐ Operatorul `=` realizează atribuirea unei valori unei variabile.
- ☐ Un număr prim are exact doi divizori: 1 și el însuși.
- ☐ `Oglindit(oglinde(n))` este întotdeauna egal cu n .

2. Se consideră următorul program.

Să se precizeze ce va afișa algoritmul pentru $nr = 1234$, urmărind execuția pas cu pas.
Să se modifice algoritmul, astfel încât să calculeze produsul cifrelor impare.

Cod Python

```
nr = int(input("nr = "))
suma = 0
while nr > 0:
    c = nr % 10
    if c % 2 == 0:
        suma = suma + c
    nr = nr // 10
print(suma)
```

MINIPROIECT – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVİ

1. Se citesc două numere naturale a și b . Să se verifice dacă cele două numere sunt **prietene**, adică suma divizorilor lui a (mai puțin a însuși) este egală cu b , iar suma divizorilor lui b (mai puțin b însuși) este egală cu a .

Exemplu: $a = 220, b = 284$. Divizorii lui 220, mai puțin 220: 1, 2, 4, 5, 10, 11, 20, 22, 44, 55, 110 → suma = 284. Divizorii lui 284, mai puțin 284: 1, 2, 4, 71, 142 → suma = 220.

Rezultat: cele două numere sunt prietene.

Să se analizeze problema, precizând datele de intrare și datele de ieșire.

Să se elaboreze algoritmul în pseudocod și implementarea acestuia în Python.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E



1.4. Evaluare sumativă

I. Rezolvă.

1. Să se asocieze fiecare termen din coloana A cu definiția corespunzătoare din coloana B.

5x0,2 p. = 1 p.

Coloana A	Coloana B
Pseudocod	O zonă de memorie în care se poate stoca o valoare ce poate fi modificată în timpul execuției programului.
Constantă	O formă intermediară de reprezentare a unui algoritm, între limbajul natural și un limbaj de programare.
Expresie logică	O valoare care nu se modifică pe parcursul execuției programului.
Variabilă	Împărțirea unui algoritm în subprobleme independente, fiecare cu un rol clar.
Proiectare modulară	O combinație de valori și operatori care produce un rezultat de tip True sau False .

2. Să se evalueze următoarele expresii Python, precizând valoarea și tipul rezultatului: 5x0,2 p. = 1 p.

- a) `int(3.9) + float(2)` b) `(7 + 3) * 2 // 6` c) `100 % 7 % 3`
 d) `not (10 == 10) or 3 != 4` e) `20 // 3 > 6 or 20 % 3 == 0`

3. Se citește `n` (`n > 1`). Să se verifice dacă `n` este **număr perfect**, adică dacă suma divizorilor lui `n`, mai puțin `n` însuși, este egală cu `n`. Implementați verificarea.

2 p.

Exemplu: n = 6, divizorii lui 6 mai puțin 6 sunt 1, 2, 3, iar 1 + 2 + 3 = 6 → DA, este perfect.

4. Se consideră următorul program scris în limbajul Python. Să se determine ce valoare se afișează la finalul execuției programului și să se precizeze rolul variabilei `ok` în cadrul algoritmului.

```
x = 1
y = 0
ok = True
while x <= 5:
    if x % 2 == 0:
        y = y + x
    else:
        ok = not ok
    if ok == True:
        y = y + 1
    x = x + 1
print(y)
```

- a) 5
b) 6
c) 7
d) 8

1 p.

II. **Miniproiect:** Miniproiectul rezolvat în cadrul secțiunii de Recapitulare.

1x2 p. = 2 p.

III. **Portofoliu:** la finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul `Unitate1_NumePrenumeElev`, următoarele rezolvări: *Fișa de lucru 1* și *Fișa de lucru 2*.

2x1 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*. Se acordă 1 punct din oficiu.

2.1. Subprograme

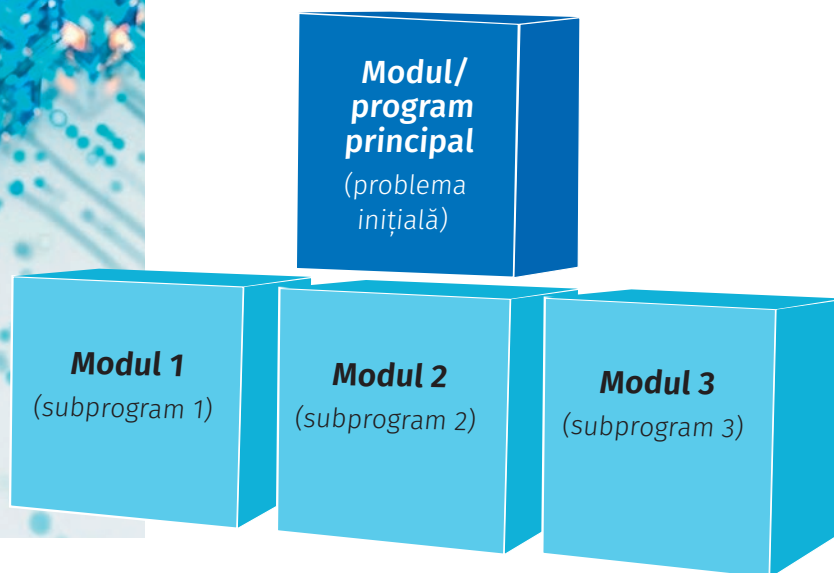
2.1.1. Subprograme – rol, caracteristici

În dezvoltarea aplicațiilor informatice, pe măsură ce problemele devin tot mai complexe, este necesară organizarea eficientă a codului pentru a-l face mai ușor de înțeles, de testat și de reutilizat.

Strategia de bază este de a descompune problema inițială în **subprobleme (module)**, fiecare dintre ele având responsabilități unice și bine delimitate. Descompunerea poate continua până când se obține un modul cu rezolvare imediată sau un modul de complexitate mică. Această tehnică de proiectare modulară a algoritmilor poartă denumirea de tehnica **top-down**.

Fiecare subproblemă va fi descrisă printr-un **subprogram**.

În exemplul de mai jos, o problemă a fost descompusă în trei subprograme.



Clasificarea modulelor (subprogramelor) după ierarhie:

► **Modul (subprogram) apelant** – este modulul care, pentru rezolvarea propriei probleme, *apelează* la alte module. În momentul *apelului*, modulul apelant, *transferă controlul* modulului apelat, pentru a fi executat. În diagrama anterioară, *Modulul principal* este modul apelant.

► **Modul (subprogram) apelat** – este modulul care este *apelat* de un alt modul, pentru a-i rezolva sarcina. După ce își termină execuția, modulul apelat *redă controlul* modulului apelant. În diagrama anterioară *Modul 1*, *Modul 2* și *Modul 3* sunt module apelate.



STUDIU DE CAZ 1

Enunțul problemei

– Să se utilizeze rețeta „Sos de roșii” pentru a prepara o pizza.

Analiza problemei:

Rețeta „Sos de roșii” – are rolul de a prepara acest tip de sos utilizând un set clar și logic de pași. Sosul este o „parte” folosită în prepararea mai multor feluri de mâncare (pizza, paste, lasagna) și are o rețetă cunoscută (este scrisă deja).

Analogia cu informatica:

Rețeta „Sos de roșii” = subprogram (modul) apelat; rețeta pizza (paste/ lasagna) = program (modul apelant).



STUDIU DE CAZ 2

Enunțul problemei – Se consideră un șir de n (n – număr natural nenul) valori numere naturale nenule. Să se determine numărul de valori din șir care au suma cifrelor un număr prim.

Analiza problemei:

- ▶ *date de intrare:* n și elementele șirului (fiecare element notat cu a)
- ▶ *date de ieșire:* numărul nr al valorilor din șir care respectă condiția din enunț

Proiectarea modulară a algoritmului: Pentru fiecare valoare din șir, avem același tip de calcul: suma cifrelor, iar pentru fiecare sumă obținută se verifică dacă este număr prim sau nu.

Descompunerea problemei în subprograme:

Modul1 (subprogram1) – calculează suma cifrelor unui număr natural nenul dat.

Modul2 (subprogram2) – verifică dacă un număr natural nenul dat este număr prim.

Modul principal (program principal) – pentru fiecare valoare citită, se utilizează Modul1 pentru calculul sumei cifrelor, iar pentru suma obținută se utilizează Modul2 – pentru a testa dacă suma este un număr prim.

Proiectare: Reprezentarea algoritmului în limbaj pseudocod

```
Modul principal – definiție modul apelant
//date de intrare – n și a: naturale
//date de ieșire – nr: natural
//date de manevră – x, y, i: naturale

citește n
nr ← 0
pentru i ← 1, n execută
    citește a
    //apelează Modul1 pentru valoarea a,
    //iar suma cifrelor lui a se memorează
    //în variabila x
    x ← apel Modul1(a)
    //apelează Modul2 pentru valoarea x,
    //iar y reține rezultatul
    //subprogramului (1 sau 0) după cum
    //x este număr prim sau nu
    y ← apel Modul2(x)
    dacă y = 1
        atunci nr ← nr + 1
    sf_dacă
sf_pentru
scrie nr
```

Modul1(a) – definiție modul apelat

```
// date de intrare – a: natural
//date de ieșire – suma: natural

suma ← 0
cât_timp a ≠ 0 execută
    suma ← suma + a mod 10
    a ← a div 10
sf_cât_timp
Ieșire cu suma din Modul1
```

Modul2(x) – definiție modul apelat

```
//date de intrare – x: natural
//date de ieșire – prim: natural

prim ← 1
dacă x < 2
    atunci prim ← 0
sf_dacă
d ← 2
cât_timp d * d ≤ x AND prim = 1 execută
    dacă x mod d = 0
        atunci prim ← 0
    sf_dacă
    d ← d + 1
sf_cât_timp
Ieșire cu prim din Modul2
```



Fluxul de execuție – ordinea executării instrucțiunilor într-un program care conține subprograme:

Fluxul de execuție urmează o structură logică precisă, de sus în jos, dar cu „sărituri” între liniile de cod atunci când subprogramele sunt apelate. Iată etapele principale ale acestui proces:

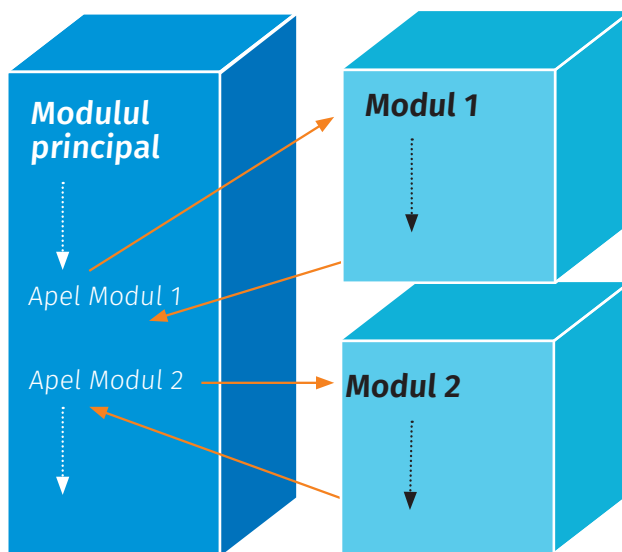
1. Execuția modulului principal – instrucțiunile din interior sunt executate secvențial (una după alta).

2. Apelarea unui subprogram – când întâlnește un apel de subprogram (exemplu: *Apel Modul1(a)*), execuția în punctul curent este suspendată temporar și controlul algoritmului „sare” la prima linie din interiorul subprogramului apelat.

3. Execuția corpului subprogramului – instrucțiunile din interiorul subprogramului sunt executate secvențial.

4. Returnarea controlului – are loc în două situații:

- ▶ returnează o valoare și oprește imediat execuția subprogramului.
- ▶ ajunge la final: subprogramul se termină după executarea ultimei instrucțiuni.



În ambele situații, controlul programului se întoarce exact în locul de unde a fost făcut apelul, continuând cu următoarea instrucțiune a modulului principal.



APLICAȚII

Propune o variantă de rezolvare modulară, în limbaj pseudocod, pentru problemele de mai jos.

1. Să se calculeze numerele „mistice” mai mici decât un număr n natural nenul dat (numărul natural x se numește număr „mistic” dacă suma cifrelor care îl compun este egală cu produsul lor; exemplu: 22, 13131).
2. Se consideră datele despre n cumpărături (n – număr natural nenul), iar pentru fiecare cumpăratură se cunosc **prețul** (un număr real nenul) și **numărul de bucăți** (un număr natural nenul). Să se calculeze costul total al cumpărăturilor.

Concluzii și caracteristici generale

- ▶ Subprogramul **este reprezentat de un set de instrucțiuni care rezolvă o anumită sarcină**, este identificabil printr-un **nume** și poate fi **lansat în execuție (apelat)** ori de câte ori este cazul.
- ▶ Pentru a lucra cu un subprogram avem nevoie de: **definiția subprogramului** și de **apelul** acestuia.
- ▶ Un subprogram este lansat în execuție (apelat) în cod executabil.
- ▶ Cele două subprograme (apelat și apelant) pot comunica prin intermediul unor **parametri de comunicare**.
- ▶ În cele mai multe dintre limbajele de programare, subprogramele sunt plasate „înaintea” secțiunii de instrucțiuni a programului propriu-zis (numit, de obicei, **program principal**).
- ▶ Un subprogram are un grad mare de generalitate pentru problema pe care o rezolvă. El poate fi utilizat în mai multe aplicații și poate fi înglobat într-o bibliotecă a limbajului de programare.

2.1.2. Subprograme definite de utilizator – implementare în limbajul Python

Noțiuni teoretice

În limbajul Python, subprogramele se numesc **funcții**.

Clasificarea subprogramelor

– **în funcție de autorul lor:**

- ▶ **Subprograme predefinite (standard)** – sunt subprograme care sunt deja definite în bibliotecile limbajului de programare. Programatorul apelează (utilizează) aceste subprograme.
- ▶ **Subprograme definite de utilizator (nestandard)** – sunt subprograme definite și apelate de către programator.

Pentru a lucra cu funcții definite de utilizator, avem nevoie de:

- ▶ Definiția funcției;
- ▶ Apelul funcției.

Definiția unei funcții respectă următoarea sintaxă:

```
def nume_funcție(listă parametri formali):
```

```
    bloc de instrucțiuni
```

Antetul funcției ■

Corpul funcției ■

Apelul unei funcții respectă următoarea sintaxă:

```
nume_funcție (listă parametri efectivi)
```

nume_funcție – este numele funcției stabilit la definiția acesteia;

listă parametri efectivi (sau **argumente**) – valorile acestora sunt cunoscute.

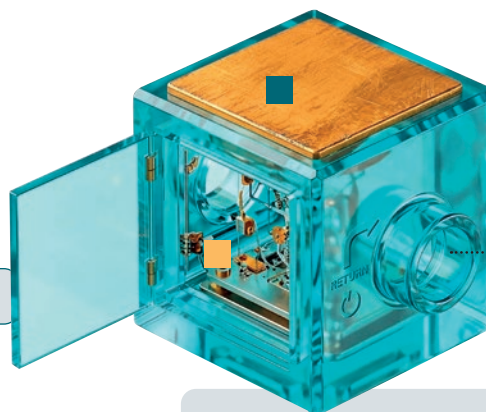
La apel, parametrii formali primesc, de la stânga la dreapta, valorile din lista parametrilor efectivi.

def – cuvânt rezervat, introduce definiția funcției;

nume_funcție – este numele funcției, este stabilit de programator și respectă regula formării unui identificator în limbajul Python;

listă parametri formali – este o listă de parametri prin care subprogramul comunică cu subprogramul apelant. Această listă este opțională, iar în cazul în care lipsește, setul de caractere **()**: este obligatoriu;

bloc de instrucțiuni – alcătuit dintr-un set de instrucțiuni care este executat în momentul în care funcția este apelată. Fiecare linie, aparținând blocului de instrucțiuni, păstrează o indentare de aceeași lungime.



Return

O funcție poate returna o valoare, sau mai multe valori, către subprogramul apelant, utilizând instrucțiunea **return** <expresii>. Instrucțiunea **return** oprește execuția unei funcții, controlul este transferat subprogramului apelant. Dacă o funcție nu conține nicio instrucțiune **return** sau conține doar **return** fără a indica o expresie, atunci funcția returnează valoarea specială **None** (constantă care indică absența unei valori, este definită în biblioteca interpretorului Python).

> > > EXEMPLUL 1

Cod Python

```
#Definiție funcție
def Functie_1():
    #Funcție fără parametri, afișează un mesaj
    print("Prima mea funcție")

#Program principal
Functie_1()
```

Exemplu de rulare

Prima mea funcție

Explicații

- Antetul funcției este `def Functie_1():`
- Numele funcției este `Functie_1`
- Nu există o listă a parametrilor formali, ca atare, nu există o listă a parametrilor efectivi.
- Definiția funcției este plasată înaintea subprogramului care conține instrucțiunea de apel.
- Apelul funcției `Functie_1()` apare în corpul subprogramului apelant (aici programul principal). La întâlnirea instrucțiunii de apel are loc executarea funcției.

> > > EXEMPLUL 2

Enunțul problemei: Să se scrie în limbajul Python un program care convertește în *grade Celsius* (notat cu °C) **trei valori reale citite de la tastatură**, reprezentând *grade Fahrenheit* (notat cu °F).

Rezolvarea se va realiza în două variante: **una cu funcții și alta fără funcții.**

Formule de conversie ale unei valori reale x :

Din grade Fahrenheit în grade Celsius	Din grade Celsius în grade Fahrenheit
$x \text{ } ^\circ\text{F} = (x - 32) * 5/9 \text{ } ^\circ\text{C}$	$x \text{ } ^\circ\text{C} = (x * 9/5 + 32) \text{ } ^\circ\text{F}$

Analiza problemei:

- *Date de intrare* – valorile reale f_1 , f_2 , f_3 (reprezintă valori în °F)
 - *Date de ieșire* – trei valori reale (reprezintă conversia datelor de intrare în °C)
- Gradul Fahrenheit și gradul Celsius sunt unități de măsură pentru temperatură.*

Rezolvare
fără funcții

```
f1=float(input("Prima valoare in grade F: "))
f2=float(input("A doua valoare in grade F: "))
f3=float(input("A treia valoare in grade F: "))

celsius1=(f1 - 32)*5/9
celsius2=(f2 - 32)*5/9
celsius3=(f3 - 32)*5/9

print("Prima valoare in grade C:",celsius1)
print("A doua valoare in grade C:",celsius2)
print("A treia valoare in grade C:",celsius3)
```



Rezolvare
cu funcții

```
def citeste():
    a1=float(input("Prima valoare in grade F: "))
    a2=float(input("A doua valoare in grade F: "))
    a3=float(input("A treia valoare in grade F: "))
    return a1,a2,a3

def Functie_2(fahr):
    celsius=(fahr - 32)*5/9
    return celsius

#Program principal
f1,f2,f3=citeste()
print("Prima valoare in grade C:",Functie_2(f1))
print("A doua valoare in grade C:",Functie_2(f2))
print("A treia valoare in grade C:",Functie_2(f3))
```

Prima valoare în grade F: 95
A doua valoare în grade F: 32
A treia valoare în grade F: 50
Prima valoare în grade C: 35.0
A doua valoare în grade C: 0.0
A treia valoare în grade C: 10.0

Explicații:

Problema enunțată are un grad mic de complexitate, așa încât, rezolvarea cu funcții este o abordare mai degrabă didactică. S-a dorit exemplificarea implementării funcțiilor cu parametri, a funcțiilor care returnează un rezultat sau a funcțiilor care returnează mai multe rezultate.

- Rezolvarea fără funcții – s-a aplicat de trei ori aceeași formulă de conversie, aici observăm un dezavantaj: codul este mai greu de actualizat (de exemplu, dacă se modifică formula de calcul).
- Rezolvarea cu funcții – au fost definite două funcții:

1) funcția `citeste()` – nu are parametri, citește de la tastatură trei valori reale `a1`, `a2` și `a3` și returnează (transmite prin instrucțiunea `return`) programului principal aceste valori, care vor fi memorate, în ordine, în variabilele `f1`, `f2` și respectiv `f3`. Variabilele `a1`, `a2` și `a3` sunt proprii funcției `citeste()` și nu pot fi utilizate în exteriorul funcției.

2) funcția `Functie_2(fahr)` aplică formula de conversie pentru parametrul formal `fahr`, codul este scris o singură dată, iar funcția este apelată de trei ori.

- În variabila `celsius` se calculează rezultatul funcției, rezultat care este returnat (întors) către programul principal prin instrucțiunea `return celsius`.
- Funcția are un parametru formal (`fahr`), ca atare va avea și un parametru efectiv (exemplu `f1`). Nu este obligatoriu ca denumirile acestora să fie identice.
- La întâlnirea apelului `Functie_2(f1)`, se execută următorii pași: parametrul formal `fahr` primește valoarea parametrului efectiv `f1`, apoi are loc executarea propriu-zisă a funcției, cu această valoare, și returnarea rezultatului funcției către programul principal. Acest tip de transmitere a parametrilor se numește transmitere prin valoare.



Exemplu numeric:

f1	Funcție_2(fahr)	fahr	celsius	Operație
95	-	-	-	Citire din subprogramul <code>citeste()</code>
95	<code>Funcție_2(95)=?</code>	95	-	Apel <code>Funcție_2(f1)</code>
95	<code>Funcție_2(95)=?</code>	95	35.0	Calcul <code>celsius=35.0</code>
95	35.0	-	-	Întoarcere în programul principal și scrie <code>35.0</code>

– Aceiași pași se vor executa și în cazul apelurilor `Funcție_2(f2)` și `Funcție_2(f3)`
 – Observăm câteva avantaje: codul, care aplică formula de conversie, este scris o singură dată și utilizat de trei ori. Atunci când testăm secvențe mici de cod, erorile pot fi depistate mai ușor.

Mecanismul de funcționare al limbajului

Python în raport cu gestionarea funcțiilor:

- la întâlnirea definiției unei funcții: interpretorul Python traduce codul într-un cod cunoscut de calculator (bytecode), având ca rezultat înregistrarea în memorie a numelui funcției, fără a executa corpul funcției;
- la întâlnirea apelului unei funcții: interpretorul Python execută corpul funcției.

**IMPORTANT**

- În mod implicit, numărul parametrilor formali este egal cu numărul parametrilor efectivi.
- Transmiterea parametrilor de tip `int`, `float`, `str` se realizează **prin valoare**: parametrul formal primește o copie a valorii parametrului efectiv, astfel încât orice modificare a parametrului formal nu afectează valoarea parametrului efectiv din subprogramul apelant.

>>> **EXEMPLUL 3**

Se consideră următoarea definiție:

```
def F(a,b):
    return a+b
```

Instrucțiuni de apel:

<code>x=7</code> <code>y=3</code> <code>print(F(x,y))</code>	<code>x=7.6</code> <code>y=9.4</code> <code>print(F(x,y))</code>	<code>x="car"</code> <code>y="te"</code> <code>print(F(x,y))</code>	<code>x="car"</code> <code>y=10</code> <code>print(F(x,y))</code>
Rezultat: <code>10</code> , tip <code>int</code>	Rezultat: <code>17.0</code> , tip <code>float</code>	Rezultat: <code>carte</code> , tip <code>str</code>	<i>Eroare de execuție!</i>
Parametrii efectivi sunt de tip întreg (<code>int</code>)	Parametrii efectivi sunt de tip real (<code>float</code>)	Parametrii efectivi sunt de tip șir de caractere (<code>str</code>).	Operatorul <code>"+"</code> nu se poate aplica pe un <code>str</code> și un <code>int</code>

Explicații: Un parametru efectiv, **neavând tipul declarat**, poate fi de orice tip, atâta timp cât instrucțiunile/operațiile din definiția funcției pot fi executate.



PROBLEMĂ REZOLVATĂ



Enunțul problemei: Se citește, de la tastatură, un număr natural. Să se verifice dacă numărul poate fi un CNP valid, afișând un mesaj corespunzător (condiții verificate: numărul trebuie să conțină 13 cifre, luna de naștere trebuie să fie validă).

Analiza problemei:

Date de intrare: valoarea naturală `cnp`;

Date de ieșire: mesaj de validare.

CNP – Cod Numeric Personal, este un cod numeric unic atribuit fiecărei persoane din România; are următoarea formă:

S	A	A	L	L	Z	Z	X	X	X	X	X	X
Secol	An	An	Lună	Lună	Zi	Zi	X	X	X	X	X	X

Proiectare modulară:

Numele funcției	Rolul funcției – indicații de rezolvare
<code>citire_CNP()</code>	Citește un număr natural și returnează valoarea citită. Acesta va fi reținut în variabila <code>cnp</code> declarată în programul principal.
<code>validare_lungime(c)</code>	Funcția primește, prin parametrul <code>c</code> , un număr natural primit la apel în subprogramul <code>validare_CNP</code> . Funcția calculează numărul de cifre ale valorii memorate în parametrul <code>c</code> și returnează 1 sau 0 după cum lungimea acestuia este egală cu 13 sau nu.
<code>validare_luna(c)</code>	Funcția primește, prin parametrul <code>c</code> , un număr natural, primit la apel în subprogramul <code>validare_CNP</code> . Funcția returnează 1 dacă luna este validă, altfel, funcția returnează 0.
<code>validare_CNP(c)</code>	Funcția primește, prin parametrul <code>c</code> , numărul natural citit, prin apel în programul principal. Integrează toate validările și afișează rezultatul validării. Folosește o variabilă declarată în această funcție, inițial cu valoarea 1; valoarea acesteia se modifică la valoarea 0 dacă: – lungimea <code>cnp</code> este diferită de 13 sau – lungimea <code>cnp</code> este egală cu 13, dar luna nu este validă.

În exemplul de mai jos: CNP - invalid (luna nu este validă).

<code>validare_lungime(c)</code>												
13	12	11	10	9	8	7	6	5	4	3	2	1
1	7	8	1	5	2	4	2	4	0	0	5	2
<code>validare_luna(c)</code>												

> > >

Program Python

```
#Definiții funcții
def citire_CNP():
    return int(input("Introduceți CNP:"))

def validare_lungime(c):
    nr_cifre=0
    while(c>0):
        nr_cifre+=1
        c=c//10
    if(nr_cifre==13):
        return 1
    else:
        return 0

def validare_luna(c):
    ll=(c//10**8)%100
    if ll<1 or ll>12:
        return 0
    return 1

def validare_CNP(c):
    lungime=validare_lungime(c)
    luna=validare_luna(c)
    validare=1
    if lungime==0:
        print("CNP nu are 13 cifre")
        validare=0
    if lungime==1 and luna==0:
        print("CNP nu are luna valida")
        validare=0
    if validare == 1:
        print("CNP valid")
    else:
        print("CNP invalid")

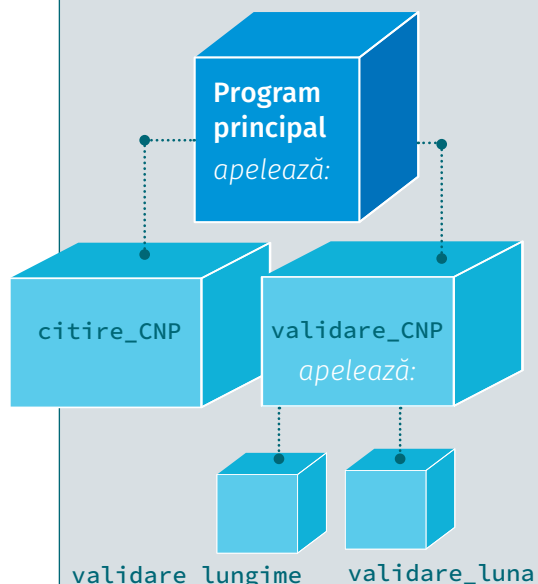
#Program principal
cnp = citire_CNP()
validare_CNP(cnp)
```

Exemplu de rulare

Introduceți CNP: 2781230500342
CNP valid

Introduceți CNP: 1801024780
CNP nu are 13 cifre
CNP invalid

Reprezentarea grafică de mai jos evidențiază structura logică a aplicației, dar și dependența și relația de subordonare între subprograme.



Activitate în echipă

Integrează alte două funcții programului prezentat anterior:
a) o funcție pentru validarea primei cifre (cuprinsă între 1 și 8);
b) o funcție pentru validarea zilei de naștere (în funcție de luna nașterii).

Avantajele utilizării subprogramelor

- ▶ **reutilizarea codului** – aspect extrem de important în eficientizarea dezvoltării aplicațiilor, codul este scris o singură dată, dar subprogramul poate fi apelat de câte ori este nevoie;
- ▶ **depanarea și actualizarea aplicației se fac mai ușor** – este mult mai ușor să se testeze, să se corecteze și să se efectueze modificări în secvențe mici de cod, decât la nivelul întregii aplicații;
- ▶ **favorizează lucrul în echipă** – membrii echipei pot lucra simultan la sarcini diferite;
- ▶ **crearea de aplicații lizibile** – codul devine mai ușor de urmărit și de înțeles;
- ▶ crește **portabilitatea programelor** – subprogramele realizate pot fi preluate de alte programe/aplicații care necesită rezolvarea de sarcini similare.

Câteva dezavantaje ale utilizării subprogramelor

- ▶ necesită o proiectare mai atentă;
- ▶ dificil pentru începători.



APLICAȚII

Să se implementeze, în limbajul Python, din Secțiunea 2.1.1:

1. Algoritmul prezentat în cadrul Studiului de caz 2.
2. Cei doi algoritmi, propuși spre rezolvare modulară, de la finalul secțiunii 2.1.1.

2.1.3. Variabile locale, variabile globale

Noțiuni teoretice. În cadrul organizării modulare a unui program, este foarte importantă zona din program în care o variabilă *acționează*, este *recunoscută și poate fi folosită*. Această zonă (porțiune) de program poartă numele de **domeniu de vizibilitate**.

Din acest punct de vedere, în limbajul Python, variabilele se pot clasifica astfel:

1 VARIABLE LOCALE – așa cum am văzut într-o lecție anterioară, sunt create în corpul unei funcții și nu pot fi utilizate din exteriorul funcției. La finalul execuției subprogramului, spațiul alocat acestor variabile este eliberat (variabilele locale sunt distruse). Parametrii unei funcții sunt considerați variabile locale.

2 VARIABLELE GLOBALE – sunt create în exteriorul definiției oricărei funcții, pot fi utilizate de programul principal, dar și de toate funcțiile definite în program. La finalul execuției programului, spațiul alocat acestor variabile este eliberat (variabilele globale sunt distruse). Există, însă, un caz în care o variabilă globală poate fi creată **în cadrul definiției unei funcții**: dacă variabila este declarată cu instrucțiunea **global**.

Reguli de acces

- ▶ o funcție are acces la toate variabilele globale (pentru *modificare* se utilizează instrucțiunea **global**; se *păstrează* orice modificare efectuată asupra variabilelor globale);
- ▶ dacă într-o funcție se declară o variabilă cu *același nume* ca și cea globală, atunci în acea funcție va funcționa variabila declarată local (pentru acces la variabila globală se poate defini variabila împreună cu instrucțiunea **global**).

Program principal



Funcție



>>> EXEMPLE

```
x=1      #x variabilă globală
```

```
def ex1(a): #a var. locală
```

```
    a=a+5
```

```
    print(a)
```

```
x=x+2    #variabilă globală
```

```
ex1(x)    #afișează 8
```

```
print(x)  #afișează 3
```

#a)

```
x=1      #x variabilă globală
```

```
def ex2(a): #a variabilă locală
```

```
    x=a+2    #x variabilă locală
```

```
    print(x)
```

```
ex2(x)      #afișează 3
```

```
print(x)    #afișează 1
```

#b)

```
x=1      #x var. globală
```

```
def ex3():
```

```
    global x #x var. globală
```

```
    x=x+1
```

```
    print(x)
```

```
ex3()      #afișează 2
```

```
print(x)    #afișează 2
```

#c)

```
x=1      #x var. globală
```

```
def ex4():
```

```
    x=x+1    #EROARE la executie
```

```
    #x var. locală
```

```
    print(x)
```

```
ex4()
```

```
print(x)
```

#d)



ACTIVITATE ÎN ECHIPĂ

Enunțul problemei:

Să se rescrie codul problemei rezolvate din Secțiunea 2.1.2 (validare CNP), utilizând funcții care prelucrează variabile globale.

Indicații de rezolvare:

Funcțiile implementate sunt fără parametri și pot **accesa datele globale**

(**cnp** – declarată în programul principal), fără a folosi instrucțiunea **global**, atât timp cât valorile respective rămân neschimbate. Evită modificarea variabilei globale **cnp**.

IMPORTANT



Comunicarea dintre un subprogram și subprogramul său apelant se poate face prin intermediul **parametrilor** și/sau al **variabilelor globale**. Tehnica utilizării parametrilor în locul variabilelor globale asigură independența unui subprogram față de alte subprograme și integritatea datelor (evită modificarea accidentală a datelor).

✓ PARAMETRI ÎN FUNCȚII



VS

✗ VARIABILE GLOBALE



Parametri
în funcții
vs. variabile
globale



SIGURANȚĂ



PREVIZIBILITATE

UȘOR DE ÎNTREȚINUT
ȘI DE TESTATDATE MODIFICATE
FĂRĂ CONTROLREZULTATE
IMPREVIZIBILECOD GREU DE
ÎNTREȚINUT
ȘI DE TESTAT



2.1.4. Funcții predefinite Python

Documentația Bibliotecii Standard oferă o prezentare detaliată a diferitelor componente ale interpretorului Python, incluzând tipurile de date, funcțiile, modulele și alte elemente recunoscute de acesta.

Funcțiile predefinite, pe care le regăsim în biblioteca Python, le putem clasifica în:

1 Funcții încorporate (**built-in**) în biblioteca standard

Ai utilizat (apelat) deja unele funcții de acest tip: `print()`, `int()`, `range()`.

2 Funcții definite în **module încorporate (built-in)** în biblioteca standard (exemplu modulul `math`).

Un modul, în limbajul Python, este o colecție de instrucțiuni și definiții de funcții, organizate într-un fișier. Pentru a accesa o funcție, definită într-un modul Python, avem nevoie să utilizăm directiva `import` urmată de numele modulului, iar apelul unei funcții va fi de forma `nume_modul.nume_funcție()`.

2.1.4.1. Funcții predefinite utile în *prelucrări matematice*

Nume funcție	Tip argument	Tip rezultat	Rezultatul funcției. <i>Funcție built-in</i> sau funcție în modulul <i>math</i> .	Exemple (→ =returnează)
<code>sqrt(arg)</code>	<code>int/float</code> (nenegative)	<code>float</code>	Rădăcină pătrată din <code>arg</code> . Modul <i>math</i> .	<code>math.sqrt(16)</code> → 4.0 <code>math.sqrt(2.25)</code> → 1.5
<code>abs(arg)</code>	<code>int/float</code>	<code>int/float</code>	Valoarea absolută a <code>arg</code> . Funcție built-in.	<code>abs(-9.5)</code> → 9.5 <code>abs(9)</code> → 9
<code>fabs(arg)</code>	<code>float</code>	<code>float</code>	Valoarea absolută a <code>arg</code> . Modul <i>math</i> .	<code>math.fabs(-2.5)</code> → 2.5 <code>math.fabs(2)</code> → 2.0
<code>round(arg1 [, arg2])</code>	<code>arg1: int/float</code> <code>arg2: int</code> (poate lipsi)	<code>int/float</code>	Numărul <code>arg1</code> rotunjit la <code>arg2</code> zecimale. Dacă <code>arg2</code> lipsește, se obține cel mai apropiat întreg de <code>arg1</code> . Funcție built-in.	<code>round(2.67, 1)</code> → 2.7 <code>round(-2.19, 1)</code> → -2.2 Caz particular - la valori exact la jumătate, rotunjește către cel mai apropiat întreg par: <code>round(2.5)</code> → 2 <code>round(1.5)</code> → 2
<code>floor(arg)</code>	<code>float</code>	<code>int</code>	Cel mai mare număr întreg mai mic sau egal cu <code>arg</code> . Modul <i>math</i> .	<code>math.floor(2.9)</code> → 2 <code>math.floor(-2.4)</code> → -3
<code>ceil(arg)</code>	<code>float</code>	<code>int</code>	Cel mai mic număr întreg mai mare sau egal cu <code>arg</code> . Modul <i>math</i> .	<code>math.ceil(2.9)</code> → 3 <code>math.ceil(-2.4)</code> → -2
<code>trunc(arg)</code>	<code>float</code>	<code>int</code>	Partea zecimală se elimină, lăsând partea întreagă. Modul <i>math</i> .	<code>math.trunc(-2.8)</code> → -2 <code>math.trunc(2.15)</code> → 2

2.1.4.2. Funcții predefinite *built-in de conversie explicită*, pe care le-ai studiat deja în Capitolul 1:

`int()`, `float()` și `str()`.

- *Conversia explicită* – utilizează funcții predefinite de conversie. Este posibilă numai dacă se respectă formatul tipului de dată țintă.
- *Conversia implicită* – convertește automat un tip de date în altul, fără utilizarea funcțiilor predefinite de conversie (exemplu: de la `int` la `float`, de la `bool` la `int`). Acest tip de conversie se realizează de obicei către tipul de date mai larg.

#pentru a concatena un număr la un str #conversie explicită: din int în str <code>a = str(35)+"xy" #a devine 35xy</code>	
#pentru a aduna un str la un număr #conversie explicită: din str în float <code>a = float('12.5')+1.5</code>	#Conversie implicită <code>a = 3 #a de tip int</code> <code>b = 3.14 #b de tip float</code> <code>c = a + b #c de tip float</code>
#Eroare la conversie explicită <code>a = int('3.14')</code>	

2.1.4.3. Funcții predefinite *built-in* utile în prelucrarea secvențelor de date

Nume funcție	Tip argumente	Tip rezultat	Rezultatul funcției	Exemple
<code>min(arg1, arg2, ...)</code>	<code>int/float</code>	<code>int/float</code>	Cea mai mică valoare dintre 2 sau mai multe argumente.	<code>min(6,7,2.1) → 2.1</code>
<code>max(arg1, arg2, ...)</code>	<code>int/float</code>	<code>int/float</code>	Cea mai mare valoare dintre 2 sau mai multe argumente.	<code>max(6,7,2.1) → 7</code>

>>> EXEMPLU

Enunțul problemei:

Să se scrie un program, în limbajul Python, care să calculeze distanța dintre două puncte în planul axelor carteziene: $A(x_a, y_a)$, $B(x_b, y_b)$. Coordonatele reale ale celor două puncte se citesc de la tastatură.

Cod Python

```
#instrucțiune de import
import math
xa=float(input("xa = "))
ya=float(input("ya = "))
xb=float(input("xb = "))
yb=float(input("yb = "))
d=math.sqrt((xa-xb)**2+(ya-yb)**2)
print("Distanța:", d)
```

Exemplu de rulare

```
xa = 3
ya = 4
xb = 0
yb = 0
Distanța: 5.0
```



Avantajele utilizării subprogramelor predefinite: crește lizibilitatea codului, reduce dimensiunea codului, funcțiile predefinite sunt optimizate și testate.



FIȘĂ DE LUCRU 3



1. Să se ruleze, în limbajul Python, secvențele de program de mai jos și justifică rezultatul:

```
# punctul a)
x = -2.5
y = 1.5
print(round(abs(x)+float(int(y))))
```

```
# punctul c)
import math
print(math.trunc(2.59))
print(int(2.59),math.floor(2.59))
print(round(2.59,1),math.ceil(2.59))
```

```
# punctul b)
a=str(1)+str(2)
b=int("1")+int("2")
print("a=",a)
print("b=",b)
```

```
# punctul d)
a=int(input("a="))
b=int(input("b="))
x=max(a-b,b-a)==abs(a-b)
print(x)
```

2. Se consideră codul Python de mai jos:

```
def cifre():
    y=n
    while y > 9:
        y = -----
    return ----

# program principal
n = int(input("n = "))
if -----:
    -----
else:
    -----
```

a) Completează spațiile libere astfel încât programul să afișeze mesajul **Adevărat**, dacă numărul natural n , citit de la tastatură, are prima cifră egală cu ultima cifră. Dacă nu îndeplinește această condiție, să afișeze mesajul **Fals**.

b) Modifică programul obținut la punctul a) astfel încât programul să afișeze suma tuturor numerelor naturale $\leq n$ care au prima cifră egală cu ultima cifră. Funcția `cifre` va primi, prin parametrul `c`, un număr natural $\leq n$.

3. Se citesc scorurile (valori naturale) obținute de trei elevi la un campionat de șah. Să se determine scorul maxim și scorul minim (cu și fără aplicarea funcțiilor `max()` și `min()`).

4. Se consideră trei puncte în planul axelor carteziane: $A(x_a, y_a)$, $B(x_b, y_b)$ și $C(x_c, y_c)$.

Să se verifice dacă cele trei puncte formează un triunghi, utilizând două funcții Python: o funcție care citește coordonatele reale ale unui punct în plan și returnează datele citite și o funcție care returnează distanța dintre două puncte.

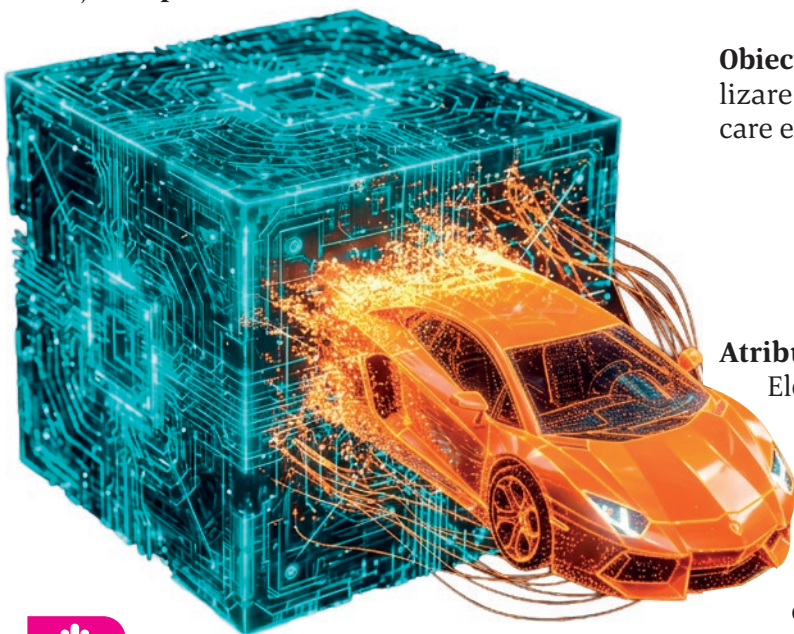
5. Să se scrie un program în Python care citește de la tastatură trei note, numere naturale. Să se definească două funcții `f1(a,b,c)` – returnează media aritmetică a celor trei note, cu două zecimale, fără rotunjire, și `f2(a,b,c)` – returnează media aritmetică a celor trei note, cu rotunjire la cel mai apropiat întreg. Să se testeze funcțiile în programul principal, pentru seturile de note (9,9,8), (7,7,8).



2.2. Introducere în programarea orientată pe obiecte în limbaj de programare



Programarea orientată pe obiecte (POO) a apărut din necesitatea de a dezvolta programe mai clare, mai bine organizate și mai ușor de întreținut pe termen lung, capabile să gestioneze aplicații complexe, prin modelarea elementelor din lumea reală. Noutatea adusă de acest concept constă în organizarea problemei sub forma unor **obiecte** care înglobează atât **caracteristici**, cât și **comportamente**.



IMPORTANT

Clasă = șablon; **Obiect** = instanță (exemplar). Obiectele au **atribute** (date) și **metode** (acțiuni). Accesul la metode se face cu `.(punct): obiect.metoda()`.



APLICAȚII

Modelează clasa **Carte**, completând următoarele informații: numele clasei, minimum 3 atribute, minimum 3 metode. Creează două obiecte de tip **Carte** (două instanțe), folosind valori concrete pentru fiecare atribut în parte.

2.2.1. Noțiuni de bază în POO

Clasa – este un **șablon** sau un model după care se creează obiecte. Ea descrie ce **atribute** (date, caracteristici) și ce **metode** (acțiuni, comportamente) vor avea obiectele.

Exemplul 1: Clasa **Masina** descrie toate mașinile, nu o mașină anume;

Exemplul 2: Clasa **Elev** descrie toți elevii, la modul general;

Obiectul – este o **instanță a unei clase**, adică o realizare concretă a șablonului. Obiectele sunt cele care există efectiv în program.

Exemple: O mașină verde, marca Dacia, carburant Diesel – este un obiect al clasei **Masina**, iar un elev din clasa a IX-a, numit Andrei, în vârstă de 15 ani – este un obiect al clasei **Elev**.

Atributele – sunt **caracteristicile** unui obiect. Ele rețin informațiile utile ale obiectelor.

Exemple: pentru **Masina**: **culoare**, **carburant**, **marca** etc; pentru **Elev**: **clasa**, **nume**, **varsta**

Metodele – **acțiunile** pe care un obiect le poate realiza. Sunt similare cu funcțiile definite de utilizator.

Exemplul 1: (pentru clasa **Masina**)
`porneste()`, `acelereaza()`,
`opreste()` etc.

Exemplul 2: (pentru clasa **Elev**)
`mergeLaScoala()`, `invata()`,
`participaLaOre()`.

Caracteristici

- Datele și funcțiile sunt grupate în același loc (în clasă); codul devine mai ușor de extins (adaugi ușor noi metode: absențe, teză).
- **Avantaje:** organizare mai bună pentru programe mai mari; ușor de lucrat cu multe entități similare (elevi, produse, conturi); reutilizare: aceeași clasă poate fi folosită în mai multe programe.
- **Dezavantaje:** la început pare mai „mult de scris”, trebuie înțelese clasele/obiectele.



>>> EXEMPLU

Se consideră următorul scenariu. Există două mașini:

- mașina1: culoare verde, marca Dacia, carburant Diesel;
- mașina2: culoare roșie, marca BMW, carburant Benzină.

Ac acestea au următoarele acțiuni:

- pornește mașina1 și o accelerează cu 30 km/h;
- pornește mașina2 și o accelerează cu 50 km/h;
- oprește mașina1 (viteza devine 0).

Cod Python - Implementarea POO

```
class Masina:
    # Constructor: __init__ este o metodă specială a unei clase
    # Rol constructor: se apelează când creăm un obiect nou
    def __init__(self, culoare, marca, carburant):
        # Atribute (caracteristici) ale obiectului
        self.culoare = culoare
        self.marca = marca
        self.carburant = carburant
        # Stări inițiale
        self.viteza = 0
        self.pornita = False
    def porneste(self):
        # Metodă: pornim mașina curentă (self = obiectul curent)
        self.pornita = True
        print(f"{self.marca} a pornit.")

    def accelereaza(self, delta):
        # Metodă: accelerează mașina curentă
        if not self.pornita:
            print("Nu poti accelera: masina este oprita.")
            return

        self.viteza += delta
        print(f"{self.marca} accelereaza la {self.viteza} km/h.")

    def opreste(self):
        # Metodă: oprește mașina curentă
        self.pornita = False
        self.viteza = 0
        print(f"{self.marca} s-a oprit.")

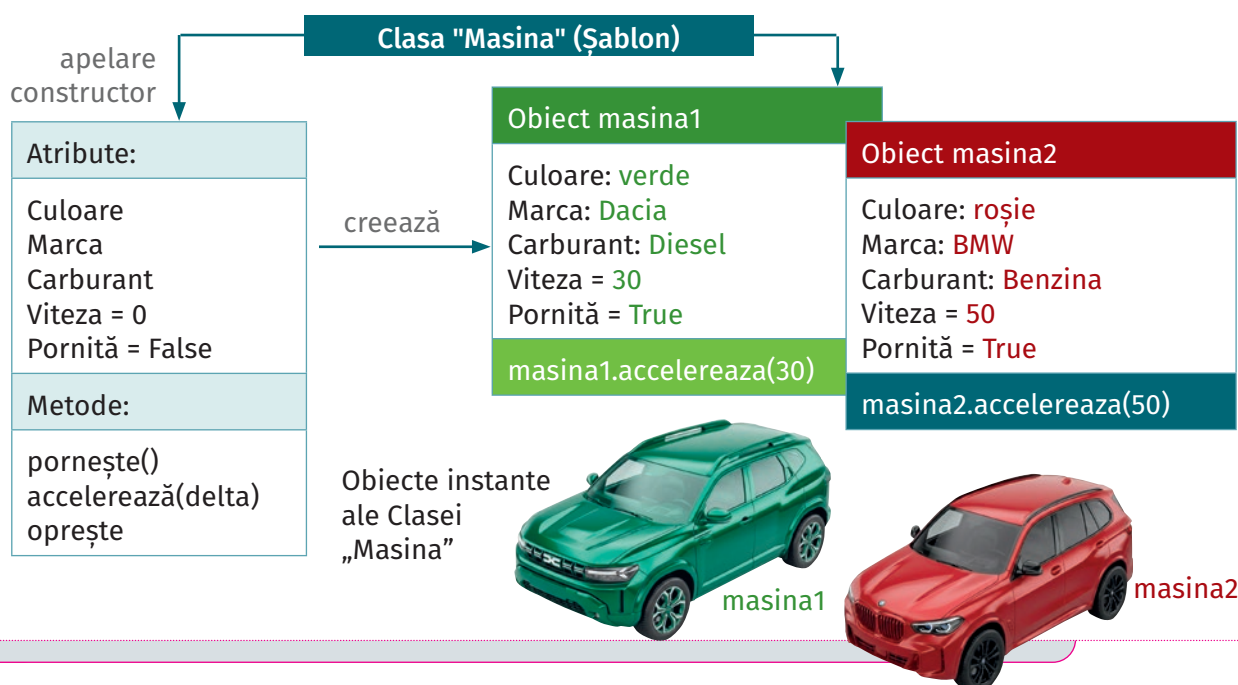
# ----- creare obiecte -----
# mașina1 și mașina2 sunt obiecte (instanțe) ale clasei Masina
masina1 = Masina("verde", "Dacia", "Diesel")
masina2 = Masina("rosie", "BMW", "Benzina")
masina1.porneste() # Dacia a pornit
masina1.accelereaza(30) # Dacia accelerează cu 30 km/h
masina2.porneste() # BMW a pornit
masina2.accelereaza(50) # BMW accelerează cu 50 km/h
masina1.opreste() # Dacia s-a oprit
```



IMPORTANT

În programarea orientată pe obiecte (POO) în Python, `self` se referă la instanța curentă a clasei, adică obiectul concret asupra căruia lucrezi. Rolurile lui `self`:

- ▶ – acces la atributele obiectului: `self.atribut`;
- ▶ – diferențiază variabilele locale de cele ale obiectului: `culoare` = parametru primit, `self.culoare` = atribut al obiectului;
- ▶ – permite apelarea altor metode din aceeași clasă.



2.2.2. Clase predefinite în limbajul Python

Limbajul de programare Python conține multe clase utile, numite clase predefinite, prezentate în cele ce urmează.

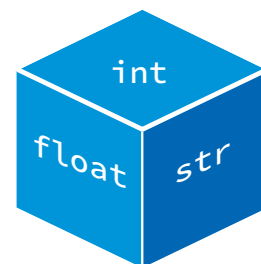


IMPORTANT

În Python, `int`, `float`, `str` sunt *imutabile*, ceea ce înseamnă că valoarea unui obiect din unul din aceste tipuri nu se poate modifica după ce acesta a fost creat. De exemplu, dacă avem `x = 10` și apoi scriem `x = x + 1`, nu se modifică obiectul `10`, ci se creează un nou obiect `11`, iar variabila `x` va face referire la acesta.

Clasele „int”, „float”, „str”

În Python, `int`, `float` și `str` nu sunt doar tipuri de date, ci și clase care definesc comportamentul acestor date. Atunci când scriem `x = 10`, nu doar atribuim o valoare, ci creăm de fapt un obiect (o instanță) al clasei `int`. Cu alte cuvinte, `10` este rezultatul instanțierii clasei `int`, iar variabila `x` face referire la acest obiect. Acest lucru reflectă principiul fundamental din Python conform căruia aproape totul este un obiect, iar valorile au atât date, cât și metode asociate.



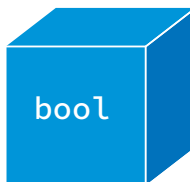
Clasa „list”

În Python, `list` este o clasă care definește comportamentul listelor. Când scriem `l = [1, 2, 3]`, de fapt creăm un obiect (instanță) al clasei `list`. Echivalent, putem scrie și `l = list([1, 2, 3])`, ceea ce arată mai clar instanțierea clasei. Variabila `l` face referire la acest obiect, iar el are metode proprii, cum ar fi `l.append(4)`, care modifică lista. Astfel, la fel ca `int`, și `list` este o clasă, iar valorile de tip listă sunt obiecte create din ea.



Clasa „bool”

În Python, un `bool` (tipul `bool`) reprezintă o valoare logică: `True` sau `False`. `Bool` apare în orice expresie logică și atunci când verificăm condiții precum: Numărul este mai mic decât 10? (`True/False`) sau Parola este corectă? (`True/False`).



IMPORTANT

În Python, *bibliotecile și modulele* sunt colecții de cod deja scris, pe care programatorii le pot folosi direct.

- Un *modul* este un fișier Python care conține instrucțiuni și funcții, de exemplu, modulul pentru calcule matematice `math`.
- O *bibliotecă* este o colecție mai mare de module, clase care oferă funcționalități pentru diverse domenii, de exemplu, biblioteca pentru interfețe grafice `Tkinter`.



FIȘĂ DE LUCRU 4

1. Pentru următoarele clase:

- Să se identifice: numele clasei, atributele, metodele, obiectele create, și mesajele afișate în consolă.
- Să se creeze 3 noi obiecte, având diverse valori pentru câmpurile definite.
- Ce alte atribute și metode putem adăuga claselor date?



Coduri Python

```
class Produs:
    def __init__(self, nume, pret):
        self.nume = nume
        self.pret = pret
    def afiseaza(self):
        print(self.nume, self.pret)
p1 = Produs("Paine", 5)
p2 = Produs("Lapte", 7)
p1.afiseaza()
p2.afiseaza()
```

```
class Semafor:
    def __init__(self, culoare):
        self.culoare = culoare
    def actiune(self):
        if self.culoare == "verde":
            print("Treci!")
        elif self.culoare == "galben":
            print("Atentie!")
        else:
            print("Oprește!")
s = Semafor("rosu")
s.actiune()
```

2. Folosind modelul claselor anterioare, să se definească clasa `Elev` cu atributele `nume`, clasa și metoda `salut()`, care afișează: „Sunt elevul X din clasa Y”. Să se creeze 2 elevi și să se apeleze metoda.

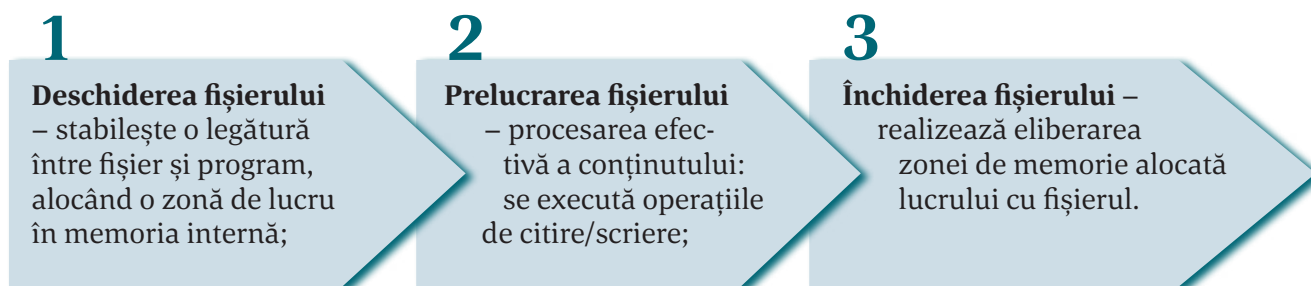
3. Să se creeze clasa `Telefon` cu atributele `brand`, `baterie` și metoda `incarca()`, în care bateria crește cu 10, în cazul în care valoarea acesteia este mai mică decât 100.



2.3. Fișiere text



Principii de lucru cu fișiere text
– există trei etape de lucru, realizate în următoarea ordine:



2.3.2. Implementarea fișierelor text în limbajul Python

2.3.2.1. Deschiderea fișierelor text

Se utilizează funcția *built-in* `open()`, cu următoarea sintaxă:
`nume_logic_fis=open(nume_fizic_fis, mod_desch, encoding="utf-8")`

IMPORTANT



Primii doi parametri sunt de tip șir de caractere. Pentru situația în care fișierul conține strict caractere ASCII, al treilea parametru este opțional.

Exemplu: `f=open("date.txt", "r")`

Deschide fișierul text, cu numele fizic `date.txt`, și returnează instanța unui obiect de tip fișier al clasei predefinite `TextIOWrapper`, atribuie noul obiect creat variabilei `f` (`f` numele logic al fișierului); fișierul este deschis în vederea operației de citire.

Operația de deschidere a unui fișier text:

- ▶ stabilește o relație între numele fizic al fișierului și numele logic al acestuia; după operația de deschidere, se va opera în codul Python cu numele logic;
- ▶ stabilește operația permisă asupra fișierului indicată prin modul de deschidere.

Moduri de deschidere disponibile pentru fișierele text:

Caracterul	Semnificație
<code>r</code>	Deschide fișierul în vederea operației de citire (este modul implicit, poate să lipsească). Fișierul trebuie să existe în folderul de lucru curent (sau în calea explicită, dacă este precizată), în caz contrar, la execuție, se obține eroarea <code>FileNotFoundError</code> .
<code>w</code>	Deschide fișierul în vederea operației de scriere. Se va crea un nou fișier în folderul de lucru curent (sau în calea explicită, dacă este precizată). Dacă fișierul există, se va realiza suprascriere.
<code>x</code>	Deschide fișierul pentru scriere, cu crearea unui nou în folderul de lucru curent (sau în calea explicită, dacă este precizată), în caz că nu există. Dacă fișierul există, la execuție se obține eroarea <code>FileExistsError</code> .
<code>a</code>	Deschide fișierul în vederea operației de adăugare la sfârșitul fișierului, dacă există. Dacă fișierul nu există, se va crea unul nou în folderul de lucru curent (sau în calea explicită, dacă este precizată).
<code>t</code>	Deschide fișierul în mod text (modul implicit și poate să lipsească).



IMPORTANT

- ▶ Numele fizic al fișierului poate să conțină și calea explicită către fișier.

Exemplu:

`f=open("D:/date.txt", "w")`
fișierul va fi creat în rădăcina discului `D`.

- ▶ La deschiderea unui fișier, **poziția de citire** (pe care o denumim **cursor** sau **pointer**) se va afla pe **primul caracter, aflat pe poziția 0**.
- ▶ Datele pe care le citim/scriem din/în fișiere text sunt de tip șir de caractere.



2.3.2.2. Prelucrarea fișierelor text

Metodele pe care le putem utiliza pentru prelucrarea unui fișier de tip text le regăsim în clasa predefinită `TextIOWrapper`.

Pentru referirea la o metodă vom folosi sintaxa:

`nume_logic_fișier.metoda()`.

» Citirea datelor dintr-un fișier text (`mod_deschidere` este "`r`")

a) Metoda `read(valoare)` – unde `valoare` este un număr întreg, funcția returnează șirul de caractere citit.

- dacă `valoare` este negativ sau lipsește, atunci întregul conținut al fișierului este citit și returnat de funcție ca șir de caractere;
- dacă `valoare` este pozitiv, atunci se citește un număr de caractere egal cu `valoare`. Funcția returnează șirul de caractere citit.

Atenție!

- Caracterele *linie nouă* ("`\n`"), aflate în fișierul text, se memorează în șirul returnat de funcția `read()`.
- Sfârșitul de fișier (EOF) se detectează prin *șirul vid* (șirul care nu conține niciun caracter: "`''`")

b) Metoda `readline()` – citește o singură linie din fișier, caracterul *linie nouă* ("`\n`") se memorează la sfârșitul șirului de caractere (mai puțin pentru ultima linie, în cazul în care nu există acest caracter). Funcția returnează șirul de caractere citit.

» Adăugarea datelor într-un fișier text (`mod_deschidere` este "`a`")

Se utilizează metoda `write(sir_de_caractere)` pentru adăugare la sfârșitul fișierului.

» Scrierea datelor într-un fișier text (`mod_deschidere` este "`w`")

Metoda

`write(sir_de_caractere)` – scrie în fișier șirul de caractere specificat și returnează numărul de caractere scrise. Obiectele de alt tip, cum ar fi valorile numerice, necesită convertirea la tipul șir de caractere.

2.3.2.3. Închiderea fișierelor text

După ce am prelucrat fișierul, urmează operația de închidere cu metoda `close()`.

Această operație este extrem de importantă din următoarele motive: se eliberează resursele sistemului pe care prelucrarea fișierului le-a blocat și se elimină eventualele erori sau pierderi de date ce pot să apară la prelucrarea fișierelor.

>>> EXEMPLE

Pentru următoarele exemple, *fișierul text* **date.txt**, are trei linii de text, editate fără diacritice, și nu are caracterul *linie nouă* ("`\n`") la final.

date.txt

31 28
Ianuarie Februarie
Gerar Faurar

Cod Python

```
#Deschide fișierul pentru citire, citește întregul
#conținut, îl memorează în variabila sir
f=open("date.txt","r")
sir=f.read()
print(sir)
f.close() #închide fișierul
```

Exemplu de rulare

31 28
Ianuarie Februarie
Gerar Faurar



Cod Python	Exemplu de rulare
<pre>#Deschide pentru citire f=open("date.txt") #Citește șirul alcătuit din primele 5 caractere sir=f.read(5) print(sir) f.close()</pre>	31 28
<pre>#Deschide și citește fișierul pe linii f=open("date.txt") #Citește prima linie, are la final "\n" sir=f.readline() while sir!="": #test de EOF #Afiseaza linia citită curent print(sir) # Citește următoarea linie #dacă nu este ultima, are la final "\n" sir=f.readline() f.close()</pre>	31 28 Ianuarie Februarie Gerar Faurar
<pre>#Deschide și citește fișierul pe linii f=open("date.txt") for sir in f: print(sir) f.close()</pre>	31 28 Ianuarie Februarie Gerar Faurar
<pre>#Deschide pentru scriere, crează un nou fișier #dacă există, suprascrie f=open("rezultate.txt","w") f.write("Prima linie\n") f.write("A doua linie") #trebuie să convertim valoarea numerică la str f.write(str(12)) f.close()</pre>	Fișierul rezultate.txt: Prima linie A doua linie12
<pre>#Deschide pentru adăugare la sfârșitul fișierului f=open("rezultate.txt","a") f.write("\n") f.write("A treia linie") f.close()</pre>	Fișierul rezultate.txt: Prima linie A doua linie12 A treia linie



INVESTIGAȚIE - lucru în echipă

Scop: Compararea timpului de executare între un program care citește datele de la tastatură și unul care citește datele din fișier; se analizează impactul privind performanța programului.
Formă de organizare – grupe de câte doi elevi.

Mod de desfășurare: Considerând cele două coduri Python de mai jos, să se stabilească timpul de executare pentru fiecare variantă. Se vor utiliza aceleași date de intrare: 1,2,3,4,...,15. În fișierul text, datele sunt scrise câte unul pe fiecare rând.

Indicație: S-a utilizat metoda `time()`, din modulul predefinit `time`, care returnează un număr real (timpul curent, exprimat în secunde, de la momentul de referință 1 Ianuarie 1970).



```
#Varianta 1 - citire tastatură
import time
start = time.time()
n = 15
for i in range(n):
    a = int(input())
stop = time.time()
print(stop - start, "secunde")
```

```
#Varianta 2 - citire din fișier
import time
start = time.time()
n = 15
f=open("date.txt")
for i in range(n):
    a =int(f.readline())
f.close()
stop = time.time()
print(stop - start, "secunde")
```

Analiza datelor și concluzii: Discutați cu profesorul vostru despre rezultatele obținute și formulați concluziile, completând (pe caiet) un tabel ca cel de mai jos.

Criteriu	Tastatură	Fișier
Timp aproximativ de execuție (secunde)		
Viteză de execuție (lentă/rapidă)		
Dependență de utilizator (Da/Nu)		
Potrivit pentru volum mare de date (Da/Nu)		

2.3.2.4. O tehnică de bună practică în lucrul cu fișierele text

Există situații în care un program conține o eroare la interpretare și atunci metoda de închidere a fișierului nu se mai execută. O bună practică de lucru este utilizarea instrucțiunii **with**, care are marele avantaj că **fișierul este închis automat**. În cadrul instrucțiunii **with**, toate instrucțiunile referitoare la prelucrarea fișierului păstrează o aceeași indentare.

>>> EXEMPLE

Cod Python

```
#Deschide fișierul pentru citire
with open("date.txt", "r") as f:
    sir = f.readline()
    while sir != "":
        print(sir)
        sir = f.readline()
#fișierul este automat închis

#Deschide fișierul pentru scriere
with open("rezultate.txt", "w") as f:
    f.write("Prima linie\n")
    f.write("A doua linie")
    f.write(str(12))
#fișierul este automat închis

#Deschide fișierul pentru adăugare
with open("rezultate.txt", "a") as f:
    f.write("\n")
    f.write("A treia linie")
#fișierul este automat închis
```

Exemplu de rulare

```
31 28

Ianuarie Februarie

Gerar Faurar
```

Fișierul rezultate.txt:

```
Prima linie
A doua linie12
A treia linie
```

**IMPORTANT**

După ce un obiect fișier a fost închis (fie apelând funcția `close()`, fie prin utilizarea instrucțiunii `with`), orice încercare de a prelucra fișierul va genera eroare. Rezolvăm această problemă, dacă deschidem din nou fișierul.

**AUTOEVALUARE**

```
with open("date.txt", "r") as f:
    n = int(-----)

g = open("rezultate.txt", "w")
for i in range(1, -----):
    if -----:
        g.write(-----)
    -----
```

Să se completeze spațiile libere astfel încât programul de mai sus să citească un număr natural nenul `n` din fișierul text `date.txt` și să scrie, în fișierul `rezultate.txt`, toți divizorii numărului `n` pe prima linie, despărțiți de câte un spațiu. Să se testeze în Python codul obținut.

Barem de evaluare: se vor acorda câte 2 puncte pentru fiecare spațiu liber completat corect. Total punctaj: 10 puncte.

**PROBLEMĂ
REZOLVATĂ**

Enunț: O companie care dezvoltă senzori pentru monitorizarea calității aerului dorește să testeze modulul *software de analiză a datelor*. Pentru simulare, se vor genera automat `n` ($n \geq 1$) valori numerice aleatoare, reprezentând concentrația zilnică a unui anumit poluant (valorile sunt cuprinse între 1 și 99). Programul trebuie să memoreze cele `n` valori în fișierul text `date.txt` și să determine câteva informații statistice: valoarea maximă înregistrată și numărul zilelor în care nu s-a depășit pragul valorii de 50.

Analiza problemei: Se va utiliza modulul predefinit `random`, care conține definiția mai multor funcții cu rol în generarea valorilor aleatoare. Una dintre aceste funcții este `randint(a,b)`, unde `a` și `b` sunt numere întregi, $a \leq b$; funcția returnează un număr aleator întreg cuprins între `a` și `b`.

Cod Python

```
import random # instrucțiune de import
n = int(input("n="))
maxv=0
nrz=0
with open("date.txt", "w") as f:
    for i in range(n):
        # apelul funcției randint
        #returneaza un nr aleator cuprins între 1 si 99
        a=random.randint(1,99)
        if a>maxv:
            maxv=a
        if a<=50:
            nrz+=1
        # scriere in fisier
        f.write(str(a))
        f.write("\n")
print("Cea mai mare valoare înregistrată:", maxv)
print("Nr de zile cu valori de cel mult 50:", nrz)
```

date.txt

```
97
40
40
94
12
45
1
32
10
29
```

Exemplu de rulare

```
n=10
Cea mai mare valoare înregistrată: 97
Nr de zile cu valori de cel mult 50: 8
```





2.3.2.5. Alte metode utile în prelucrarea fișierelor text



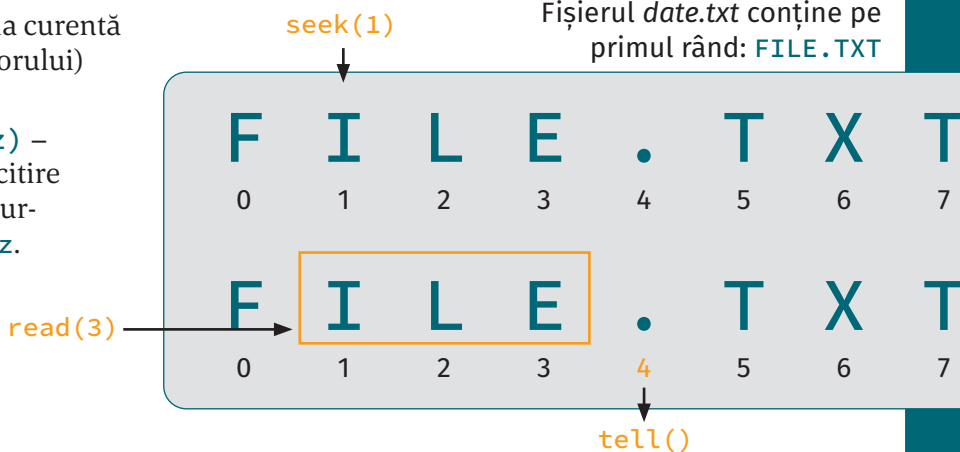
1 Metoda `tell()`
– returnează poziția curentă a pointerului (cursorului) de citire.

2 Metoda `seek(poz)` – mută pointerul de citire la poziția `poz` și returnează valoarea `poz`.

EXEMPLU

> > >

Fișierul `date.txt` conține pe primul rând: `FILE.TXT`



Cod Python

```
with open("date.txt") as f:
    f.seek(1) #poziționare pe al 2-le caracter
    #citește 3 caractere, le memorează în variabila sir
    #și avansează pointerul pe următorul caracter
    sir=f.read(3)
    print(sir)
    print(f.tell())
```

Exemplu de rulare

```
ILE
4
```



FIȘĂ DE LUCRU 5

1. Să se testeze codul de mai jos, în situațiile precizate, și să se justifice efectul obținut.

```
contor=0
with open("date.txt", "r") as f:
    sir = f.readline()
    while sir != "":
        contor += 1
        print(int(sir))
        sir = f.readline()
print("Fișierul are",contor,"linii")
```

- a) Fișierul `date.txt` nu există în folder-ul de lucru curent.
- b) Fișierul `date.txt` nu are conținut (este gol).
- c) Fișierul `date.txt` are următorul conținut:

varianta 1	varianta 2	varianta 3
23	23	23
ab		1
17	17	17

2. Să se verifice dacă două fișiere text conțin același număr de linii, afișând un mesaj corespunzător.

3. În fișierul `date.txt` se află mai multe numere naturale scrise pe linii diferite.

Să se verifice dacă numerele din fișierul `date.txt` sunt în ordine crescătoare, afișând pe ecran mesajul *Sunt crescătoare*, respectiv *Nu sunt crescătoare*, în caz contrar.

RECAPITULARE



2.4. Recapitulare

1. Să se scrie un program în Python care citește de la tastatură soldul unui client al unei bănci comerciale. Să se definească două funcții: una care actualizează soldul după depunerea unei sume în cont și alta care determină soldul după retragerea unei sume (dacă este posibil). Să se testeze funcționarea ambelor funcții în programul principal.
2. Să se definească o funcție Python care să rezolve ecuația de gradul al doilea cu coeficienți reali, citiți de la tastatură. Funcția primește ca parametri coeficienții ecuației, iar apelul acesteia se va realiza în programul principal.
3. Se citesc de la tastatură n numere naturale (n – natural, $1 \leq n \leq 10$). Să se verifice dacă numărul format cu prima cifră a fiecărui număr citit este palindrom. Pentru rezolvare se vor defini cel puțin două funcții.
4. Să se construiască un program în limbajul Python care să genereze 30 de numere întregi aleatoare din intervalul $[-50, 50]$ și să scrie aceste numere în fișierul `date1.txt`, câte unul pe rând. Să se copieze valorile negative din fișierul `date1.txt` în fișierul `date2.txt`, pe prima linie despărțite printr-un spațiu. În cazul în care, în fișierul `date1.txt` nu există valori negative, atunci se va scrie în fișierul `date2.txt` mesajul `Nu exista`.

V
A
L
U
A
R
E

MINIPROIECT – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVİ

1. Să se proiecteze o aplicație modulară Python, de tip calculator matematic, pentru cele patru operații de bază: $+$, $-$, $*$, $/$, aplicate pe date numerice de tip real. Veți implementa un meniu care să permită selectarea, în mod repetat, a operației dorite. Cerințe:
 - a. Să se descompună problema în cel puțin 4 subprograme și să se stabilească specificațiile fiecărui subprogram: rol, parametri, valori returnate; să se realizeze proiectarea modulară în limbaj pseudocod.
 - b. Să se implementeze în limbajul Python soluția problemei.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E

2.5. Evaluare sumativă

V

A

L

U

A

R

E

I. Rezolvă:

1. Să se precizeze afirmația care descrie corect o funcție în Python? 0,5 p.
- Este o variabilă specială care memorează valori reale.
 - Este un bloc de instrucțiuni care poate fi apelat de mai multe ori.
 - Este o structură repetitivă.
 - Este un tip de date numeric.
2. Să se stabilească valoarea de adevăr a următoarelor afirmații (Adevărat/Fals). 5x0,3 p. = 1,5 p.
- Definiția unei funcții trebuie să conțină instrucțiunea `return`.
 - O funcție poate să returneze mai multe valori.
 - O variabilă locală poate fi accesată în programul principal.
 - Metoda `read()` citește primul rând dintr-un fișier text și returnează un șir de caractere.
 - Instrucțiunea `f.seek(3)` mută cursorul de citire, în fișierul `f`, pe al patrulea caracter.
3. Se consideră secvența de program de mai jos: 2x0,5 p. = 1 p.

```
f = open("date.txt", "r")
linie = f.readline()
while linie != "":
    print(linie)
```

- Să se identifice eroarea logică din cod.
- Să se rescrie secvența de program astfel încât să se afișeze toate liniile din fișier.

4. În fișierul `date.txt` se află, pe prima linie, un număr natural alcătuit din cel puțin 2 cifre. Se cere să se scrie cifrele numărului, în fișierul `rezultate.txt`, pe prima linie, despărțite printr-un spațiu. 1 p.

5. Să se asocieze fiecare termen din **coloana A** cu definiția corespunzătoare din **coloana B**. 4x0,25 p. = 1 p.

Coloana A	Coloana B
Clasă	Acțiune sau comportament (definit într-o clasă) care va caracteriza un obiect
Obiect	Șablon sau model care descrie atributele și metodele viitoarelor obiecte
Metodă	Dată sau proprietate (definită într-o clasă) care va caracteriza un obiect
Atribut	O realizare concretă a clasei, o instanță a clasei

- II. **Miniproiect.** Miniproiectul rezolvat în cadrul secțiunii de *Recapitulare*. 1x2 p. = 2 p.

- III. **Portofoliu.** La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate2_NumeElev*, următoarele rezolvări: *Fișa de lucru 3* și *Fișa de lucru 5*. 2x1 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*.
Se acordă 1 punct din oficiu.

3.1. Modelul conceptual liniar

3.1.1. Introducere

Oamenii își organizează viața de zi cu zi, fie că își dau seama sau nu, prin intermediul listelor. De la sarcini simple până la obiective importante, acestea devin instrumente esențiale pentru orientarea într-un mediu mereu schimbător, aducând ordine în dezordine și limpezime în momentele de incertitudine.

Probabil cea mai răspândită formă este **lista de cumpărături**. La prima vedere pare un simplu inventar de produse, dar reprezintă de fapt o strategie esențială de organizare și eficiență, ne reîmprospătează memoria, ne ajută, spre exemplu, să respectăm un buget. Pe lângă nevoile materiale, viețile noastre sunt ghidate și de **liste de sarcini** (*to-do lists*), care ne permit să prioritizăm, să delegăm și să urmărim progresul, reduc stresul, sporesc productivitatea și ne oferă satisfacția de a bifa o sarcină îndeplinită.

Clasamentele sunt liste care ordonează elemente pe baza unor criterii specifice: cele mai bune universități, topuri muzicale, clasamentele sportive; ne ajută să înțelegem ierarhiile, să comparăm performanțele și să ne formăm opinii. Ele sunt atât instrumente de informare, cât și de influențare a percepției publice.

Top 6 cărți recomandate adolescenților

De veghe în lanul de secară	(J. D. Salinger)
Să ucizi o pasăre cântătoare	(Harper Lee)
Împăratul muștelor	(William Golding)
Micul Prinț	(Antoine de Saint-Exupéry)
Marile speranțe	(Charles Dickens)
Jurnalul unui adolescent	(Stephen Chbosky)

Nu uităm **listele de dorințe** (*wishlists*), care depășesc sfera necesarului și pătrund în cea a aspirațiilor; nu sunt doar simple colecții de obiecte, ci și imagini ale dorințelor noastre, amintindu-ne constant de ceea ce prețuim și ce ne motivează, jucând un rol important în menținerea entuziasmului și optimismului.

În concluzie, listele sunt mult mai mult decât simple înșiruri de cuvinte. Ele sunt o reflectare a modului în care mintea umană procesează informațiile, prioritizează și planifică. Fie că sunt folosite pentru nevoi practice, pentru gestionarea timpului, pentru exprimarea dorințelor sau pentru înțelegerea lumii, listele sunt instrumente indispensabile care ne ajută să navigăm eficient prin complexitatea existenței cotidiene, aducând ordine, claritate și un sentiment de realizare în fiecare aspect al vieții noastre.

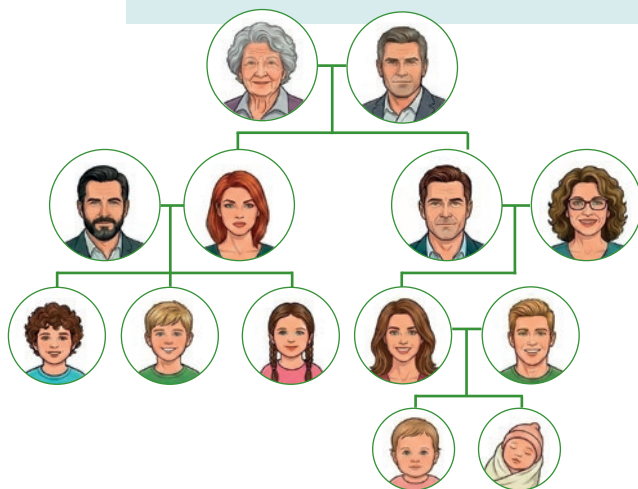


Un **model conceptual** explică modul în care informațiile sunt organizate în memorie și felul în care putem lucra cu ele. Prin intermediul unui astfel de model, datele capătă structură și sens, fiind grupate, relaționate și accesate într-o manieră coerentă. Astfel, el nu doar descrie memoria, ci și facilitează utilizarea ei eficientă, reducând complexitatea și făcând sistemele mai ușor de înțeles și de utilizat.



Modelul ierarhic

- Organigrama unei companii
- Arborele genealogic



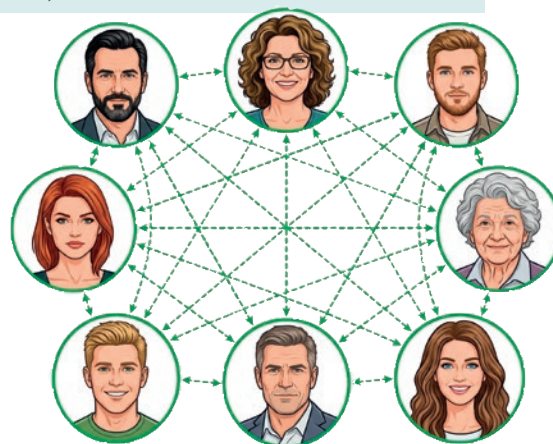
Modele conceptuale frecvent utilizate:

Modelul **liniar** – elementele sunt aranjate într-o **secvență** (liste, șiruri).

- O coadă la supermarket
- Zilele săptămânii:
Lu – Ma – Mi – Jo – Vi – Sâ – Du

Modelul **rețea/grafic** – noduri conectate prin legături multiple.

- Sistemul de drumuri și autostrăzi
- Rețelele de socializare



3.1.2. Lista

Lista este cea mai folosită structură **liniară** de date. Parcurgând exemplele de mai sus, observăm că:

- elementele unei liste sunt, aproape toate, de aceeași natură (spunem „aproape” pentru că, de exemplu, o listă de dorințe poate conține electronice, cărți, îmbrăcăminte, rechizite etc.);
- locul în care sunt trecute în listă poate reprezenta un număr de ordine;
- lista se poate modifica prin ștergerea, modificarea sau adăugarea unor noi elemente.



Din șirul/lista celor 6 baschetbaliști pleacă jucătorul cu indicele 3, iar cei care au rămas își actualizează poziția.



Caracteristicile care definesc lista ca structură liniară de date:

- ▶ **Ordonată** – elementele au o poziție fixă, numită **index**. Asta face ca lista să fie indexabilă, adică elementele să poată fi **accesate direct** prin poziție. În Python, ca și în alte limbaje de programare, **primul element** al listei va fi la poziția/**indexul 0**, ultimul din cele n elemente aflându-se la poziția $n-1$.
- ▶ **Mutabilă** – poate fi modificată, operațiile standard pe care le putem efectua fiind:
 - **inserare**: adăugarea unui element nou;
 - **ștergere**: eliminarea unui element existent;
 - **căutare**: identificarea unui element după valoare sau index;
 - **parcurgere**: vizitarea fiecărui element exact o singură dată;
- ▶ **Omogenă** – adică toate elementele sale au același tip de date (toate sunt întregi, toate sunt caractere etc.), cazul limbajului C;
- ▶ **Eterogenă** – poate conține tipuri diferite de date, precum în Python.



IMPORTANT

În clasa a IX-a folosim doar liste liniare omogene pentru stocarea elementelor de tip numeric sau caracter. În literatura de specialitate sau în alte limbaje de programare, lista în care toate elementele sunt de același tip se numește vector / array, sau tablou unidimensional.



STUDIU DE CAZ 1

Sistemul de Rating 1–5 Stele

Context Real

Un site colectează recenzii de la clienți, ce pot oferi o notă de la **1 la 5**. La finalul unei săptămâni, managerul are o listă cu 1 000 de răspunsuri (exemplu: [5, 4, 5, 2, 1, 5, 4, ...])

Obiectiv

„Suntem un site de 5 stele sau avem probleme?”

Se introduce conceptul de

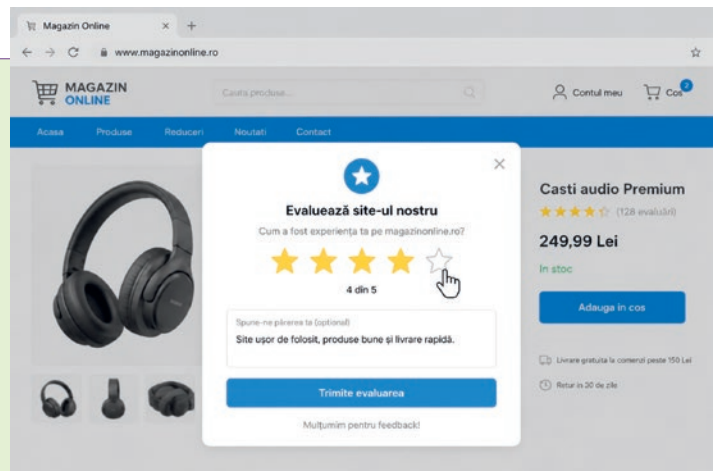
Listă de frecvențe:

o structură în care indexul (sau cheia) reprezintă varianta de răspuns: **căsuța cu indexul i reține/memorează numărul de voturi pentru i stele.**

Identificarea tendințelor generale

Managerul poate trage concluzii imediate:

- **tendința pozitivă**: Dacă la indicii 4 și 5 am valorile cele mai mari, afacerea merge bine.
- **polarizarea**: Dacă avem multe valori la 1 și multe la 5, dar puține la



3, înseamnă că experiența clienților este inconsistentă (ori foarte bună, ori foarte rea).

– **cea mai frecventă notă**: Nota care are valoarea maximă în lista de frecvențe ne indică percepția dominantă a publicului.

De ce această metodă este eficientă?

- ▶ **Viteză (performanță)**: parcurgem lista de răspunsuri **o singură dată**.
- ▶ **Simplitate**: Folosirea indexului listei ca „etichetă” pentru notă face codul foarte curat.
- ▶ **Pregătire pentru vizualizare**: Se poate ușor genera un grafic de tip bare (Histogramă).



STUDIU DE CAZ 2

Prelucrarea bonurilor fiscale într-un hypermarket fără memorarea lor

Un hypermarket procesează zilnic mii de bonuri fiscale generate la casele de marcat. Fiecare bon conține informații despre produsele cumpărate, cantități, prețuri și valoarea totală.

La finalul zilei, conducerea magazinului dorește să afle rapid:

- valoarea totală a vânzărilor;
- numărul total de produse vândute;
- cel mai mare bon fiscal;
- numărul bonurilor cu valoare peste 500 lei.

Volumul de date este însă uriaș. Dacă toate bonurile ar fi memorate simultan într-o listă sau într-un fișier mare încărcat în memorie, consumul de memorie ar deveni foarte mare și prelucrarea mai lentă. Din acest motiv, sistemul folosește **prelucrarea secvențială** a datelor, **fără memorarea** tuturor bonurilor.

Modul de funcționare

„Citește -> Verifică -> Acționează (actualizări/calcul) -> Uită”

Bonurile sunt analizate unul câte unul, imediat după emitere:

- se citește bonul curent;
- se actualizează statisticile necesare;
- informațiile inutile sunt eliminate;
- se trece la următorul bon.

Astfel, în memorie există permanent doar bonul curent și câteva variabile pentru calcule.

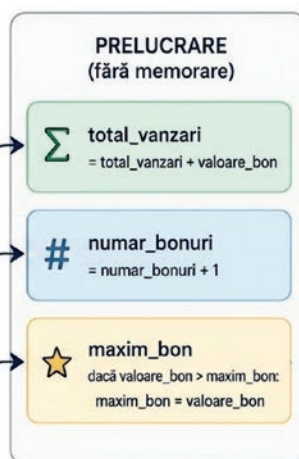
Avantaje:

- consum foarte mic de memorie;
- viteză mare de procesare;
- posibilitatea prelucrării unui număr foarte mare de bonuri;
- sistemul poate funcționa continuu, în timp real.

PRELUCRAREA BONURILOR FISCALE ÎNTR-UN HYPERMARKET FĂRĂ MEMORAREA LOR



Bonurile sunt citite pe rând (unul câte unul) din sistem.



Pentru fiecare bon citit, se actualizează doar câteva variabile (statistici).



La final, sistemul păstrează doar rezultatele calculate, fără a memora bonurile.



Avantaje: economie de memorie, prelucrare rapidă, eficiență crescută.



FIȘĂ DE LUCRU 6



1. Care sunt indicii primului element și respectiv al ultimului element dintr-o listă cu n elemente (n număr natural)?
 a) 0 și $n-1$; b) 1 și n ; c) -1 și $n+1$; d) depinde de listă.
2. Care este indexul elementului din mijloc într-o listă cu 7 elemente?
 a) 2; b) 4; c) 3; d) 3.5.
3. Se dă lista $V = [10, 20, 30, 40, 50]$. Ce valoare se află la $V[2]$?
4. Ce reprezintă expresia $v[n-1]$ într-o listă cu n elemente?
 a) primul element; b) ultimul element; c) eroare; d) penultimul element.
5. Ce se întâmplă dacă modifici în lista v valoarea de la $v[0]$?
 a) ștergi lista; b) toate elementele se decalează cu o poziție;
 c) modifici doar primul element al listei; d) indicele 0 devine invalid.
6. Care este relația dintre poziția p a unui element „al câtelea” și indicele său i în memorie?
 a) nu există nicio relație; b) $i=p+1$; c) $i=p-1$; d) $i=p$.

3.2. Clasa „list” din Python



3.2.1. Crearea unei liste

Pentru lucrul cu secvențe de date de tip listă în *Python* există clasa `list`.

O listă se creează scriind elementele între **paranteze drepte** `"[", "]"`, separate prin virgulă. Lista care nu conține elemente se numește *listă vidă* și poate fi definită prin folosirea de paranteze drepte goale. O listă care conține printre elemente alte liste se numește *listă imbricată*.

EXEMPLUL 1

> > >

```
lv = [] # listă goală sau listă vidă
numere = [1, 2, 3, 4] # listă de numere
nume = ["Alina", "Ion"] # listă de șiruri de caractere
l_eterogena = [[1, 3], "doi", [5, 6, 7]] # liste eterogene
elevi = [("Alina", 10), ("Mihai", 7), ("Irina", 9)] # listă de perechi
```

IMPORTANT

Indexarea începe de la 0, elementul de pe poziția `i` este `lista[i]`:
`lista[i]` = elementul de pe poziția/indicele `i` (numerotare de la 0).

Indicii pot fi numere negative și permit numărarea elementelor de la dreapta spre stânga. Pentru exemplul 2 avem:

- ▶ `note[-1]` – reprezintă ultimul element, 9;
- ▶ `note[-2]` – reprezintă penultimul element, 10 (după modificare).

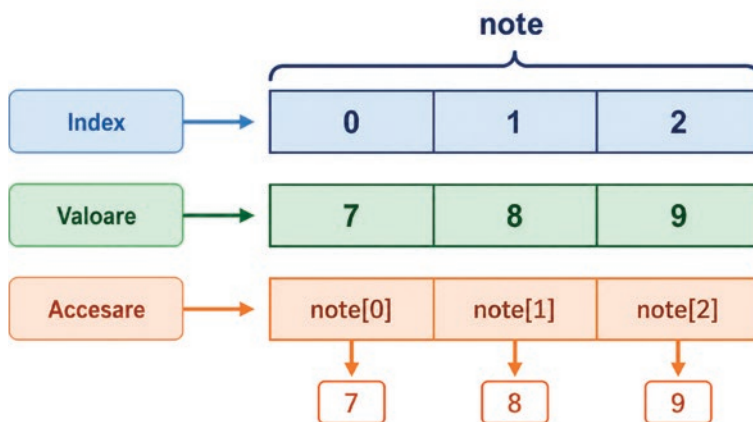
Dacă indicele nu este valid, apelul `note[i]` va da eroare (`IndexError`).

3.2.2. Indexarea elementelor din listă. Parcurgerea unei liste

Accesarea unei componente se face prin index, care se trece între paranteze drepte. Un element poate fi modificat folosind indexul său.

EXEMPLUL 2 >>>

Inițial avem lista `note = [7, 8, 9]`.
 Instrucțiunea `note[1] = 10` modifică valoarea de pe indexul 1 (al doilea element din listă).
 Iar instrucțiunea `print(note)` va afișa lista `[7, 10, 9]`.



APLICAȚIE

Ce va afișa următoarea secvență de cod pentru `lista = [5, 8, 2, 4, 6, 2]`

```
print(lista[0])
print(lista[3])
print(lista[-2])
```

Parcurgerea unei liste înseamnă prelucrarea, pe rând, a fiecărui element al acesteia. În Python, listele pot fi parcurse în două moduri principale.

1 Parcurgerea directă a elementelor

Aceasta este metoda recomandată atunci când avem nevoie doar de valori:

EXEMPLUL 3 >>>

Cod Python	Exemplu de rulare	La fiecare pas al buclei, variabila <code>x</code> primește valoarea elementului curent din listă. Este potrivită în majoritatea problemelor – calcul de sumă, numărare, verificare condiții.
<pre>lista = [5, 8, 2] for x in lista: print(x)</pre>	<pre>5 8 2</pre>	

2 Parcurgerea prin intermediul indicilor

Această metodă este utilă atunci când avem nevoie de poziția elementului:

EXEMPLUL 4

> > >

Cod Python

```
L = [5, 8, 2]
for i in range(3):
    print("I:", i, "V:", L[i])
```

Exemplu de rulare

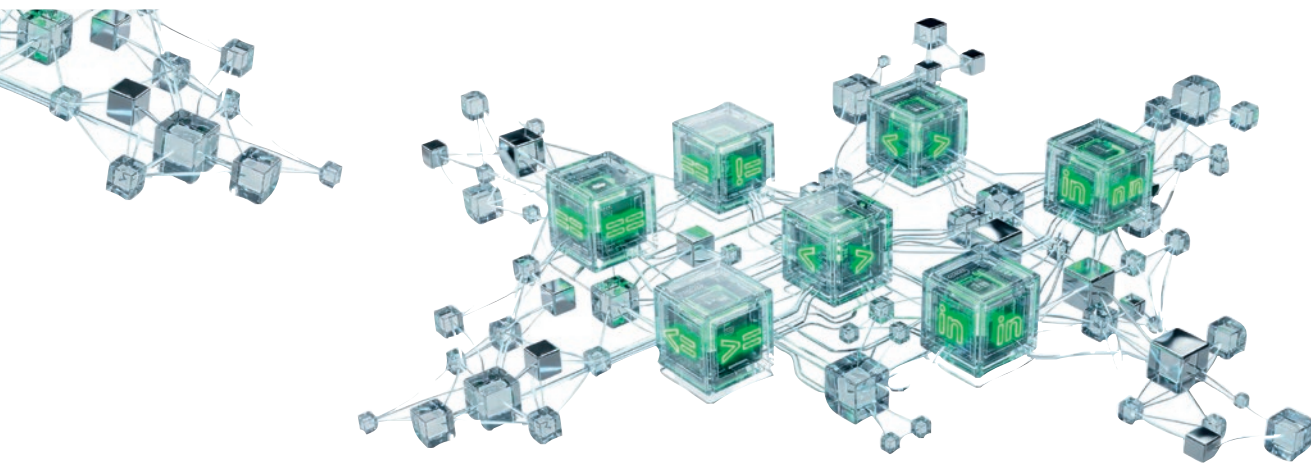
```
I: 0 V: 5
I: 1 V: 8
I: 2 V: 2
```

Variabila **i** reprezintă indexul sau indicele elementului, iar accesul la valoare se face prin **L[i]**.

3.2.3. Operatori

Clasa **list** conține o serie de operatori utili pentru manipularea listelor.

Operator	Descriere	Exemplu
+	Folosit pentru a concatena două liste.	<pre>a = [5, 8, 2] b = [4, 6, 2] c = a + b</pre> c va avea valoarea [5, 8, 2, 4, 6, 2]
*	Creează o listă formată prin repetarea listei inițiale de un număr dat de ori.	<pre>a = [3] * 5 b = [1, 2, 3] * 2</pre> a va avea valoarea [3, 3, 3, 3, 3] b va avea valoarea [1, 2, 3, 1, 2, 3]
==, !=, <, >, <=, >=.	Compară două liste între ele. Compararea este lexicografică, adică se face element cu element, luând elementele în aceeași ordine.	<pre>a = [1, 2, 3] b = [1, 2, 3] c = [1, 2, 4] d = [3, 2, 1]</pre> a == b va avea valoarea True a != c va avea valoarea True a <= c va avea valoarea True a == d va avea valoarea False
in și not in	Pot fi folosiți pentru a verifica apartenența unui element la o listă.	<pre>L = [1, 2, 3, 4, 5, 6, 7, 8] 3 in L este True 11 in L este False</pre>





3.2.4. Metode ale clasei „list”

Clasa `list` pune la dispoziție o serie de metode pe care le putem folosi în lucrul cu liste.

Metodă	Efect	Parametri	Returnează
<code>append(x)</code>	adaugă <code>x</code> la finalul listei	<code>x</code> – valoarea adăugată	<code>None</code>
<code>extend(l)</code>	adaugă la final toate elementele din lista <code>l</code>	<code>l</code> – listă	<code>None</code>
<code>sort()</code>	ordonează lista crescător	opțional <code>reverse=True</code>	<code>None</code>
<code>pop()</code>	șterge ultimul element	opțional – indexul de șters	elementul șters
<code>remove(x)</code>	șterge prima apariție a lui <code>x</code>	<code>x</code> - valoarea	<code>None</code>
<code>insert(i,x)</code>	inserează elementul <code>x</code> pe poziția <code>i</code>	<code>i</code> – indice <code>x</code> – valoare	<code>None</code>
<code>count(x)</code>	numără aparițiile valorii <code>x</code>	<code>x</code> – valoare	numărul rezultat
<code>index(x)</code>	returnează indexul primei apariții	<code>x</code> – valoare	indice
<code>clear()</code>	golește lista	–	<code>None</code>
<code>reverse()</code>	inversează ordinea elementelor	–	<code>None</code>
<code>copy()</code>	copiază lista	–	lista nouă

>>> EXEMPLE

<code>note = [7, 9]</code> <code>note.append(8)</code>	<code>note = [7, 9, 8]</code>
<code>note2 = [6, 5]</code> <code>note.extend(note2)</code>	<code>note = [7, 9, 8, 6, 5]</code>
<code>note.sort()</code>	<code>note = [5, 6, 7, 8, 9]</code>
<code>x = note.pop()</code>	<code>x = 9</code> <code>note = [5, 6, 7, 8]</code>
<code>note.pop(0)</code>	<code>note = [6, 7, 8]</code>
<code>note.remove(8)</code>	<code>note = [6, 7]</code>
<code>note.insert(1, 8)</code>	<code>note = [6, 8, 7]</code>
<code>note.reverse()</code>	<code>note = [7, 8, 6]</code>
<code>ls = note.copy()</code> <code>ls.append(4)</code>	<code>note = [7, 8, 6]</code> <code>ls = [7, 8, 6, 4]</code>
<code>note.count(8)</code>	1
<code>note.count(1)</code>	0
<code>note.index(8)</code>	1
<code>note.index(4)</code>	eroare
<code>note.clear()</code>	<code>note = []</code>



PROBLEMĂ REZOLVATĂ 1

Enunțul problemei: Se citesc de la tastatură cele 5 note ale unui elev la Informatică. Să se modifice a doua notă obținută de elev (cu indexul/poziția 1), să se șteargă nota de la indexul 2, apoi să se mai adauge două note.

Analiza problemei:

- *Date de intrare:* cele 5 note inițiale, valoarea pentru nota a doua și cele două note de adăugat.
- *Date de ieșire:* lista cu notele actuale ale elevului.



Rezolvare:

Program Python

```

note = []
for i in range(5):
    n1 = float(input(f"Citeste nota {i + 1}: "))
    note.append(n1)

print("\nNote initiale:", note)

# Modificarea celei de-a doua note
n2 = float(input("Citeste noua nota: "))
note[1] = n2
print("Dupa modificare:", note)

# Ștergerea notei de pe index 2
n3 = note.pop(2)
print(f"Dupa ștergerea notei ({n3}):", note)

# Adăugarea a două note noi
n4 = float(input("Citeste prima nota: "))
n5 = float(input("Citeste a doua nota: "))
note.append(n4)
note.append(n5)

# note.extend([n4,n5])

# Afișarea listei finale cu note
print("\nLista finala cu note:", note)

Citeste nota 1: 7
Citeste nota 2: 8
Citeste nota 3: 10
Citeste nota 4: 6
Citeste nota 5: 8

Note initiale: [7.0, 8.0, 10.0, 6.0, 8.0]
Citeste noua nota: 9
Dupa modificare: [7.0, 9.0, 10.0, 6.0, 8.0]
Dupa ștergerea notei (10.0): [7.0, 9.0, 6.0, 8.0]
Citeste prima nota: 4
Citeste a doua nota: 5

Lista finala cu note: [7.0, 9.0, 6.0, 8.0, 4.0, 5.0]

```

Explicații

Se folosește un `for` pentru a citi 5 note și a le adăuga în listă cu `append()`.

Elementul este accesat prin index. Mutabilitatea listei permite modificarea directă.

`pop(2)` elimină elementul de la indexul 2 și îl returnează.

`append()` adaugă fiecare element la sfârșitul listei, păstrând ordinea.

Se putea face și cu `extend()`.

Afișarea rezultatului final: Lista finală reflectă toate modificările succesive.

Exemplu de rulare



3.2.5. Funcții predefinite pentru liste

Python pune la dispoziție o serie de funcții predefinite ce pot fi folosite pentru a prelucra liste, fără a mai fi necesar să le definim de fiecare dată. În continuare, vom enumera cele mai utilizate funcții cu detalii și exemple:

- **len(lista)** – determină lungimea unei liste;

```
numere = [4, 7, 2, 9]
```

`len(numere)` este 4

- **min(lista)** și **max(lista)** – determină minimul și maximul dintr-o listă. Lista trebuie să aibă cel puțin un element, altfel programul va da eroare.

```
numere = [4, 7, 2, 9]
```

`min(numere)` este 2

`max(numere)` este 9

- **sum(lista)** – calculează suma elementelor dintr-o listă, poate fi utilizată doar pentru liste cu elemente de tip numeric;

```
numere = [4, 7, 2, 9]
```

`sum(numere)` este 22

- **sorted(lista)** – returnează o listă nouă ordonată, fără a modifica lista inițială.

```
numere = [4, 7, 2, 9]
ordonata = sorted(numere)
print(numere)
print(ordonata)
numere.sort()
print(numere)
```

Rezultatele afișărilor sunt:

[4, 7, 2, 9]

[2, 4, 7, 9]

[2, 4, 7, 9]



IMPORTANT

Diferența dintre **sort()** și **sorted()**:

sorted() – funcție, creează listă nouă;

sort() – metodă din clasa *list*, modifică lista existentă.



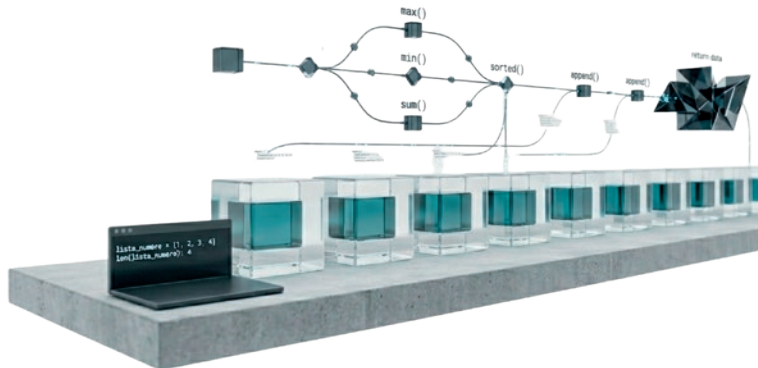
PROBLEMĂ REZOLVATĂ 2

La disciplina Informatică se înregistrează notele obținute de mai mulți elevi. Pentru fiecare elev se introduc de la tastatură notele obținute, una câte una. Introducerea notelor pentru un elev se oprește atunci când se introduce valoarea 0, care reprezintă o valoare de oprire și nu face parte din lista notelor.

Să se realizeze un program Python care:

- să citească numărul *n* de elevi;
- să citească notele fiecărui elev și să le memoreze într-o listă;
- pentru fiecare elev să determine: lista notelor introduse, numărul de note, nota minimă, nota maximă, suma notelor, media aritmetică a notelor.

Programul va utiliza subprograme pentru citirea notelor și pentru generarea raportului statistic.





Rezolvare:

Program Python

```
def citire_note():
    # Citește notele unui elev până la introducerea valorii 0
    note = []
    while True:
        nota = float(input("Introduceti o nota (0 pentru oprire): "))
        if nota == 0:
            break
        note.append(nota)
    return note

def raport_note(note): # Parametrul note - de tip listă
    if len(note) == 0:
        return None
    numar_note = len(note)
    nota_min = min(note)
    nota_max = max(note)
    suma_note = sum(note)
    media = suma_note / numar_note
    return numar_note, nota_min, nota_max, suma_note, media

# Program principal
n = int(input("Introduceti numarul de elevi: "))
for i in range(n):
    print(f"Elevul {i + 1}")
    lista_note = citire_note()
    rezultat = raport_note(lista_note)
    print("Lista notelor:", lista_note)
    if rezultat is None:
        print("Elevul nu are note.")
    else:
        numar_note, nota_min, nota_max, suma_note, media = rezultat
        print("Numar note:", numar_note)
        print("Nota minima:", nota_min)
        print("Nota maxima:", nota_max)
        print("Suma notelor:", suma_note)
        print("Media:", media)
```

```
Introduceti numarul de elevi: 2
Elevul 1
Introduceti o nota
(0 pentru oprire): 7
Introduceti o nota
(0 pentru oprire): 9
Introduceti o nota
(0 pentru oprire): 10
Introduceti o nota
(0 pentru oprire): 0
Lista notelor: [7.0, 9.0, 10.0]
Numar note: 3
Nota minima: 7.0
Nota maxima: 10.0
Suma notelor: 26.0
Media: 8.666666666666666
```

```
Elevul 2
Introduceti o nota
(0 pentru oprire): 8
Introduceti o nota
(0 pentru oprire): 6
Introduceti o nota
(0 pentru oprire): 0
Lista notelor: [8.0, 6.0]
Numar note: 2
Nota minima: 6.0
Nota maxima: 8.0
Suma notelor: 14.0
Media: 7.0
```



FIȘĂ DE LUCRU 7

1. Se consideră programul:

```
L = [2, 7, 6, 4, 9, 8, 1, 3]
lpar = []
limpar = []
for i in range(len(L)):
    if L[i] % 2 == 0:
        lpar.append(L[i])
    else:
        limpar.append(L[i])
print(lpar)
print(limpar)
```

Cerințe:

- Ce se afișează la execuția programului?
- Ce reprezintă variabila `i`?
- Cum se poate rescrie programul fără utilizarea indicilor?

2. Se consideră programul:

```
L = [1, 2, 3, 4]
for x in L:
    if x % 2 == 0:
        L.remove(x)
print(L)
```

Cerințe:

- Ce se dorește să facă programul?
- De ce rezultatul poate fi incorect? Dă un exemplu.
- Propune o variantă corectă.

3. Un antrenor dorește să analizeze performanța unui grup de sportivi.

Se citesc de la tastatură un număr natural n și cele n scoruri obținute de sportivi.

Să se calculeze scorul minim, scorul maxim și media scorurilor.

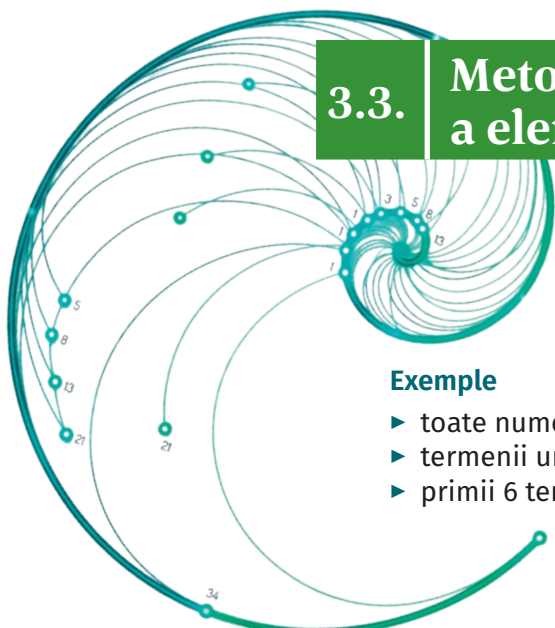
4. Se citesc de la tastatură un număr natural n și o listă de n numere. Să se calculeze numărul de perechi de valori pare aflate pe poziții consecutive din listă. Să se verifice dacă algoritmul este corect testând cazurile limită (extremități, liste scurte, liste fără elemente pare).

5. Să se citească de la tastatură un număr natural n și o listă de n numere întregi.

Să se creeze o listă cu elementele pozitive, iar la final să fie cele negative.

3.3.

Metode de generare sistematică a elementelor unei liste



Exemple

- ▶ toate numerele pare între 2 și 20;
- ▶ termenii unei expresii matematice $2 \cdot n + 1$ pentru $n = 0, 1, 2, \dots, 5$;
- ▶ primii 6 termeni ai unui șir Fibonacci.

În informatică, multe probleme presupun **generarea unei secvențe de valori** care respectă anumite proprietăți sau reguli. În loc să introducem valorile manual, putem folosi algoritmi simpli pentru a construi automat aceste secvențe sub forma unor liste.

Pentru a genera liste, cea mai folosită metodă este parcurgerea unui interval de valori, adăugând elementele într-o listă goală.

EXEMPLE



```
numere_pare = []
for i in range(2, 21, 2):
    # start=2, stop=21, pas=2
    numere_pare.append(i)
print(numere_pare)
```

1. Secvență de numere pare;

[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

```
termeni = []
for n in range(6):
    termeni.append(2*n + 1)
print(termeni)
```

2. Termeni ai unei expresii matematice, de exemplu: $2*n+1$ pentru $n=0,1,2,3,4,5$.

[1, 3, 5, 7, 9, 11]

```
fibonacci = [0, 1]
for i in range(2, 6):
    fibonacci.append(fibonacci[i-1] + fibonacci[i-2])
print(fibonacci)
```

3. Termeni ai unor șiruri recurente, de exemplu șirul Fibonacci (primele 6 valori).

[0, 1, 1, 2, 3, 5]

Pașii pentru a genera secvențe sunt următorii:

1	Identificarea regulilor – ne punem întrebarea „Cum se obține fiecare element?” <i>Exemplu:</i> $x = 2*n+1$, Fibonacci: $F_n = F_{n-1} + F_{n-2}$
2	Stabilirea intervalului sau numărului de termeni. – <code>range(start, stop, pas)</code> sau numărul de pași
3	Inițializarea listei. – De obicei: <code>lista = []</code> sau lista cu valori de început (ex. <code>[0,1]</code> la Fibonacci)
4	Parcurgerea intervalului și adăugarea elementelor. – <code>for i in range(...)</code> și <code>append()</code>
5	Aplicarea algoritmilor de bază. – Sumă, medie, numărare, determinarea maximului/minim, verificări condiționale.

PROBLEMĂ REZOLVATĂ 1

Un exemplu concret este generarea listei temperaturilor medii anuale pentru un oraș, într-un scenariu de încălzire globală, știind că temperaturile cresc treptat de la un an la altul.

Pașii pentru construirea listei:

1. Identificarea regulii: Trebuie să știm cum se modifică fiecare termen.

Exemplu: temperatura crește cu 0,2 °C față de anul precedent, notat **increment**:

- temperatura anului i = temperatura inițială + increment $\times i$

2. Stabilirea valorii inițiale și a numărului de termeni:

- temperatura din primul an este punctul de plecare, de exemplu: 14,0 °C;
- numărul de ani ne dă câți termeni trebuie să generăm (exemplu: 10).

3. Crearea unei liste goale:

- `temperaturi = []`

4. Parcurgerea intervalului și adăugarea elementelor conform regulii:

- folosim o buclă `for`;
- calculăm fiecare valoare și o adăugăm cu `append()`.

5. Lista poate fi folosită ulterior pentru alte calcule.

Rezolvare:

Cod Python

```
T = []
T0 = 14.0
nr_an = 10
increment = 0.2
for i in range(nr_an):
    T1 = T0 + i * increment
    T.append(T1)
print(T)
```

Exemplu de rulare

```
[14.0, 14.2, 14.4, 14.6, 14.8, 15.0, 15.2, 15.4, 15.6, 15.8]
```

PROBLEMĂ REZOLVATĂ 2

Enunțul problemei: Să se citească de la tastatură un număr natural n .

Să se calculeze suma $S=1+1*2+1*2*3+\dots+1*2*3*\dots*n$.

- Să se calculeze suma S folosind un **algoritm de calcul direct**, în care fiecare termen este determinat independent.
- Să se calculeze aceeași sumă folosind un **algoritm de generare recurent**, în care fiecare termen este obținut pe baza termenului precedent.
- Să se justifice utilizarea algoritmului recurent prin compararea eficienței celor două metode.

Explicații:

a) Algoritm de calcul direct

Idea: pentru fiecare k , calculăm separat produsul $1*2*3*\dots*k$, apoi îl adunăm la sumă. Pentru fiecare termen reluăm calculul produsului de la 1.

De exemplu, pentru $k = 5$ refacem și $1*2*3*4$ deja calculate anterior.

b) Algoritm de generare recurent

Observăm relația $1*2*3*\dots*n = [1*2*3*\dots*(n-1)]*n$. Putem construi fiecare termen din precedentul și nu mai recalculăm produsul, folosim rezultatul anterior și îl înmulțim cu k curent.



c) Implementare Python

Algoritm de calcul direct

```
n = int(input("n = "))
S = 0

for k in range(1, n + 1):
    prod = 1
    for i in range(1, k + 1):
        prod = prod * i
    S = S + prod

print("S =", S)
```

n = 4
S = 33

Algoritm de generare recurent

```
n = int(input("n = "))
S = 0
prod = 1

for k in range(1, n + 1):
    prod = prod * k
    S = S + prod

print("S =", S)
```

n = 4
S = 33

d) Compararea eficienței

	Algoritm de calcul direct	Algoritm de generare recurent
Număr operații	$1 + 2 + \dots + n = \frac{n * (n + 1)}{2}$	$2 * n$ (o înmulțire și o adunare la fiecare pas)

e) Concluzie

Algoritmul recurent este mult mai eficient deoarece:

- evită recalcularea inutilă a produselor;
- pentru valori mari ale lui n, diferența devine semnificativă.



FIȘĂ DE LUCRU 8

1. Se consideră programul:

```
n = 6
a = 1
for i in range(1, n):
    a = a * 2 + i
print(a)
```

Cerințe:

- a) Ce se afișează la execuția programului?
b) Corectează codul astfel încât să includă și termenul n.

2. Un cont bancar are o sumă inițială și o dobândă anuală (procentuală).

Se cere să se determine numărul de ani după care soldul contului depășește o valoare dată.

Se vor citi de la tastatură suma inițială **s0**, dobânda **r**, și valoarea țintă **s1**

și se vor afișa lista soldurilor pentru fiecare an și numărul minim de ani necesari.

3. Să se citească de la tastatură un număr natural n.

Să se determine lista primelor n pătrate perfecte.

4. Se consideră șirul lui Fibonacci definit prin relația de recurență:

$$f_0 = 0, f_1 = 1, f_n = f_{(n-1)} + f_{(n-2)}, n \geq 2$$

Se cunoaște un termen din șirul lui Fibonacci și se dorește

afișarea termenilor în **ordine descrescătoare**:

$$f_n, f_{n-1}, f_{n-2}, \dots, f_1.$$

5. Se consideră șirul definit prin $f_0 = 3$ și relația de recurență: $f_n = 2 * f_{n-1}$.

Să se scrie un program Python eficient care afișează primii n termeni ai șirului.



3.4. Metode de sortare

3.4.1. Introducere

Dacă în viața de zi cu zi a sorta înseamnă a alege, a aranja, a repartiza, în programarea calculatoarelor, sortarea elementelor este sinonimă cu ordonarea lor crescătoare sau descrescătoare după anumite criterii. Ordonarea elementelor unei liste stă la baza eficienței în informatică, cel mai puternic argument fiind viteza de regăsire a informației.

Iată principalele motive pentru care sortăm datele:

- 1 **Căutarea este substanțial mai rapidă**
Pentru a găsi un element într-o listă nesortată trebuie să verifici fiecare poziție în parte (căutare liniară), pe când într-o listă ordonată se pot folosi algoritmi de căutare rapidă.
- 2 **Identificarea duplicatelor**
Într-o listă neordonată, ca să vezi dacă un element se repetă trebuie să îl compari cu toate celelalte. Într-o listă sortată, duplicatele sunt adiacente (stau unul lângă altul).
- 3 **Găsirea extremelor**
Dacă ai nevoie de cel mai mic sau cel mai mare element, o listă sortată crescător îți oferă răspunsul instantaneu: minimul este la indexul 0, iar maximul este la indexul $n-1$.
- 4 **Pregătirea datelor pentru alți algoritmi**
Multe procese complexe depind de date ordonate pentru a funcționa corect sau rapid:
 - ▶ algoritmi de compresie a datelor;
 - ▶ randarea grafică (pentru a desena obiectele de la spate spre față);
 - ▶ sisteme de baze de date (indexarea).



IMPORTANT



Sortarea se efectuează nu pentru a avea o listă „frumoasă”, ci pentru a reduce complexitatea computațională a tuturor acțiunilor ulterioare asupra datelor.

Analogie din viața reală

- ▶ Să ne imaginăm că suntem într-o bibliotecă unde cărțile nu sunt sortate alfabetic sau pe domenii, ci sunt aruncate la întâmplare pe rafturi. Ca să fie găsită o anumită carte, ar trebui să fie citite cotoarele tuturor cărților. Sortarea transformă orele de căutare în secunde.
- ▶ De fiecare dată când pe un site al unui magazin online se folosește un filtru (de exemplu: „Preț: de la mic la mare”), se rulează un algoritm de sortare.
- ▶ Sortarea alfabetică (**lexicografică**) a agendei telefonice permite căutarea/găsirea rapidă a unui contact.





Există foarte multe metode de sortare și o paletă largă a domeniilor de aplicare:

1. Informatică – aproximativ 40 de algoritmi clasici, plus metode pentru situații specifice.

2. Logistică și Industrie

- ▶ *Cross-Belt Sorting*: folosită în aeroporturi (mini-benzi perpendiculare pe banda principală).
- ▶ *Sortarea gravitațională*: obiectele alunecă pe jgheaburi diferite în funcție de greutate.

3. Statistică și Știința datelor

- ▶ *Sortarea topologică*: stabilirea ordinii sarcinilor care depind unele de altele.

4. Metode „umane” și de arhivare

- ▶ *Sistemul Decimal Dewey*: folosit în biblioteci pentru a sorta cărțile pe domenii și autori.
- ▶ *Triajul medical*: sortarea pacienților în funcție de urgență (cod roșu, galben, verde).

De ce există atât de multe? Pentru că niciun algoritm nu este perfect. Metoda ce urmează a fi utilizată se poate alege în funcție de:

- natura obiectului (Este un număr în memorie sau un colet de 10 kg?);
- volumul (Sunt 10 elemente sau 10 miliarde?);
- costul (Consum de procesor vs. consum de curent/resurse etc.).

3.4.2. Sortarea prin Selecție (Selection Sort)

Este una dintre metodele cele mai des folosite pentru ordonarea unei liste de valori, datorită simplității ideii pe care se bazează: în lista ordonată crescător, elementul de pe prima poziție este cel mai mic, al doilea este al doilea cel mai mic etc. Astfel, algoritmul caută cel mai mic element din listă și îl pune direct la locul lui final în lista ordonată.

Descrierea algoritmului (pentru **ordonarea crescătoare**)

Se pleacă cu toată lista inițială neordonată și se construiește pe parcurs o parte **sortată** (la stânga), lăsând partea **nesortată** (la dreapta) din ce în ce mai mică, executându-se următoarele:

- ▶ Se caută cel mai mic număr din partea nesortată.
- ▶ Se schimbă acest număr cu primul element din zona nesortată.
- ▶ Se mută „granița” dintre cele două zone cu o poziție la dreapta.
- ▶ Se repetă procesul până când toată lista este sortată.

Lista de valori [29, 10, 14, 37, 13]

Poziție/ indice 0 1 2 3 4

>>> EXEMPLU

Pas	Operație
1	„Granița” dintre zone este la 0. Minimul din zona nesortată (toată lista) este 10, la poziția 1 . Se schimbă minimul cu elementul de pe „graniță” și lista devine [10, 29, 14, 37, 13].
2	„Granița” dintre zone este la 1. Minimul din zona nesortată (de la 1 la n-1 în lista) este 13, la poziția 4 . Se schimbă minimul cu elementul de pe „graniță” și lista devine [10, 13, 14, 37, 29].
3	„Granița” dintre zone este la 2. Se caută minimul în [14, 37, 29]; este 14, la locul lui , lista rămâne aceeași [10, 13, 14, 37, 29].
4	„Granița” dintre zone este la 3. Se caută minimul în [37, 29]; e 29 și lista devine [10, 13, 14, 29, 37].

Observație: Ne oprim după n-1 pași, pentru că dacă sunt fixate pe locurile lor cele mai mici n-1 elemente, al n-lea rămas este automat cel mai mare și își are locul pe ultima poziție.

Implementarea este foarte intuitivă: folosim o variabilă `min_idx` pentru a ține minte unde se află cel mai mic element găsit, „granița” la pasul `i` fiind tocmai `i`.



Cod Python

```
def selection_sort_cresc(lista):
    n = len(lista)
    for i in range(n-1): # Parcurgem lista până la penultimul element
        min_idx = i      # Inițializarea poziției minimului
        # Căutăm un eventual element mai mic în dreapta graniței(zona nesortată)
        for j in range(i + 1, n):
            if lista[j] < lista[min_idx]:
                min_idx = j # Un nou minim la dreapta, la poziția j
        # minimul găsit la min_idx se interschimbă cu elementul de pe granița i
        lista[i], lista[min_idx] = lista[min_idx], lista[i]
```



Exemplu de rulare

```
date = [29, 10, 14, 37, 13]
selection_sort_cresc(date)
print("Lista sortata crescător prin selecție: ",date)
# Pe ecran se va afișa
Lista sortata crescător prin selecție: [10, 13, 14, 29, 37]
```

IMPORTANT



Interschimbarea/swap
valorilor variabilelor
a și **b** se poate scrie
`a, b = b, a`

Analiza Performanței

Observație: Algoritmul de sortare prin selecție nu se oprește nici-odată mai devreme.

A. Complexitatea Timp

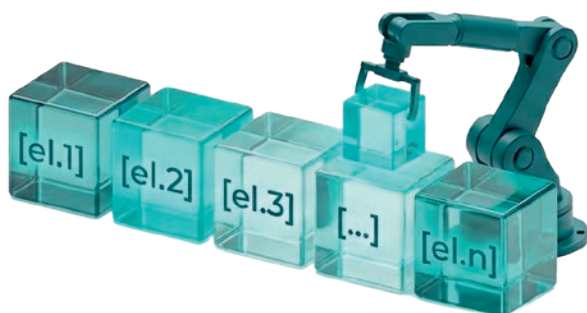
Analiza pașilor pentru o lista cu n elemente.

- ▶ Pentru primul loc, comparăm elementul curent cu restul de $n-1$ elemente.
- ▶ Pentru al doilea loc, comparăm cu restul de $n-2$ elemente.
- ▶ Continuăm până la ultima comparație.

Matematic, numărul total de comparații este suma: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$, adică $(n^2 - n)/2$ calcule elementare.

B. Complexitatea Spațiu

Câtă memorie ocupă? Algoritmul nu are nevoie să creeze o copie a listei sau să rezerve memorie suplimentară proporțională cu numărul de elemente, folosește doar câteva variabile temporare, adică Sortarea prin Selecție a unei liste de n elemente este un *algoritm „in place”*.





PROBLEMĂ REZOLVATĂ



Enunțul problemei:

La ora de sport, cei n elevi ai unei clase trebuie să formeze un șir în care cele k fete ocupă primele poziții în ordine crescătoare, iar băieții ocupă în ordine descrescătoare următoarele poziții. Se dau n și k (numere naturale, $k \leq n$) și cele n valori numere naturale ce reprezintă înălțimile elevilor în centimetri. Să se ordoneze crescător fetele (primele k elemente) folosind sortarea prin selecția maximului și descrescător băieții, folosind sortarea prin selecția minimului.

Exemplu: Pentru $n=6$, $k=3$ și elementele **5, 2, 8,** **1, 9, 3** => **2, 5, 8,** **9, 3, 1**

Analiza problemei:

Putem spune că avem două probleme într-una, dar în realitate avem de aplicat același algoritm de sortare prin selecție pe două zone distincte din lista inițială, cu atenție la sensul de parcurgere, la graniță și la elementul căutat (minim sau maxim).

Cod Python

```
# citire n
n = int(input("n = "))
# citire n elemente
v = []
for i in range(n):
    x = int(input("v[" + str(i) + "] = "))
    v.append(x)
k = int(input("k = "))
# sortare crescatoare a primelor k elemente
for i in range(k - 1, 0, -1):
    poz_max = 0
    for j in range(1, i + 1):
        if v[j] > v[poz_max]:
            poz_max = j
    v[i], v[poz_max] = v[poz_max], v[i]
# sortare descrescatoare
for i in range(n - 1, k, -1):
    poz_min = k
    for j in range(k + 1, i + 1):
        if v[j] < v[poz_min]:
            poz_min = j
    v[i], v[poz_min] = v[poz_min], v[i]
# afisare
print("Șirul final este:\n", v)
```

Exemplu de rulare

```
n = 7
v[0] = 4
v[1] = 3
v[2] = 2
v[3] = 1
v[4] = 6
v[5] = 7
v[6] = 8
k = 4
Șirul final este:
[1, 2, 3, 4, 8, 7, 6]
```

A. Complexitatea Timp

Ordonarea

- fetelor: $(k^2 - k)/2$
- băieților: $((n-k)^2 - (n-k))/2$

B. Complexitatea Spațiu

Sortarea prin selecție nu are nevoie de memorie în plus în mod semnificativ, ci doar de câteva variabile auxiliare (este "in-place").

3.4.3. Sortare cu liste de frecvență (Counting Sort)

Sortarea cu liste de frecvență (cunoscută și sub numele de **Counting Sort**) este un algoritm de sortare neconvențional, deoarece nu se bazează pe comparații între elemente ca Sortarea prin Selecție, ci pe **numărarea aparițiilor** fiecărei valori. Este o metodă extrem de eficientă atunci când avem de sortat **un volum uriaș de date** cu valori dintr-un **interval relativ mic** (de exemplu, note de la 1 la 10 sau vârste de la 0 la 100).

Cum funcționează ?

Să ne imaginăm că trebuie ordonate punctajele (numere naturale între 0 și 100) pentru 200.000 de elevi: un algoritm de comparare (prin selecție sau metoda bulelor) compară notele între ele de miliarde de ori (200.000^2 , adică 40.000.000.000).

Cu **lista de frecvență**, procesul este mult mai simplu:

Deoarece numărul de valori posibile este relativ restrâns (între 0 și 100), vom folosi 101 „cutii”

etichetate de la 0 la 100, inițial toate goale, fiecare reținând câte valori egale cu eticheta sa au apărut. Aceste cutii vor fi de fapt elementele unei liste de lungime 101 cu valoare inițială 0

(`frecventa [ ] = [0, 0, ....., 0]`).

Numărarea (scanarea datelor) se va realiza trecând o singură dată prin șirul celor 200.000 de note: la întâlnirea unei note, valoarea cutiei corespunzătoare se mărește cu 1 – ai găsit un 88 ? aduni 1 la poziția 88; ai găsit un 75? aduni 1 la poziția 75, iar la final te uiți în fiecare cutie și vei ști exact câți de 88 sunt, câți de 75 etc., fără să fi făcut vreo comparație.

Sortarea propriu-zisă se realizează pur și simplu prin scrierea punctajelor în ordine, fiecare fiind afișat de atâtea ori cât este frecvența de apariție: dacă în cutia „75” ai 500, scrii de 500 de ori 75, dacă în cutia „88” ai 1 200, scrii 88 de 1 200 de ori ... și tot așa până la 100.

indici/ note	→	0	1		75		88		95		100
nr. apariții	→	21	112		500		1200		977		710

Cod Python

```
def sort_frecventa(lista):
    n = len(lista)
    frecventa = [0] * 101    # Inițializăm lista de frecvență cu 0
    for i in range(n):
        frecventa[lista[i]] += 1    # Completăm lista
    print("Valorile sortate sunt:")
    # Parcurgem lista de frecvență de la 0 la 100 pentru afișare
    for i in range(101):
        for j in range(frecventa[i]):    # Dacă 'i' apare
            print(i, end=" ") # Îl afișăm de frecventa[i] ori
```

Exemplu de rulare

```
date = [75, 88, 67, 90, 75, 64, 88, 75]
sort_frecventa(date)
#Pe ecran se va afișa :
Valorile sortate sunt:
64 67 75 75 75 88 88 90
```

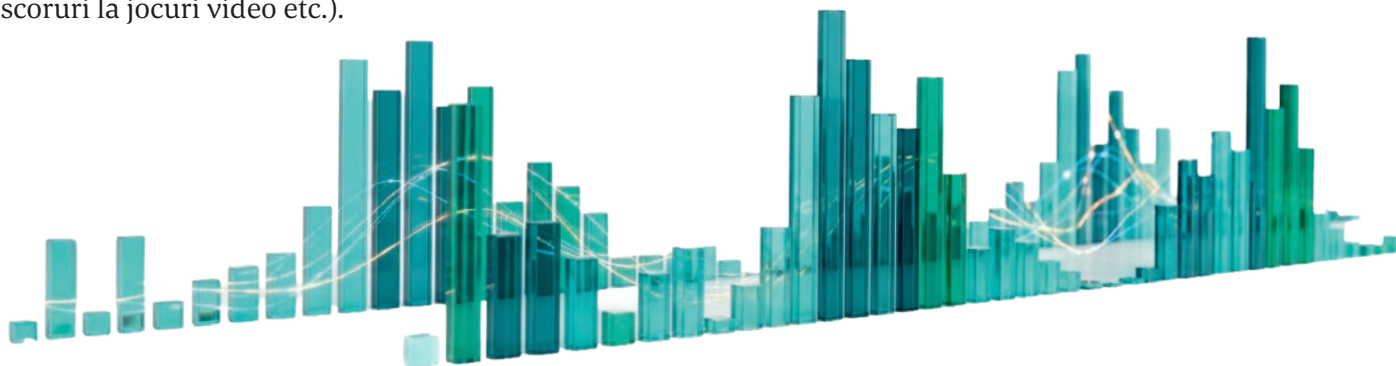


Avantaje și limitări

Avantaje	Limitări
Viteză mare: Numărul de pași efectuați este $n+k$, ceea ce o face metoda cea mai rapidă pentru date masive.	Consum de memorie: Dacă numerele sunt foarte mari (de exemplu, vrei să sortezi CNP-uri), ai avea nevoie de un vector de frecvență uriaș, ceea ce ar bloca memoria.
Simplitate: Codul este foarte scurt și ușor de înțeles.	Doar numere întregi: Nu poți folosi ușor liste de frecvență pentru nume sau cuvinte.

Când se folosește?

- ▶ Când sunt multe elemente (milioane).
- ▶ Când intervalul de valori este mic (note, vârste, zilele lunii, scoruri la jocuri video etc.).



STUDIUL DE CAZ 1

De multe ori, notele rezultate în urma unor examinări sau evaluări au valori numere reale pozitive cu 2 zecimale, mai mici sau egale cu 10. Este evident că nu putem utiliza „cutii” pentru fiecare valoare posibilă, dar **dacă înmulțim cu 100 fiecare notă, vom obține numere naturale** între 0 și 1000; trebuie să nu uităm ca la afișare valoarea să fie împărțită cu 100 și scrisă ca număr real cu două zecimale.

indici	→	100	200		783		850		950		1000
note	→	1	2		7,83		8,50		9,50		10

Activitate în echipă

Să se adapteze programul anterior pentru afișarea notelor (numere reale pozitive mai mici sau egale cu 10) obținute de cei n candidați în ordine descrescătoare.



STUDIU DE CAZ 2

Este posibil ca intervalul de valori, deși restrâns, să acopere și valori negative. În acest caz, pentru a ne putea folosi de indicii elementelor unei liste, se va face o translație către valorile naturale prin adunarea unei valori pozitive la fiecare valoare din listă. De exemplu, dacă avem o listă a temperaturilor zilelor unui an calendaristic în care minima a fost de -35°C , iar maxima de 40°C , prin adunarea lui 35 la fiecare valoare vom obține doar temperaturi pozitive între 0 și 75, deci vom putea folosi o listă de frecvență. Trebuie doar să nu uităm să scădem 35 la afișarea fiecărei temperaturi sau la efectuarea unor eventuale calcule (temperaturi medii etc.).

indici	→	0	1	2	3	...	35	36	...	73	74	75
temperaturi	→	-35°	-34°	-33°	-32°	...	0°	1°	...	38°	39°	40°

Activitate în echipă

Să se scrie un program Python care, pe baza temperaturilor înregistrate într-o lună de iarnă, să determine frecvențele de apariții ale temperaturilor, temperaturile minimă și, respectiv, maximă înregistrate, precum și temperatura medie a lunii.

Pe ecran să se afișeze pe primul rând $T_{\min}$ $T_{\max}$ T_{Med}

iar pe fiecare din următoarele linii, crescător după T , o pereche TF , unde T este temperatura, iar F este numărul de zile în care s-a înregistrat T .



FIȘĂ DE LUCRU 9

1. Fie enunțul: Pentru a sorta descrescător prin metoda selecției o listă cu 20 de elemente numere reale, nu sunt necesare mai mult de x determinări ale valorii maxime. Enunțul este adevărat dacă x este egal cu:

- a) 0 b) 10 c) 19 d) 20

2. Dacă lista inițială este $[7, 3, 9, 1]$, cum se schimbă după primul pas al sortării prin selecția minimului?

- a) $[3, 7, 9, 1]$ b) $[1, 3, 9, 7]$ c) $[1, 7, 9, 3]$ d) $[7, 3, 1, 9]$

3. Care este vectorul de frecvență pentru lista: $[2, 1, 3, 2, 1, 0, 2]$?

4. Având vectorul de frecvență $f = [1, 2, 0, 3]$, care este lista inițială sortată crescător?

5. Să se precizeze varianta care completează corect afirmația următoare:

Sortarea cu vectori de frecvență...

- a) este un algoritm eficient pentru valori foarte mari;
 b) este eficient pentru valori mici;
 c) folosește comparații între elemente;
 d) nu funcționează direct pentru valori reale arbitrare.

R E C A P I T U L A R E

3.5. Recapitulare

1. Se dă fragmentul de cod:

```
v = [5, 8, 2]
i = 1
print(v[i+1])
```

Ce valoare va fi afișată?

- a) 5 b) 1 c) 8 d) 2
2. După câte parcurgeri ale șirului [2, 40, 17, 1, 51, 34, 20, 63] valoarea 20 ajunge pe locul final în lista sortată crescător prin metoda selecție minimului?

- a) 1 b) 2 c) 3 d) 4
3. În varianta clasică a algoritmului de sortare prin selecție, la fiecare pas se caută:
- a) media elementelor; b) cel mai mare și cel mai mic simultan;
c) elementul minim din zona nesortată; d) primul element par.

4. Să se citească de la tastatură un număr natural n și o listă de n numere întregi. Să se afișeze elementele pozitive din listă.

5. Să se citească de la tastatură un număr natural n și o listă de n numere întregi. Să se înlocuiască fiecare element cu pătratul său.

6. Să se citească de la tastatură un număr natural n și o listă de n numere întregi. Să se determine suma elementelor prime din listă.

7. Se consideră șirul definit prin $x_0=1$ și relația $x_n=3^n$, n natural strict pozitiv. Să se elaboreze un program Python eficient care afișează primii n termeni ai șirului.

8. Se dă o listă cu n numere naturale (n este număr natural nenul). Să se afișeze în ordine crescătoare valorile prime din această listă.

Exemplu:

date.in	date.out
7	3 7 11 13
13 1 10 15 3 7 11	

- **Date de intrare:** Programul va citi de pe prima linie a fișierului `date.in` numărul n ($n \leq 1000$), iar de pe linia următoare cele n numere naturale (mai mici decât 1.000.000.000).
- **Date de ieșire:** Programul va scrie în fișierul `date.out` valorile prime din vector, în ordine crescătoare, separate prin exact un spațiu.



R E C A P I T U L A R E

V A L U A R E

MINIPROIECT – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVİ

Sortarea urgențelor în UPU

La intrarea în Unitățile de Primiri Urgențe (UPU), pacientul este evaluat și primește un cod ce indică nivelul de urgență și timpul maxim de așteptare până când va fi preluat de un medic.

Codurile de urgență în UPU

Cod Culoare	Nivel Urgență	Semnificație	Timp de așteptare
ROȘU	1 – Resuscitare	Funcții vitale compromise	0 min.
GALBEN	2 – Critic	Risc de afectare a unei funcții vitale	15 min.
VERDE	3 – Urgent	Funcții vitale stabile, necesită investigații	60 min.
ALBASTRU	4 – Non-urgent	Afecțiuni care nu pun viața în pericol	120 min.
ALB	5 – Consult	Cazuri rezolvabile la medicul de familie	240 min.

Cerință: În triajul UPU așteaptă n pacienți pentru care s-au stabilit codurile de urgență (numere de la 1 la 5). Să se afișeze ordinea în care vor fi preluați (la același nivel de urgență este preluat pacientul care a ajuns mai devreme – cu indicele mai mic).

Exemplu:	triaj.in	triaj.out	Explicații
	5 4 2 5 3 4	2 4 1 5 3	$n=5$ Indexarea în sala de așteptare este de la 1. Ordinea preluării pacienților va fi dată de ordinea crescătoare a codului de urgență. Pentru același cod se alege pacientul cu index mai mic.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E

3.6. Evaluare sumativă

V
A
L
U
A
R
E

I. Rezolvă.

1. Să se stabilească valoarea de adevăr a următoarelor enunțuri: 5x0,30 p. = 1,50 p.
- O listă în Python poate conține elemente de tipuri diferite.
 - Metoda `append()` adaugă un element la începutul listei.
 - Operatorul `in` verifică dacă un element aparține unei liste.
 - Operația `lista1 + lista2` realizează concatenarea a două liste.
 - Metoda `pop()` elimină și returnează un element din listă.

2. Se consideră lista: `L = [3, 7, 2, 9]` Care este rezultatul execuției următoarelor instrucțiuni?

```
L.append(5)
print(len(L))
```

- a) 4 b) 5 c) [3, 7, 2, 9, 5] d) Eroare de execuție
3. Se consideră secvența de program de mai jos: 2x0,75 p. = 1,50 p.

```
numere = [5, 2, 8, 1, 9]
minim = 0
for x in numere:
    if x < minim:
        minim = x
print("Minimul este:", minim)
```

- Să se identifice eroarea logică din program.
- Să se scrie varianta corectă a programului.

4. Se citesc de la tastatură un număr natural n și o listă de n numere naturale de maxim 3 cifre. Să se determine valoarea numărului care apare de cele mai multe ori în listă. 1 p.

5. Se consideră șirul definit prin $x_0=2$ și relația de recurență $x_n = x_{n-1} + 2^n$, $n > 0$. Să se scrie un program Python eficient care afișează primii n termeni ai șirului. 1 p.

- II. Miniproiect. Se va puncta rezolvarea proiectului din secțiunea de *Recapitulare*. 2 p.

- III. Portofoliu. La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate3_NumeElev*, următoarele rezolvări:
Fișa de lucru 8 și Fișa de lucru 9. 1x2 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*. Se acordă 1 punct din oficiu.

Recapitulare finală

PROIECT – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVI

Un profesor păstrează într-un fișier text numit `numere.txt` o listă de numere naturale nenule, câte un număr pe fiecare linie. Profesorul dorește să prelucreze aceste numere.

Să se scrie un program Python organizat modular care:

- să citească numerele din fișierul `numere.txt` și să le memoreze într-o listă `numere`;
- să definească următoarele subprograme (funcții):
 - ▶ `prim(n)` – returnează `True` dacă `n` este prim, `False` altfel;
 - ▶ `suma_cifre_pare(n)` – returnează suma cifrelor pare ale lui `n`;
 - ▶ `oglindit(n)` – returnează numărul obținut prin inversarea cifrelor lui `n`;
 - ▶ `cmmdc(a, b)` – returnează cel mai mare divizor comun, folosind Algoritmul lui Euclid cu împărțiri repetate;
- să construiască o listă `noua_lista`, în care, pentru fiecare număr `n` din fișierul `numere.txt`, se adaugă:
 - ▶ `oglindit(n)`, dacă `n` este prim, în caz contrar se adaugă `suma_cifre_pare(n)`;
- să calculeze CMMDC al tuturor valorilor nenule din `noua_lista`;
- să scrie în fișierul `rezultate.txt`, pe câte o linie:
 - elementele listei `noua_lista`;
 - valoarea CMMDC calculat la punctul d.

Barem de evaluare propus:

» Citire din fișier/scriere în fișier conform cerințelor.....	2 p.
» Corectitudinea funcțiilor de calcul.....	4 p.
» Construirea listelor.....	2 p.
» Determinare CMMDC	1 p.
» Din oficiu	1 p.

Evaluare finală

- Care este rezultatul apelului unei funcții care nu conține nicio instrucțiune `return` sau conține doar `return` fără a indica o expresie? 0,5 p.
 - Returnează șirul de caractere vid.
 - Produce eroare la execuția funcției.
 - Returnează valoarea specială `None`.
 - Returnează valoarea `0`.
- Care instrucțiune adaugă elementul `10` la sfârșitul listei `L`? 0,5 p.
 - `L.add(10)`
 - `L.insert(10)`
 - `L.append(10)`
 - `append(L,10)`
- Să se completeze corect spațiile libere. 2x0,5 p. = 1 p.
 - Cele cinci etape ale elaborării unui program sunt: analiza problemei,, implementarea, testarea și
 - Eroarea care apare la rulare când programul execută o operație invalidă (de exemplu, împărțirea la zero) se numește eroare de

4. Să se asocieze fiecărei secvențe de cod problema pe care o rezolvă cu structuri de date de tip listă.

4 x 0,5 p. = 2 p.

1)	<pre>for i in range(n - 1): for j in range(i + 1, n): if a[i] < a[j]: x = a[i] a[i] = a[j] a[j] = x</pre>
2)	<pre>m = a[0] for i in range(1, n): if a[i] < m: m = a[i]</pre>
3)	<pre>m = a[0] for i in range(1, n): if a[i] > m: m = a[i]</pre>
4)	<pre>ok = 0 for i in range(1, n): if a[i] != a[0]: ok = 1</pre>

- A. Verifică dacă lista **a** are toate elementele egale.
- B. Determină maximul din lista **a**.
- C. Sortare descrescătoare a listei **a**.
- D. Determină minimul din lista **a**.

5. Să se asocieze fiecărei expresii din coloana A rezultatul corect din coloana B

4x0,25 p. = 1 p.

Coloana A (expresie Python)	Coloana B (rezultat)
1) 25 // 4 + 25 % 4	A) 8
2) (15 - 3) * 2 ** 2 // 6	B) True
3) not (5 > 3) or (4 == 4)	C) 2
4) 100%9+ 100 // 9 % 5	D) 7

6. Să se stabilească valoarea de adevăr a fiecărei propoziții (**Adevărat** sau **Fals**).

4 x 0,25 p. = 1 p.

- a. Un algoritm trebuie să fie finit, corect, general și complet.
- b. Operatorul „=” în Python exprimă egalitatea matematică.
- c. Rezultatul expresiei `math.floor(-2.5)` este `-3`.
- d. Pentru a scrie valoarea întreagă `100` într-un fișier text este necesar să utilizăm funcția de conversie `str(100)`.

7. Să se citească de la tastatură un număr natural **n** alcătuit din cel puțin două cifre. Să se formeze o listă cu toate numerele obținute din **n** eliminând, pe rând, exact o cifră, astfel încât cifrele rămase să nu își modifice ordinea. Să se scrie un program în Python care afișează pe ecran cea mai mare și cea mai mică valoare din lista obținută. Pentru rezolvarea problemei se vor utiliza cel puțin trei funcții definite de utilizator.

3 p.

Exemplu: Dacă se citește `n=3849`, se obține lista `[384, 389, 349, 849]`; se afișează pe ecran `849` și `349`.

Din oficiu:..... 1 p.

Total punctaj:.....10 p.

Soluții (selectiv)

CAPITOLUL 1

Recapitulare: 1. F, F, A, A, A, F. 2. Programul calculează suma cifrelor pare ale numărului. Pentru `nr = 1234`, cifrele pare sunt 4 și 2, deci `suma = 4 + 2 = 6`.

Evaluare sumativă:

1. Pseudocod - O formă intermediară de reprezentare a unui algoritm, între limbajul natural și un limbaj de programare.

Constantă - O valoare care nu se modifică pe parcursul execuției programului.

Expresie logică - O combinație de valori și operatori care produce un rezultat de tip `True` sau `False`.

Variabilă - O zonă de memorie în care se poate stoca o valoare ce poate fi modificată în timpul execuției programului.

Proiectare modulară - Împărțirea unui algoritm în subprobleme independente, fiecare cu un rol clar.

2. a) `5.0 - float`; b) `3 - int`; c) `2 - int`; d) `True - bool`; e) `False - bool`;

CAPITOLUL 2

Autoevaluare: `f.read()`, `n+1`, `n%i==0`, `str(i)+" "`, `g.close()`

Evaluare sumativă: 1. b); 2. a-F, b-A, c-F, d-F, e-A; 3. a) buclă infinită.

CAPITOLUL 3

Evaluare sumativă: 1. a. A b. F c. A d. A e. A 2. b)

3. a) Eroarea este inițializarea greșită a variabilei `minim`

EVALUAREA FINALĂ:

1. c); 2. c); 3. a) proiectarea, depanarea b) execuție; 4. 1-C, 2-D, 3-B, 4-A; 5. 1-D, 2-A, 3-B, 4-C;

6. a) A, b) F, c) A, d) A

NOTĂ:

În limbajul de programare Python, se recomandă evitarea utilizării caracterelor cu diacritice în denumirile variabilelor, pentru a menține codul clar, lizibil și ușor de manipulat.

În schimb, în mesajele afișate utilizatorului, folosirea diacriticelor este permisă, dar nu obligatorie; în consecință, manualul prezintă exemple corespunzătoare atât cu, cât și fără diacritice.

Bibliografie

Tutorial Python în limba română:

<https://docs.python.org/ro/dev/tutorial/index.html>

Funcții predefinite *built-in*:

<https://docs.python.org/ro/dev/library/functions.html>

Modulul predefinit `math`:

<https://docs.python.org/ro/dev/library/math.html>

Vizualizarea pas cu pas a codului Python și a fluxului de execuție:

<https://pythontutor.com/visualize.html#mode=edit>

Link descărcare PyCharm:

<https://www.jetbrains.com/pycharm/download/>

CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

INFORMATICĂ

Curriculum de specialitate (CS), filiera teoretică,
profilul real, specializarea științe ale naturii – clasa a IX-a